VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF ELECTRICAL-ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS

LABORATORY REPORT

COMPUTER ARCHITECTURE (EE3043)

Class: L01 - Group: 24 - Semester: 251

Milestone 2

Design of a Single Cycle RISC-V Processor

Instructor: MS. Trần Hoàng Linh

TASK ASSIGNMENT TABLE

Full name	Student ID	Task	Grading
Võ Phan Tiến Đạt	2310716	Coding Simulation Report writing	100%
Lê Minh Tiến	2313427	Coding Simulation Report writing	100%

Date: 1/3/2025

Design of Pipelined RISC-V Processor

1. Introduction

Trong kiến trúc máy tính, việc cải thiện hiệu năng xử lý là một trong những mục tiêu quan trọng. Sau khi đã hoàn thành thiết kế bộ xử lý đơn chu kỳ (Single-cycle processor) ở Milestone 2, Milestone 3 sẽ nâng cấp kiến trúc lên mô hình xử lý đường ống (Pipelining). Đây là kỹ thuật cho phép thực thi song song nhiều lệnh ở các giai đoạn khác nhau, giúp tăng băng thông (throughput) của hệ thống.

Mục tiêu chính của báo cáo này là trình bày quá trình thiết kế, hiện thực và kiểm chứng các mô hình vi xử lý RV32I sử dụng kỹ thuật Pipelining. Cụ thể, báo cáo tập trung vào việc giải quyết các thách thức cốt lõi khi chia nhỏ vi xử lý thành 5 tầng (Fetch, Decode, Execute, Memory, Writeback), bao gồm:

- Xử lý xung đột (Hazards)
- Kỹ thuật Forwarding
- Dự đoán rẽ nhánh (Branch Prediction)

Bên cạnh việc thiết kế phần cứng, báo cáo cũng sẽ trình bày kết quả kiểm tra chức năng (Functional Verification) thông qua môi trường ISA Test và đánh giá hiệu năng thực tế sau khi tổng hợp (synthesis). Kết quả từ các mô hình khác nhau sẽ được so sánh để làm rõ ưu nhược điểm của từng kỹ thuật

2. Design Strategy

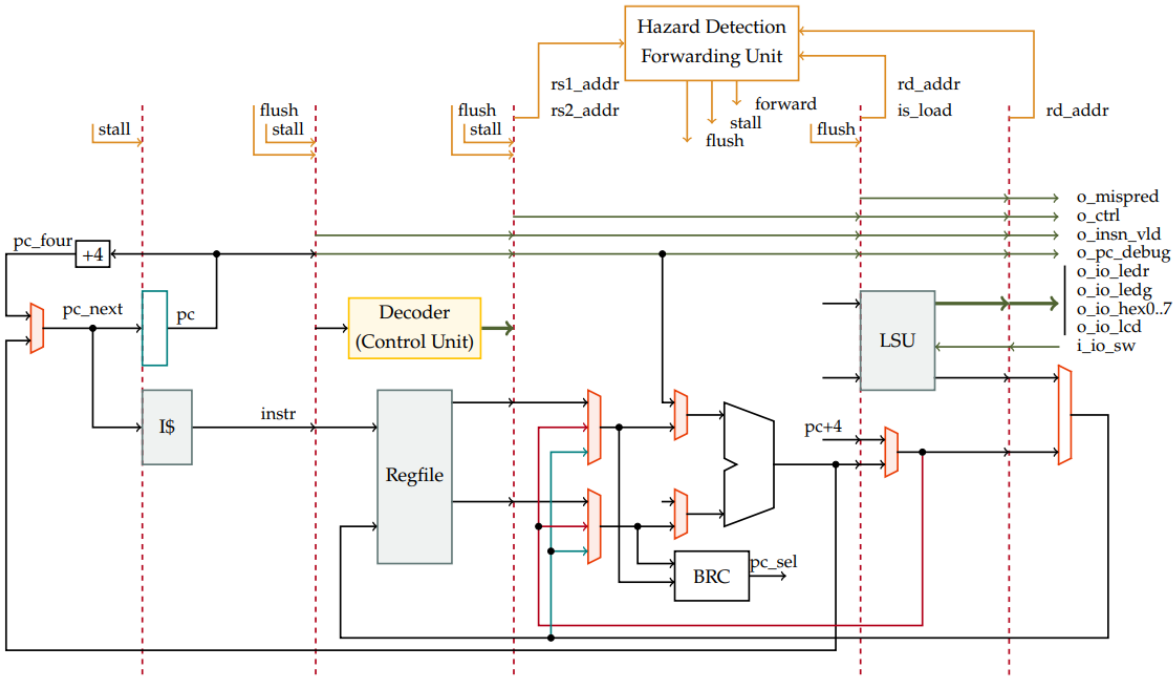
2.1. Overview

Nhóm sẽ tuân thủ thiết kế do môn học cung cấp trong file Milestone 3.

Vào tuần 1, nhóm sẽ tập trung vào việc tối ưu hóa xử lý theo kiểu đường ống và thiết kế bộ nhớ mới.

Vào tuần 2, nhóm sẽ chọn ba mô hình được giới thiệu trong file hướng dẫn để thực hiện, bao gồm mô hình: Non-Forwarding, Forwarding always not taken và Forwarding Always Taken.

Lưu ý có một số khối sẽ không thay đổi so với Milestone 2 đó là: Arithmetic Logic Unit (ALU) và Branch Comparison Unit (BRC)



Lưu ý quan trọng trong quá trình thiết kế: Không sử dụng các toán tử SystemVerilog như cộng/trừ (+/-), so sánh (<,>), dịch chuyển (<<,>>), nhân (*), chia (/), lấy dư (%) và các toán tử không thể synthesis khác trong thiết kế.

2.2. Top module: Pipelined theo mô hình forwarding kết hợp với always taken.

Đây là module Top-Level. Nó không chứa logic xử lý chi tiết, mà nhiệm vụ chính của nó là đấu nối dây giữa các tầng (stages) và các bộ điều khiển trung tâm. Module này đóng vai trò là bộ xử lý chính, nơi tất cả các linh kiện rời rạc (ALU, RegFile, Hazard Unit, Branch Unit...) được lắp ráp lại với nhau để tạo thành một bộ vi xử lý hoàn chỉnh

2.2.1. Design ideal

a. Kiến trúc đường ống 5 tầng

Ý tưởng cốt lõi của module này là chia nhỏ quá trình thực thi lệnh thành 5 giai đoạn độc lập, hoạt động song song để tăng thông lượng (throughput) xử lý.

- IF (Instruction Fetch): Nạp lệnh từ bộ nhớ Instruction Memory
- ID (Instruction Decode): Giải mã lệnh và đọc Register File
- EX (Execute): Tính toán ALU, tính địa chỉ rẽ nhánh
- MEM (Memory Access): Truy cập bộ nhớ dữ liệu (Data Memory)
- WB (Write Back): Ghi kết quả ngược lại vào Register File.

Để thực hiện ý tưởng này, file pipelined.sv sử dụng 4 bộ Thanh ghi đường ống (Pipeline Registers) nằm giữa các giai đoạn (IF_ID, ID_EX, EX_MEM, MEM_WB). Nhiệm vụ của chúng là lưu trữ trạng thái của lệnh hiện tại để đẩy sang giai đoạn tiếp theo ở chu kỳ xung nhịp kế tiếp.

b. Tổ chức Datapath (Đường dữ liệu)

Module pipelined.sv thực hiện việc truyền dữ liệu chạy suốt chiều dài bộ xử lý, nó sẽ kết nối bộ đếm PC và đường hồi tiếp quan trọng.

c. Tích hợp Control Path (Đường điều khiển)

Đây là nơi tích hợp các khối điều khiển mà chúng ta đã thiết kế trước đó:

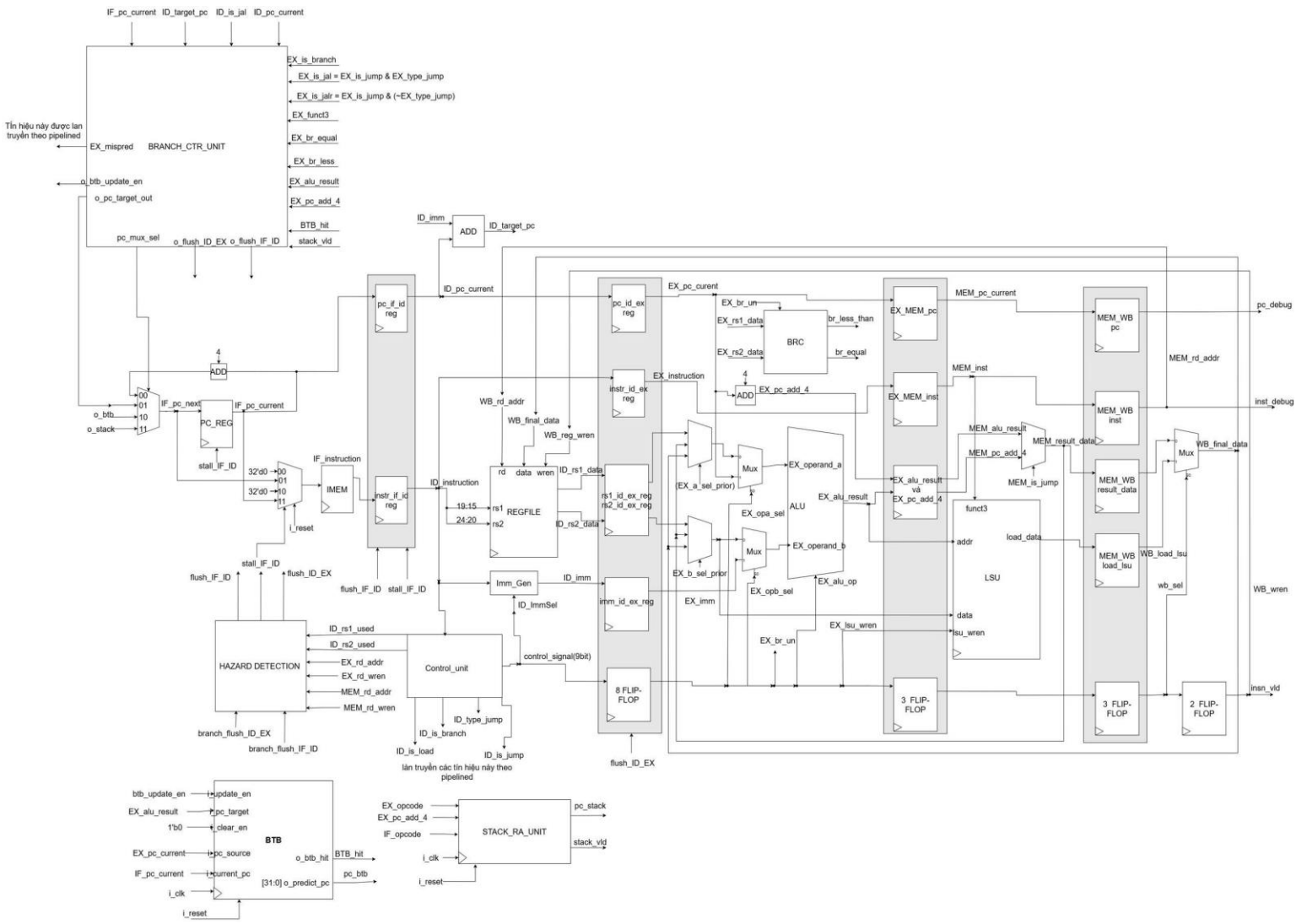
- Hazard Detection Unit: Được đặt ở vị trí giám sát tầng ID và EX
- Branch Control Unit: Được đặt song song với tầng EX
- Forwarding Control Unit: Giám sát tầng EX, MEM và WB.
- Control Unit: Tạo các tín hiệu điều khiển của từng lệnh tương ứng và lan truyền theo các giai đoạn của pipelined.

d. Giao tiếp Ngoại vi và Debug (I/O Interface)

Khác với các thiết kế lý thuyết, module pipelined.sv này được thiết kế thực tế để chạy trên kit FPGA (như DE10/DE2), thể hiện qua các cổng giao tiếp:

- Input: i_io_sw (Công tắc) - Dùng để nhập dữ liệu trực tiếp
- Output: o_io_ledr, o_io_hex (Đèn LED, LED 7 đoạn)
- Output thống kê: o_insn_vld, o_mispred - Các tín hiệu này giúp đo đếm hiệu năng (số lệnh thực thi, số lần dự đoán sai)

Sơ đồ tổng thể của khối pipelined.sv sẽ có dạng sau:



Tại module này, nhóm tụi em bổ sung thêm một bộ cộng tại pha ID, mục đích là để thực hiện lệnh Jal nhanh hơn chỉ bằng cách cộng pc với imm, tất nhiên sẽ cần các logic điều khiển cho trường hợp này, được tích hợp tổng quát trong module Branch Control Unit sẽ được xây dựng ở các phần sau. Lý do chỉ áp dụng cho Jal: Jal tính toán nhanh từ pc và imm mà không cần thông qua thanh ghi, việc bổ sung bộ cộng sẽ không làm tăng đáng kể thời gian delay của pha ID do hầu như nó xử lý song song so với các khác khối trong cùng pha ID, nhóm em sẽ triển khai các giải thuật cộng nhanh chống hơn như CLA và CSA.

2.2.2. Specification

- Top-level module: pipelined.sv
- I/O ports:

Signal name	Width	Direction	Description
i_clk	1	input	Global clock, active on the rising edge
i_reset	1	input	Global low active reset
o_pc_debug	32	output	Debug program counter
o_insn_vld	1	output	Instruction valid
o_ctrl	1	output	Control transfer instructions (branch or jump)
o_mispred	1	output	Mispredict
o_io_ledr	32	output	Output for driving red LEDs
o_io_ledg	32	output	Output for driving green LEDs
o_io_hex0…7	7	output	Output for driving 7-segment LED displays
o_io_lcd	32	output	Output for driving the LCD register
i_io_sw	32	input	Input for switches

2.3. Forwarding Control Unit

Trong kiến trúc Pipeline, một lệnh ở tầng thực thi (EX) có thể cần sử dụng kết quả của các lệnh đi trước nó. Tuy nhiên, các lệnh đi trước có thể chưa kịp ghi kết quả về thanh ghi (Register File - ở tầng WB).

Nếu không xử lý: Lệnh ở EX sẽ đọc phải dữ liệu cũ (stale data) dẫn đến kết quả sai.

2.3.1. Design ideal

Khối Forwarding Control Unit chỉ đơn giản là điều khiển 2 con MUX 4-1 để lấy dữ liệu forwarding về từ tầng MEM Và WB

Nguyên lý hoạt động:

a. **Kiểm tra điều kiện Forwarding (Validation)** Forwarding chỉ xảy ra khi lệnh đi trước thực sự có ghi dữ liệu, phải thỏa mãn 3 điều kiện sau:

- i_reg_we_MEM (hoặc WB) = 1: Lệnh đó có chức năng ghi thanh ghi

- i_rd_MEM (hoặc WB) != x0: Thanh ghi đích không phải là x0 (vì x0 luôn bằng 0, không cần forward)
- i_rs_EX == i_rd_MEM (hoặc WB): Địa chỉ thanh ghi nguồn cần dùng trùng với địa chỉ đích đang được ghi

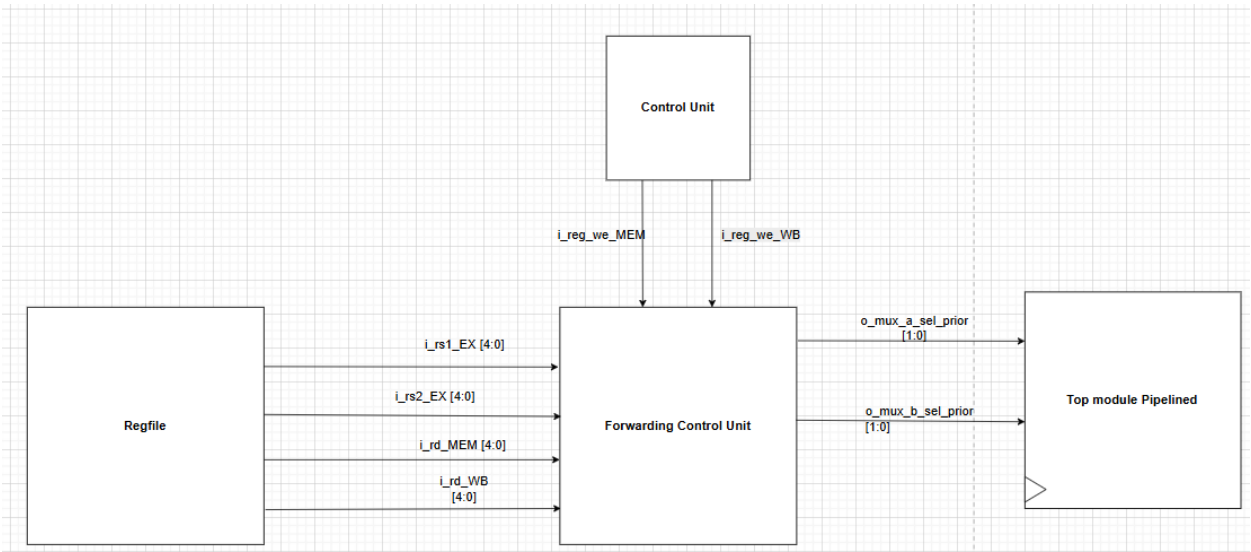
b. Cách xử lý

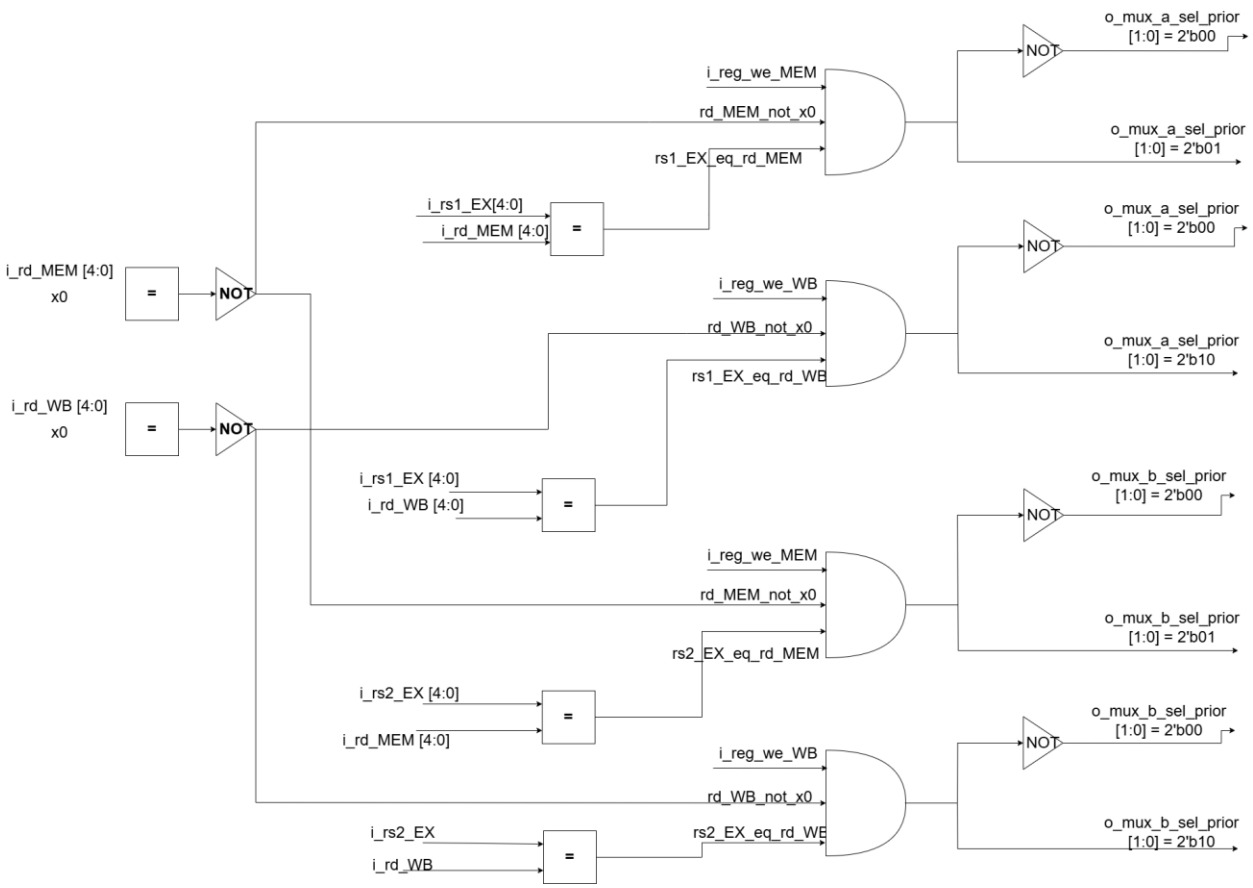
Việc đặt điều kiện kiểm tra MEM lên trước và WB ở sau đảm bảo rằng nếu cả hai tầng đều chứa dữ liệu cần thiết, bộ xử lý sẽ luôn chọn dữ liệu từ tầng MEM (mới nhất), đảm bảo tính đúng đắn của chương trình

Các tín hiệu o_mux_a_sel_prior và o_mux_b_sel_prior sẽ điều khiển hai bộ MUX 4-1 đặt trước ALU:

- 00: No Forwarding. Dữ liệu lấy từ thanh ghi rs (đọc ở tầng ID)
- 01: Forward from MEM. Dữ liệu lấy từ ngõ ra ALU của lệnh trước đó (đang nằm ở pipeline register EX/MEM). Giúp giải quyết hazard với độ trễ 1 chu kỳ.
- 10: Forward from WB. Dữ liệu lấy từ kết quả ghi lại (đang nằm ở pipeline register MEM/WB hoặc ngõ ra bộ nhớ). Giúp giải quyết hazard với độ trễ 2 chu kỳ

Sơ đồ khối của bộ Forwarding Control Unit có dạng sau:





2.3.2. Specification

2.4. Top-level module: forwarding_ctrl_unit.sv

- I/O ports:

Signal name	Width	**Direction**	**Description**
i_rs1_EX	5	input	Địa chỉ thanh ghi nguồn 1 của lệnh đang ở EX
i_rs2_EX	5	input	Địa chỉ thanh ghi nguồn 2 của lệnh đang ở EX
i_rd_MEM	5	input	Địa chỉ thanh ghi đích đang được ghi ở MEM
i_reg_we_MEM	1	input	Cờ cho phép ghi RegFile tại tầng MEM
i_rd_WB	5	input	Địa chỉ thanh ghi đích đang được ghi ở WB
i_reg_we_WB	1	input	Cờ cho phép ghi RegFile tại tầng WB

o_mux_a_sel_prior	2	output	Điều khiển MUX A
o_mux_b_sel_prior	2	output	Điều khiển MUX B

2.5. Hazard Detection Unit

Khối Hazard Detection Unit có nhiệm vụ chính là phát hiện các xung đột mà phần cứng forwarding không thể giải quyết được, từ đó đưa ra quyết định Stall (dừng pipeline) hoặc Flush (xóa lệnh) để đảm bảo tính đúng đắn của dữ liệu và luồng chương trình.

2.5.1. Design ideal

Khối này sẽ xử lý hai loại xung đột chính:

- Data Hazard (Load-Use Hazard): Khi lệnh phía sau cần dữ liệu từ lệnh Load phía trước mà dữ liệu chưa kịp về
- Control Hazard (Branch/Jump): Khi chương trình thay đổi do lệnh nhảy, cần loại bỏ các lệnh đã sai

Lưu ý rằng các tín hiệu i_branch_flush_ID_EX và i_branch_flush_IF_ID sẽ được gửi về từ khối Branch control unit, nguyên lý hoạt động như sau:

a. Stall

Điều kiện để stall là có Load use Hazard và không có Branch Taken, bởi vì nếu đang Branch/Jump thì lệnh ở ID sắp bị xóa, không cần stall nữa mà flush luôn

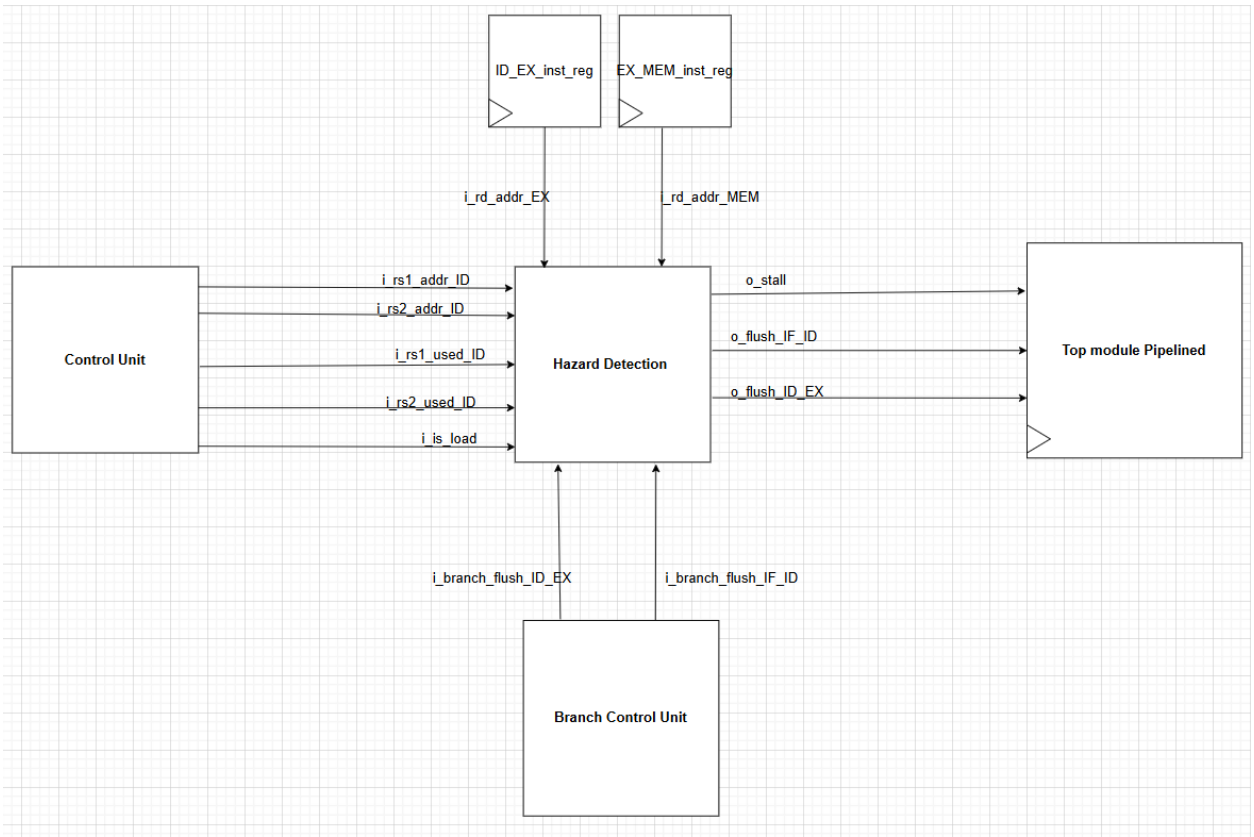
b. Flush ID/EX

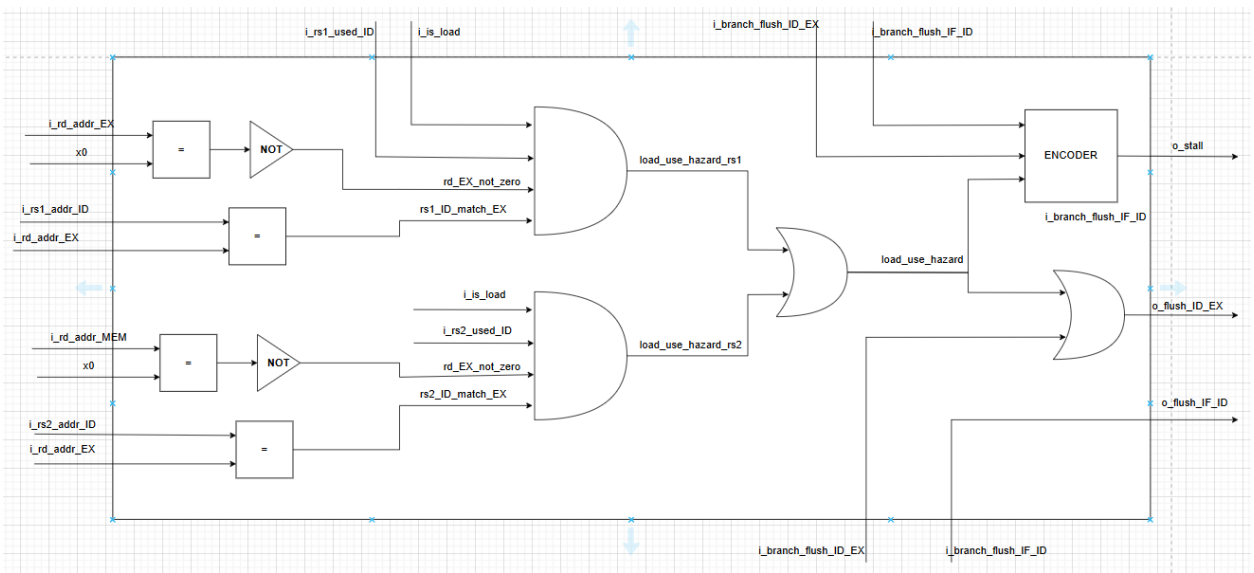
Điều kiện để Flush ID/EX là khi có Load-Use Hazard để chèn NOP vào EX không chạy lệnh rác và khi có Branch Taken để xóa lệnh nhầm đã vào ID/EX

c. Flush ID/EX

Điều kiện để Flush ID/EX là khi có Branch Taken

Sơ đồ khối của khối Hazard Detection Unit sẽ có dạng sau:





2.5.2. Specification

- Top-level module: hazard_detection_unit.sv
- I/O ports:

Signal name	Width	Direction	Description
i_rs1_addr_ID	5	input	Địa chỉ thanh ghi nguồn rs1 của lệnh đang ở ID
i_rs2_addr_ID	5	input	Địa chỉ thanh ghi nguồn rs2 của lệnh đang ở ID
i_rs1_used_ID	1	input	Cờ báo lệnh này có dùng rs1 không
i_rs2_used_ID	1	input	Cờ báo lệnh này có dùng rs2 không
i_rd_addr_EX	5	input	Địa chỉ thanh ghi đích (rd) của lệnh đang ở EX
i_rd_addr_MEM	5	input	Địa chỉ thanh ghi đích của lệnh đang ở MEM
i_is_load	1	input	Tín hiệu báo lệnh ở EX là lệnh load (LW, LH…)
i_branch_flush_ID_EX	1	input	Tín hiệu yêu cầu flush tầng ID/EX
i_branch_flush_IF_ID	1	input	Tín hiệu yêu cầu flush tầng IF/ID
o_stall	1	output	Dừng thanh ghi PC và IF/ID
o_flush_IF_ID	1	output	Xóa lệnh ở tầng IF/ID
o_flush_ID_EX	1	output	Xóa lệnh ở tầng ID/EX

2.6. Branch Control Unit

Khối Branch Control Unit hoạt động như bộ xử lý các lệnh rẽ nhánh và điều khiển thanh ghi PC (Program Counter). Nhiệm vụ cốt lõi của nó là đảm bảo CPU luôn fetch đúng lệnh tiếp theo, bất kể là do dự đoán hay do sửa lỗi

Nói tóm lại, Branch control unit sẽ phát hiện, xử lý các hazard liên quan đến lệnh branch, jump. Sau đó nó sẽ gửi về các tín hiệu sau:

- Tín hiệu o_pc_mux_sel để lựa chọn ngõ ra cho MUX trước PC
- Tín hiệu o_btb_update_en để cho phép cập nhật địa chỉ đúng vào BTB
- Các tín hiệu flush IF/ID và flush ID/EX để gửi về hazard dectection unit, khối hazard dectection unit sẽ kết hợp các tín hiệu này với load use hazard để quyết định xem có flush trong top module pipelined không

2.6.1. Design ideal

Khối Branch Control Unit hoạt động theo cơ chế Phân cấp ưu tiên:

- Mức 1 (Cao nhất): Sửa lỗi tại tầng EX
- Mức 2: Tối ưu hóa lệnh JAL tại tầng ID
- Mức 3: Dự đoán hướng đi tại tầng IF (RAS & BTB).
- Mức 4 (Mặc định): Tuần tự (PC + 4).

Chi tiết Nguyên lý Hoạt động

a. Xử lý tại tầng EX (Execute Stage)

Đây là nơi kết quả thực tế của lệnh nhánh được xác định (thông qua cờ từ ALU).

Bước 1: Xác định thực tế: Dựa vào EX_funct3 và các cờ EX_br_equal, EX_br_less, khối này tính toán tín hiệu EX_taken (Có nhảy hay không?)

Bước 2: Phát hiện dự đoán sai (Misprediction Detection): Khối này so sánh địa chỉ đúng vừa tính được (EX_correct_next_pc) với địa chỉ lệnh đang nằm chờ ở tầng ID (ID_pc)

Bước 3: Hành động sửa sai: Nếu EX_mispredict . = 1:

- Điều khiển PC: o_pc_mux_sel = 00
- Xả đường ống (Flush) xóa sạch 2 lệnh sai đang nằm trong pipeline
- Cập nhật BTB: Gửi tín hiệu o_btb_update_en hoặc o_btb_clear_en để cập nhật lại bộ dự đoán cho lần sau.

b. Xử lý tại tầng ID (Decode Stage) - Tối ưu hóa JAL

Lệnh nhảy không điều kiện (JAL) có thể xác định địa chỉ đích ngay sau khi giải mã (ID), không cần đợi đến EX.

- Điều kiện: ID_is_jal = 1
- Phát hiện sai: So sánh đích đến của JAL (ID_target_pc) với lệnh đang fetch ở IF (IF_pc)

- Hành động: Nếu phát hiện sai (JAL chưa được BTB dự đoán đúng): o_pc_mux_sel = 01 (Lấy PC từ kết quả tính tại ID)
- Flush: Chỉ cần o_flush_IF_ID = 1 (Xóa lệnh rác đang fetch). Lệnh JAL ở ID vẫn đúng nên giữ lại (o_flush_ID_EX = 0)

c. Dự đoán tại tầng IF (Fetch Stage)

Nếu không có xung đột hay sửa lỗi từ EX/ID, khối này sẽ dựa vào bộ dự đoán để chọn PC tiếp theo:

RAS (Return Address Stack): Dành cho lệnh JALR (trả về từ hàm). Nếu stack_valid = 1 (Stack không rỗng và đoán là lệnh RET), ưu tiên lấy địa chỉ từ Stack

BTB (Branch Target Buffer): Nếu BTB_hit = 1 (Tìm thấy trong bộ nhớ BTB), lấy địa chỉ từ BTB.

d. Mặc định (Default)

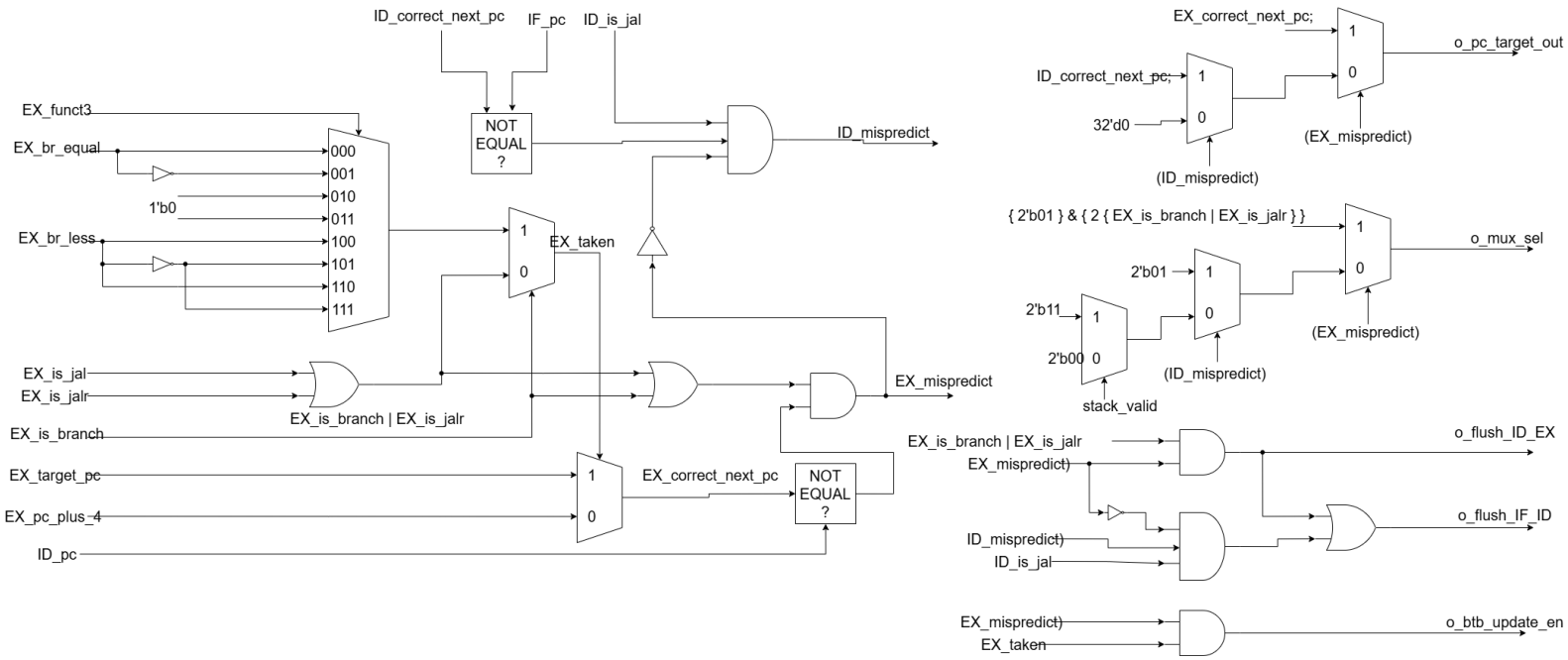
Nếu không có sự kiện nào ở trên xảy ra: o_pc_mux_sel = 11 (Chọn PC + 4).

e. Quản lý Stack (RAS Control)

Khối này cũng điều khiển tín hiệu cho RAS (Return Address Stack) để hỗ trợ gọi hàm:

- Push: Khi gặp lệnh gọi hàm (JAL/JALR với rd=x1 hoặc rd=x5)
- Pop: Khi gặp lệnh trả về (JALR với rs1=x1 hoặc rs1=x5 và rd=x0)

Sơ đồ khối của Branch Control Unit có dạng như sau:



2.6.2. Specification

- Top-level module: branch_control_unit.sv
- I/O ports:

Signal name	Width	**Direction**	**Description**
EX_is_branch	1	input	Lệnh tại EX là Branch (BEQ, BNE...)
EX_is_jal	1	input	Lệnh tại EX là JAL
EX_is_jalr	1	input	Lệnh tại EX là JALR
EX_funct3	3	input	Funct3 để phân biệt loại Branch
EX_br_equal	1	input	Flag từ BRC: rs1 == rs2
EX_br_less	1	input	Flag từ BRC: rs1 < rs2
EX_target_pc	32	input	Địa chỉ đích tính từ ALU
EX_pc_plus_4	32	input	PC+4 của lệnh tại EX
ID_pc	32	input	PC của lệnh đang ở ID
ID_is_jal	1	input	Phát hiện JAL tại ID
ID_target_pc	32	input	Địa chỉ đích của JAL (tính sớm)
IF_pc	32	input	PC của lệnh đang Fetch

BTB_hit	1	input	BTB dự đoán có lệnh nhảy tại địa chỉ này
stack_valid	1	input	RAS (Return Address Stack) dự đoán cho RET
o_pc_mux_sel	2	output	Chọn nguồn PC tiếp theo
o_pc_target_out	32	output	Địa chỉ PC cần nạp
o_flush_ID_EX	1	output	Xóa lệnh ở IF/ID
o_flush_IF_ID	1	output	Xóa lệnh ở ID/EX
o_btb_update_en	1	output	Cho phép ghi vào BTB
o_btb_target_data	32	output	Địa chỉ đích cần ghi
o_mispred	1	output	Cờ báo có misprediction

2.7. Branch Target Buffer (BTB)

2.7.1. Design ideal

Khối BTB giống như một bộ nhớ, được thiết kế nhằm dự đoán địa chỉ nhảy của các lệnh rẽ nhánh ngay tại tầng Fetch (IF).

Hoạt động của khối BTB được chia thành hai pha chính: Pha Đọc (tại tầng IF) và Pha Ghi/Cập nhật (từ tầng EX)

a. Logic đọc (Prediction Logic - Tầng IF)

Tại mỗi chu kỳ xung nhịp, khi PC hiện tại (i_current_pc) được đưa vào, BTB thực hiện các bước sau để xác định xem có dự đoán nhảy hay không:

Tách địa chỉ: read_index (8 bits) dùng để truy xuất trực tiếp vào mảng nhớ và read_tag (22 bits) dùng để so sánh định danh.

Kiểm tra Hit/Miss: So sánh: Nếu valid_mem[read_index] = 1 và tag_mem[read_index] = read_tag thì được kết quả:

- o_btb_hit = 1: Lệnh hiện tại là lệnh nhảy đã biết. Ngõ ra o_predict_pc lấy giá trị từ target_mem
- o_btb_hit = 0: Không tìm thấy hoặc dữ liệu không hợp lệ. Ngõ ra mặc định là 0 (CPU sẽ chọn PC+4).

b. Logic ghi và cập nhật (Update Logic - Tầng EX)

Việc cập nhật BTB được điều khiển bởi khối Branch Control Unit từ tầng Execute (EX) thông qua các tín hiệu i_update_en và i_clear_en. Địa chỉ dùng để ghi là i_pc_source (PC của lệnh nhánh tại EX).

Trường hợp 1: Cập nhật Entry mới (update)

- Điều kiện: Khi có một lệnh nhánh được thực thi và kết quả là TAKEN (nhảy), tín hiệu i_update_en được bật.
- Hành động: Tại dòng write_index (tính từ i_pc_source), BTB sẽ đặt valid_mem lên 1, ghi write_tag vào tag_mem và ghi địa chỉ đích thực tế (i_pc_target) vào target_mem

Trường hợp 2: Xóa Entry sai

- Điều kiện: Khi phát hiện dự đoán sai (Misprediction), ví dụ: BTB dự đoán nhảy nhưng thực tế không nhảy, hoặc dự đoán sai địa chỉ đích. Tín hiệu i_clear_en được bật
- Hành động: Đặt valid_mem tại dòng tương ứng về 0 để vô hiệu hóa entry đó

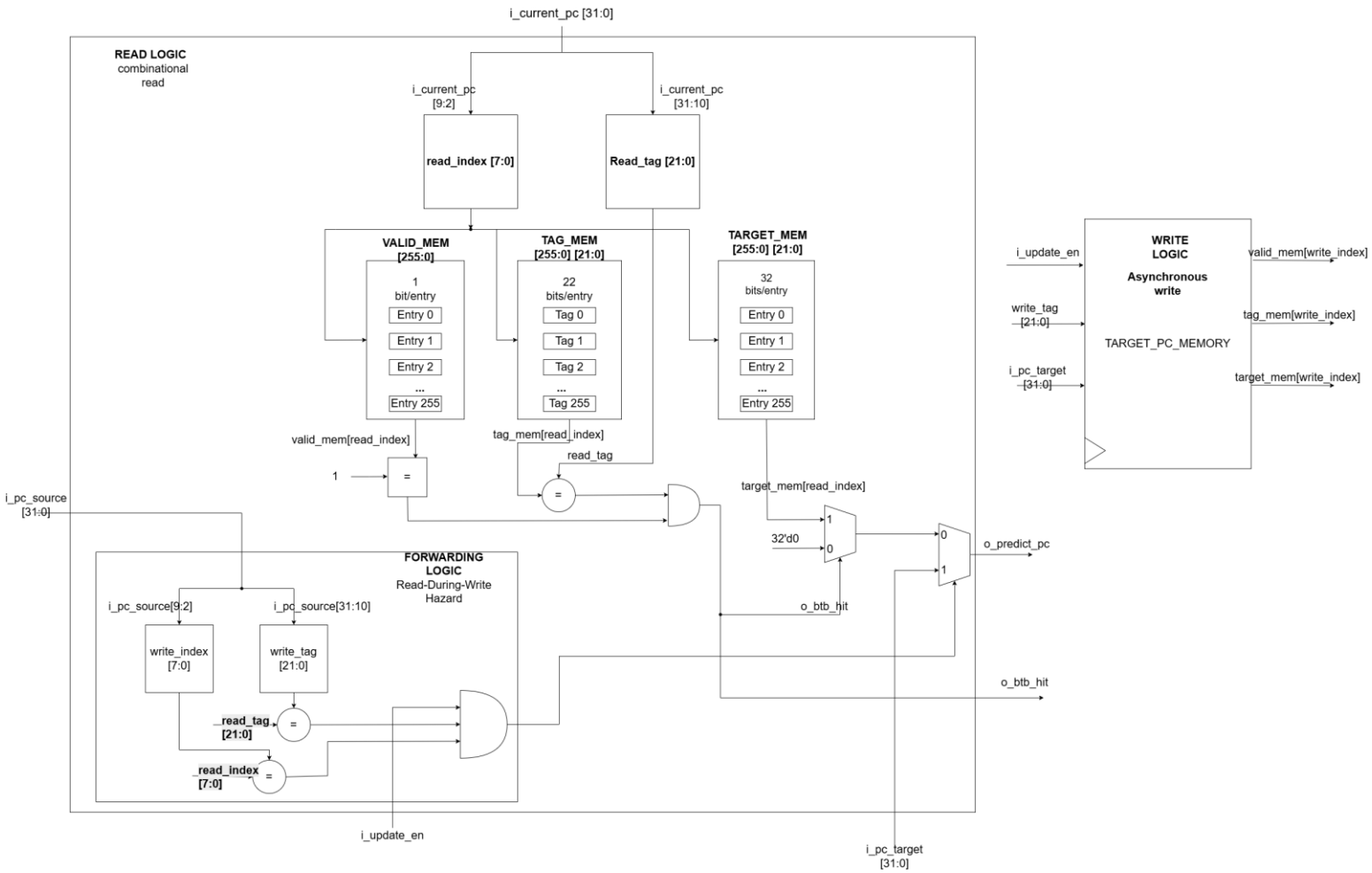
c. Ngoài ra khối BTB còn xử lý xung đột (Forwarding/Hazard Handling)

Khối BTB này có thể xử lý Read-During-Write Hazard (Vừa đọc vừa ghi cùng lúc)

Vấn đề: Trong cùng 1 chu kỳ, nếu tầng EX đang cập nhật lại địa chỉ đích cho một lệnh nhánh (ví dụ lệnh Loop), và tầng IF cũng đang fetch đúng lệnh đó (dựa trên index), BTB có thể đọc ra dữ liệu cũ chưa kịp cập nhật

Giải pháp: Module BTB.sv có logic forwarding nội bộ: Nếu phát hiện đang ghi (i_update_en) vào đúng dòng đang đọc (write_index == read_index & tag match), ngõ ra o_predict_pc sẽ lấy trực tiếp từ i_pc_target (giá trị mới nhất) thay vì lấy từ bộ nhớ target_mem

Sơ đồ khối của khối BTB sẽ có dạng như sau:



2.7.2. Specification

- Top-level module: BTB.sv
- I/O ports:

Signal name	Width	**Direction**	**Description**
i_clk	1	input	Xung clock toàn cục
i_rstn	1	input	Chân reset tích cực thấp
i_current_pc	32	input	Địa chỉ pc đang ở tầng IF
o_predict_pc	32	output	Địa chỉ đích mà BTB dự đoán
O_btb_hit	1	output	Tín hiệu báo lệnh pc có trong BTB không
i_update_en	1	output	Tín hiệu báo khi lệnh là Branch hoặc Taken

I_pc_source	32	input	Địa chỉ pc của lệnh Branch đã tới EX
I_pc_target	32	output	Địa chỉ đích thật sự đã tính xong

2.8. DMEM:

Bộ nhớ DMEM được sử dụng để lưu trữ data (các giá trị, các kết quả tính toán,…).

2.8.1. Specification

Module name: dmem.sv

Tên Tín hiệu	Loại	Độ rộng	Mô tả
i_clk	Input	1 bit	Xung clock
i_access_en	Input	1 bit	Cho phép Truy cập (Chip Enable).- 1: Module hoạt động (Đọc hoặc Ghi).- 0: Module bị vô hiệu hóa, không ghi dữ liệu, đầu ra đọc trả về 0.
i_wren	Input	1 bit	Cho phép Ghi (Write Enable).- 1: Thực hiện thao tác Ghi (Write).- 0: Thực hiện thao tác Đọc (Read).
i_bmask	Input	8 bits	Byte Mask (Byte Enable). Kiểm soát việc đọc/ghi từng byte riêng lẻ cho cả hai cổng địa chỉ:- Bits [3:0]: Tương ứng với 4 byte tại i_addr.- Bits [7:4]: Tương ứng với 4 byte tại i_next_addr. Nếu bit là 1: Byte đó được ghi hoặc đọc. Nếu 0: Byte đó bị bỏ qua(đối với ghi) hoặc trả về 0 (đối với đọc).
i_addr	Input	9 bits	Địa chỉ truy cập chính. Trỏ đến từ nhớ (word) 32-bit cần truy cập.
i_wdata	Input	32 bits	Dữ liệu ghi cho cổng 1. Dữ liệu này sẽ được ghi vào địa chỉ i_addr (chỉ các byte được bật bởi i_bmask[3:0]).
o_rdata	Output	32 bits	Dữ liệu đọc từ cổng 1. Giá trị tại địa chỉ i_addr. Các byte không được chọn bởi i_bmask[3:0] sẽ trả về 0.
i_next_addr	Input	9 bits	Địa chỉ truy cập phụ (thường là i_addr + 1). Dùng để truy cập phần dữ liệu tràn sang word nhớ tiếp theo trong trường hợp xử lý dữ liệu misaligned.
i_wdata_next_addr	Input	32 bits	Dữ liệu này sẽ được ghi vào địa chỉ i_next_addr (chỉ các byte được bật bởi i_bmask[7:4]).
o_rdata_next_addr	Output	32 bits	Giá trị tại địa chỉ i_next_addr. Các byte không được chọn bởi i_bmask[7:4] sẽ trả về 0.

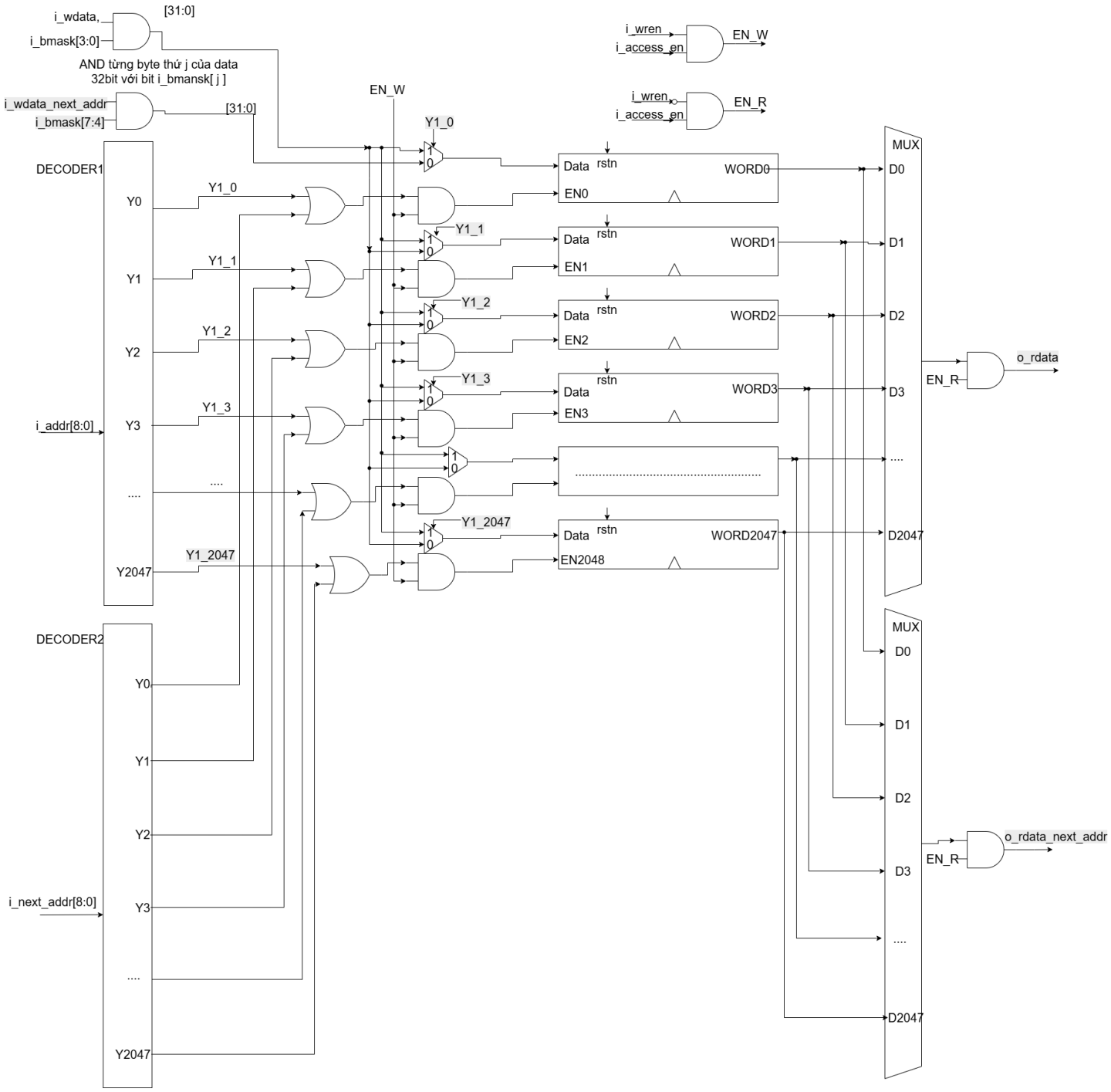
Để thuận tiện cho việc ánh xạ DMEM trên Quartus, ta sẽ không sử dụng tín hiệu i_rst có nhiệm vụ reset các word nhớ trên module này.

Một điều cần lưu ý: Dữ liệu đọc sẽ được đồng bộ theo clock.

2.8.2. Thiết kế DMEM:

Ý tưởng của DMEM: Thiết kế bộ nhớ xử lý vấn đề misaligned. Sử dụng hai bộ decoder với

hai đường địa chỉ là i_addr và i_next_addr. Điều này giúp truy cập đồng thời hai vị trí trong bộ nhớ. Để xử lý misaligned, i_next_addr = i_addr + 1, ta cần thêm một bộ cộng (nó sẽ được đặt trong module LSU sẽ phân tích sau). Các ngõ EN của các word được bật khi EN_W = 1 (tức i_wren = 1, cho phép ghi và i_access_en = 1, ngõ i_access_en là tín hiệu từ lsu để lựa chọn vùng nhớ được truy cập, sẽ được nói rõ hơn ở LSU) và phải có tín hiệu ngõ ra có vị trí tương tướng với vị trí của word được truy cập từ một trong hai bộ decoder. Do có hai ngõ vào data ghi là i_wdata và i_wdata_next_addr, nên một word có thể nhận một trong hai ngõ, ta cần đặt tại đầu vào word một mux2-1 để lựa chọn. Tín hiệu lựa chọn sẽ là các tín hiệu ngõ ra Y1_0, Y1_2, Y1_3,… Y1_2047 từ bộ decoder 1 tương ứng với ngõ vào i_addr. Tức là khi i_addr truy cập một ô word, ô word đó sẽ được ghi data từ i_wdata, ngược lại ta sẽ mặc định ghi i_wdata_next_addr. Do việc truy cập bộ nhớ không chỉ dừng ở việc truy cập word mà còn truy cập byte và half-word, ta cần các bit i_bmask[7:0] có nhiệm vụ lọc data nào được ghi vào word, i_bmask[3:0] lọc i_wdata, i_bmask[7:4] lọc i_wdata_next_addr. Bên cạnh việc ghi, ta cần xây dựng cấu hình đọc cho DMEM. Với cấu hình đọc đồng bộ, dữ liệu liên tục được load tại các ngõ ra của các word nếu có tín hiệu EN, ngõ EN_R = 1 tức cho phép đọc khi i_wren = 0 và i_access_en = 1. Bit này sẽ được AND với ngõ ra bộ của 2 MUX dùng để lựa chọn data đọc từ word với tín hiệu lựa chọn là i_addr (cho MUX1) và và i_addr_next_addr (cho MUX2). Ta sẽ thực hiện lọc dữ liệu đọc được từ DMEM thông qua các bit bmask ngay trong module LSU (phần sau).



2.9. LSU

2.9.1. Specification

Module name: lsu.sv

Tên Cổng	Loại	Độ rộng (Width)	Mô tả

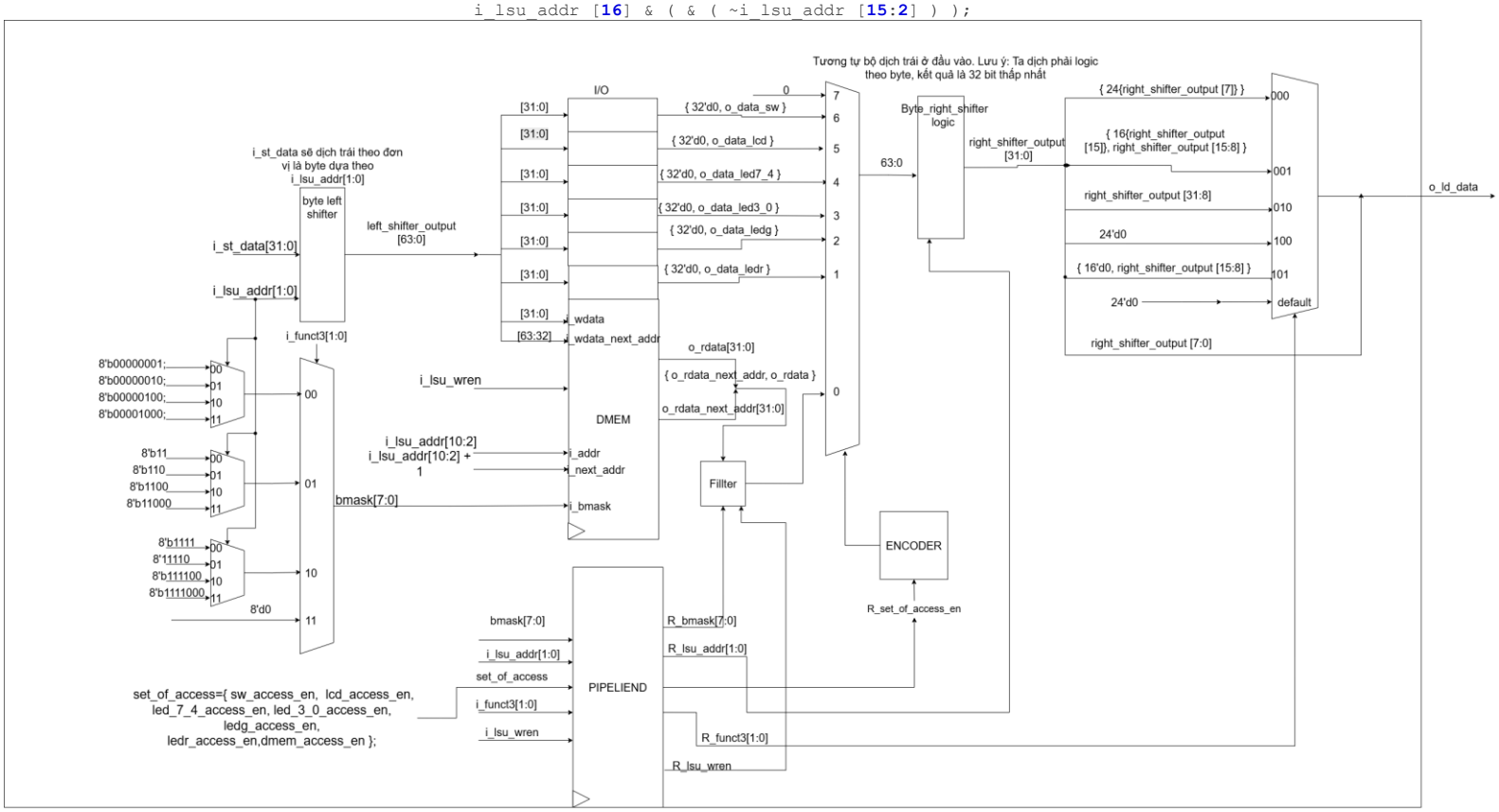
i_clk	Input	1 bit	Xung nhịp hệ thống (Clock).
i_reset	Input	1 bit	Tín hiệu Reset hệ thống.
i_lsu_addr	Input	32 bit	Địa chỉ truy cập bộ nhớ/ngoại vi. Được sử dụng để giải mã địa chỉ (dmem, led, lcd, v.v.) và tính toán byte shift.
i_st_data	Input	32 bit	Dữ liệu cần ghi (Store Data) vào bộ nhớ hoặc ngoại vi.
i_lsu_wren	Input	1 bit	Tín hiệu cho phép ghi (Write Enable). - 1: Ghi (Store). - 0: Đọc (Load).
funct3	Input	3 bit	Mã lệnh điều khiển kiểu truy cập (Byte/Half/Word) và kiểu mở rộng dấu (Signed/Unsigned). - [1:0]: 00 (Byte), 01 (Half), 10 (Word). - [2]: 1 (Unsigned), 0 (Signed).
i_io_sw	Input	32 bit	Dữ liệu đầu vào từ các Switches ngoại vi.
o_ld_data	Output	32 bit	Dữ liệu đọc ra (Load Data) từ bộ nhớ hoặc ngoại vi, đã được xử lý dịch bit và mở rộng dấu.
o_io_ledr	Output	32 bit	Dữ liệu điều khiển cho LED Đỏ (Red LEDs).
o_io_ledg	Output	32 bit	Dữ liệu điều khiển cho LED Xanh (Green LEDs).
o_io_hex0	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX0).
o_io_hex1	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX1).
o_io_hex2	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX2).
o_io_hex3	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX3).
o_io_hex4	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX4).
o_io_hex5	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX5).
o_io_hex6	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX6).
o_io_hex7	Output	7 bit	Dữ liệu điều khiển cho LED 7 đoạn (HEX7).
o_io_lcd	Output	32 bit	Dữ liệu điều khiển cho màn hình LCD.

2.9.2. Ý tưởng thiết kế:

Đầu tiên, ta tạo các tín hiệu access là các minterm tổ hợp bởi các bit của i_lsu_addr[31:0]. Các tín hiệu access của các vùng I/O và vùng nhớ (DMEM). Các tín hiệu này cho phép truy cập với vùng I/O hay DMEM tương ứng:

```
//// enable access ////
logic dmem_access_en, ledr_access_en, ledg_access_en,
      led_3_0_access_en, led_7_4_access_en, lcd_access_en,  //// enable accessc for each memory domain ////
                         sw_access_en;
assign dmem_access_en    = & ( ~i_lsu_addr [31:11] );
assign ledr_access_en    = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:2] ) );
assign ledg_access_en    = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:13] ) ) &
                           i_lsu_addr [12] & ( & ( ~i_lsu_addr [11:2] ) );
assign led_3_0_access_en = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:14] ) ) &
                           i_lsu_addr [13] & ( & ( ~i_lsu_addr [12:2] ) );
assign led_7_4_access_en = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:14] ) ) &
                           i_lsu_addr [13] & i_lsu_addr [12] & ( & ( ~i_lsu_addr [11:2] ) );
```

```
assign lcd_access_en      = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:15] ) ) &
                            i_lsu_addr [14] & ( & ( ~i_lsu_addr [13:2] ) );
assign sw_access_en       = ( & ( ~i_lsu_addr [31:29] ) ) & i_lsu_addr [28] & ( & ( ~i_lsu_addr [27:17] ) ) &
                            i_lsu_addr [16] & ( & ( ~i_lsu_addr [15:2] ) );
```



Do yêu cầu từ bộ nhớ DMEM load đồng bộ, ta không thể nào xây dựng LSU có cấu trúc đọc dữ liệu như một hệ tổ hợp từ đầu đến cuối, ta phải chia LSU thành hai giai đoạn: ghi (thuộc giai đoạn EX của cpu, nữa đầu bộ LSU)) và đọc (thuộc giai đoạn MEM, nữa sau bộ LSU) thông qua các thanh ghi pipelined được bổ sung vào, bộ nhớ DMEM nhìn chung cũng có vai trò như một thanh ghi pipelined. Đối với hoạt động ghi, dữ liệu từ i_st_data được đưa vào một bộ dịch trái theo byte dựa theo độ dịch là i_lsu_addr[1:0]. Bộ dịch trái có đầu là vào 32 bit và kết quả dịch có 64 bit. Điều này để xử lý vấn đề misaligned trong các địa chỉ không căn chỉnh như i_lsu_addr[1:0] = 01,10,11. Sau khi dịch, 32 bit đầu sẽ được đưa vào i_wdata, và 32 bit cao sẽ được đưa vào i_wdata_next_addr của bộ DMEM. Để có thể đáp ứng việc truy cập byte, half_word, ta cần tạo một bộ xử lý bmask dựa trên tín hiệu

i_lsu_addr[1:0] (đại diện cho địa chỉ trên toàn bộ 1 word nhớ của DMEM) và funct3 trong mã lệnh trong việc lựa chọn truy suất byte, half-word hay word. Kế tiếp, trong hoạt động đọc, ta cần đem các tín hiệu cần thiết qua thanh ghi pipelined (như trên hình). Đối với ngõ ra của dữ liệu đọc từ DMEM, ta cần tạo một khối giúp lọc các byte được phép đi qua thông qua các bit bmask. Ta cần một bộ MUX để lựa chọn vùng để xuất ngõ ra. Đây là MUX8-1, có 64 bộ tương ứng xử lý 64 bit. Đưa vào bộ MUX, ta cần ghép {o_rdata_next_addr, o_rd_data} của bộ DMEM. Kế tiếp, ta đưa 64 bit ngõ ra của MUX vào một bọ dịch phải cũng theo byte và dựa theo độ dịch là i_lsu_addr, đầu ra là 32 bit thấp nhất của kết quả phép dịch. Việc dịch phải này tương tự dịch trái, giúp xử lý misaligned. Sau khi đó, ta đưa 32 bit (right_shifter_output) xuất ra từ bộ dịch phải vào một bộ MUX mở rộng dấu cho kết quả load, ta xuất thẳng byte thấp nhất ra trước, do byte này không nằm trong việc mở rộng dấu. Dựa trên hàm funtc3[2:0] để lựa chọn việc mở rộng dấu tương ứng với lệnh. Nếu funtc3 = 000, thì mở rộng dấu cho lb, nếu 001 là mở rộng dấu cho lh, nếu 010 là lw nên ta đưa giữ nguyên toàn bộ 32bit cần load, nếu 100 là lbu nên không cần mở rộng dấu mà ta lấp đầy các bit cao bằng 0, nếu 101 là lhu cũng tương tự ý tưởng của lbu.

2.10. Regfile:

Tập thanh ghi (Regfile) là thành phần cốt lõi của CPU, đóng vai trò là bộ lưu trữ dữ liệu ngắn hạn và nhanh hơn đáng kể so với các bộ nhớ khác. Trong kiến trúc RV32I, tập này chứa 32 thanh ghi đa năng (được đặt tên từ x0 đến x31), mỗi thanh ghi có độ rộng 32 bit.

2.10.1. Design ideal

Hoạt động của nó thông qua các ngõ đọc và ghi (tương tự DMEM):

Read Ports (Asynchronous): Regfile có cấu hình đọc kép (tức hai ngõ độc lập)

- Nó nhận hai đầu vào địa chỉ 5 bit, i_rs1_addr và i_rs2_addr, trong mã lệnh nhận từ IMEM.

- Ngõ đọc theo cấu trúc tổ hợp, tức bất đồng bộ.
- Nó xuất dữ liệu 32 bit trên o_rs1_data và o_rs2_data. Điều này có nghĩa là dữ liệu đọc gần như ngay lập tức thông qua các địa chỉ đọc nhận được, mà không cần chờ cạnh xung clock.

Write Port (Synchronous): Regfile có cấu hình ghi đơn (tức chỉ có một ngõ ghi).

- Nó nhận đầu vào địa chỉ 5 bit, i_rd_addr (địa chỉ ghi), đầu vào i_rd_data (dữ liệu ghi) 32 bit và tín hiệu i_rd_wren (cho phép ghi) 1 bit từ Control Unit (i_rd_wren = 1 là ghi, ngược lại là đọc).
- Ghi theo cấu hình đồng bộ: Ghi chỉ chỉ xảy ra ở cạnh clock.
- Nếu i_rd_wren =1, dữ liệu trên i_rd_data được ghi vào thanh ghi được chỉ định bởi địa chỉ i_rd_addr. Nếu i_rd_wren ở mức thấp, không có thanh ghi nào bị thay đổi.

Thực hiện bổ sung cho Regfile: Để đảm bảo xử lý các hazard liên quan đến việc truy cập regfile mà tại đó việc đọc và ghi diễn ra trong cùng một chu kỳ, ta sẽ thiết kế để Regfile có thể phát hiện hazard liên qua đến nó, đọc dữ liệu chính xác từ dữ liệu đang được ghi vào để xử lý tốt hazard.

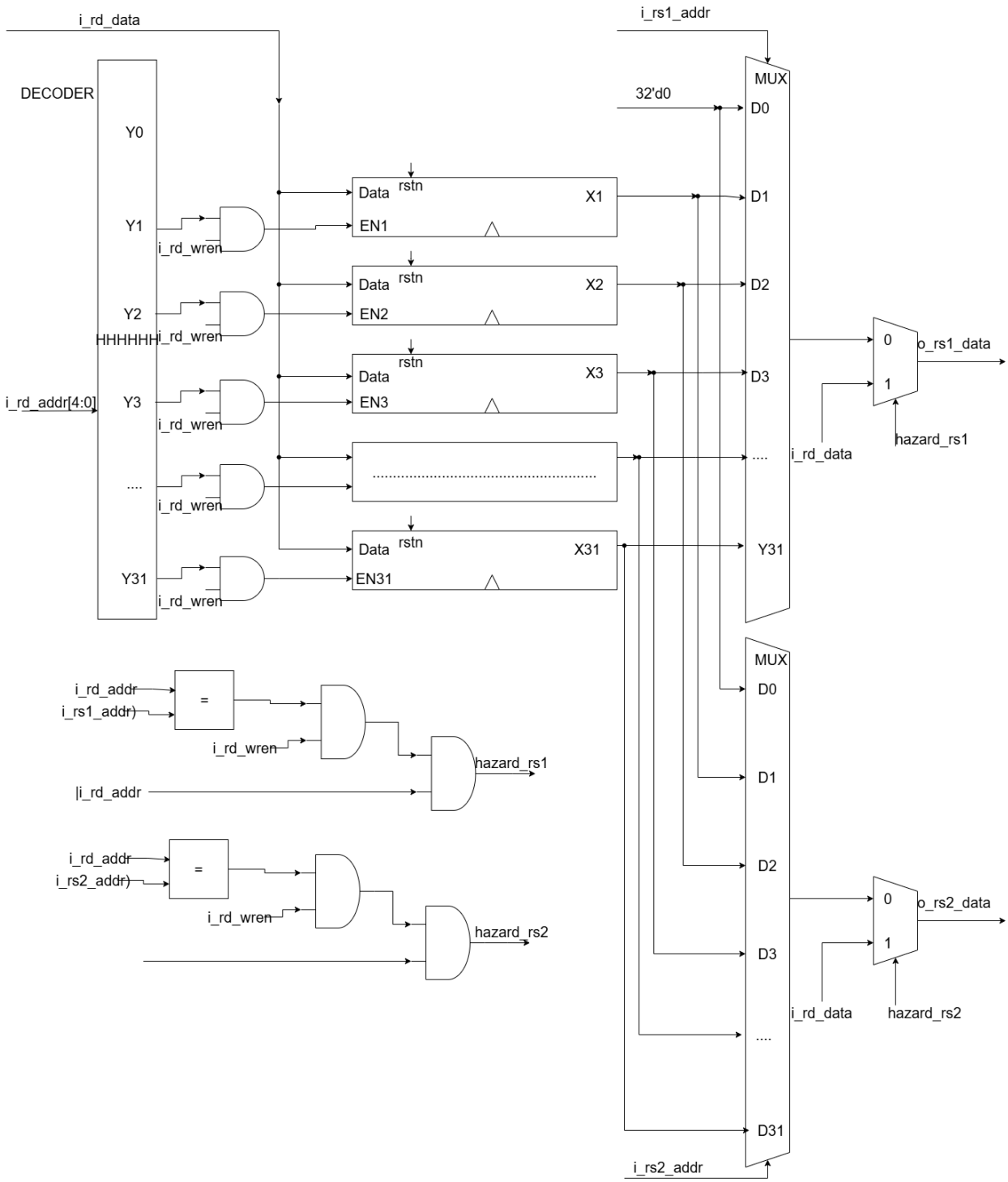
2.10.2. Specification

Module name: regfile.sv

Signal	Width	Direction	Description
i_clk	1	Input	Global clock, active on the rising edge.
i_rstn	1	Input	Global low active reset
i_rd_wren	1	Input	Register writes enable
i_rd_addr	5	Input	Destination registers address (0-31). Specifies which register to write to
i_rd_data	32	Input	Write data. The 32-bit data to be written into the destination register

i_rs1_addr	5	Input	Register 1 address (0-31). (Specifies the address for the first read port)
o_rs1_data	32	Output	Read data 1. (The 32-bit data read from the register at i_rs1_addr).
i_rs2_addr	5	Input	Register 2 address (0-31). (Specifies the address for the second read port)
o_rs2_data	32	Output	Read data 2. (The 32-bit data read from the register at i_rs2_addr)

2.10.3. Thiết kế Regfile: Ý tưởng tương tự DMEM, tuy nhiên, ta chỉ cần một decoder để truy cập việc ghi và cũng sử dụng hai MUX cho hai ngõ ra đọc. Và bổ sung việc sử lý hazard.



2.11. IMEM

IMEM (Bộ nhớ Lệnh) là một bộ nhớ dùng để lưu trữ và đọc các lệnh để thực thi của CPU.

Trong thiết kế nhóm em, khối này hoạt động tương tự dmem, đọc dữ liệu lệnh đồng bộ clock.

1.1.1. Design ideas

Bộ nhớ IMEM sẽ nhận được địa chỉ đầu vào 32 bit, từ Bộ đếm Chương trình (PC). Dựa trên địa chỉ này, nó sẽ truy xuất lệnh 32 bit được lưu trữ tại địa chỉ tương ứng ở xung clock kế tiếp. Nó xuất lệnh 32 bit này ra datapath (đến các khối khác như Control Unit) để giải mã và thực thi.

Cấu trúc của IMEM tương tự như DMEM nhưng được giảm đi một số chân không cần thiết, tuy nhiên, chỉ cần một MUX để có thể đọc nội dung lệnh tương ứng với địa chỉ nhận từ PC. Do i_addr có 32 bit, nên bộ nhớ có thể có lên tới 2^32 word nhớ.

1.1.2. Specification

Module name: imem.sv

Signal	Width	Direction	Description
i_addr	32	Input	Instruction address (the address received from the PC)
o_rdata	32	Output	Instruction read data. (The 32-bit instruction fetched from memory)
i_clk	1	input	Tạo xung clock.

2.12. Control Unit:

Bộ Điều khiển (Control Unit) về cơ bản là "bộ não" của CPU. Nhóm thiết kế nó theo dạng một khối logic tổ hợp điều khiển hoạt động của toàn bộ đường dẫn dữ liệu bằng cách tạo ra tất cả các tín hiệu điều khiển cần thiết.

Trong bộ xử lý chu kỳ đơn, tất cả các tín hiệu điều khiển này được tạo ra dựa tên mã bit lệnh và duy trì thời gian của một chu kỳ xung clock cho đến khi được thay đổi bởi mã lệnh kế tiếp.

2.12.1. Design ideal

Input: Đầu vào chính của nó là chính mã lệnh 32 bit.

Function: Nhiệm vụ chính của Bộ Điều khiển là giải mã lệnh. Nó đọc mã lệnh để xác định loại lệnh (ví dụ: loại R, loại I, Load, Store, Branch, Jump). Dựa trên nhận dạng này, nó tạo ra một tập hợp các tín hiệu đầu ra để điều khiển CPU tương ứng với lệnh.

Output: Các tín hiệu đầu ra này là các dòng nhị phân "bật/tắt" hoặc "chọn" để điều khiển các thành phần phần cứng khác.

2.12.2. Specification

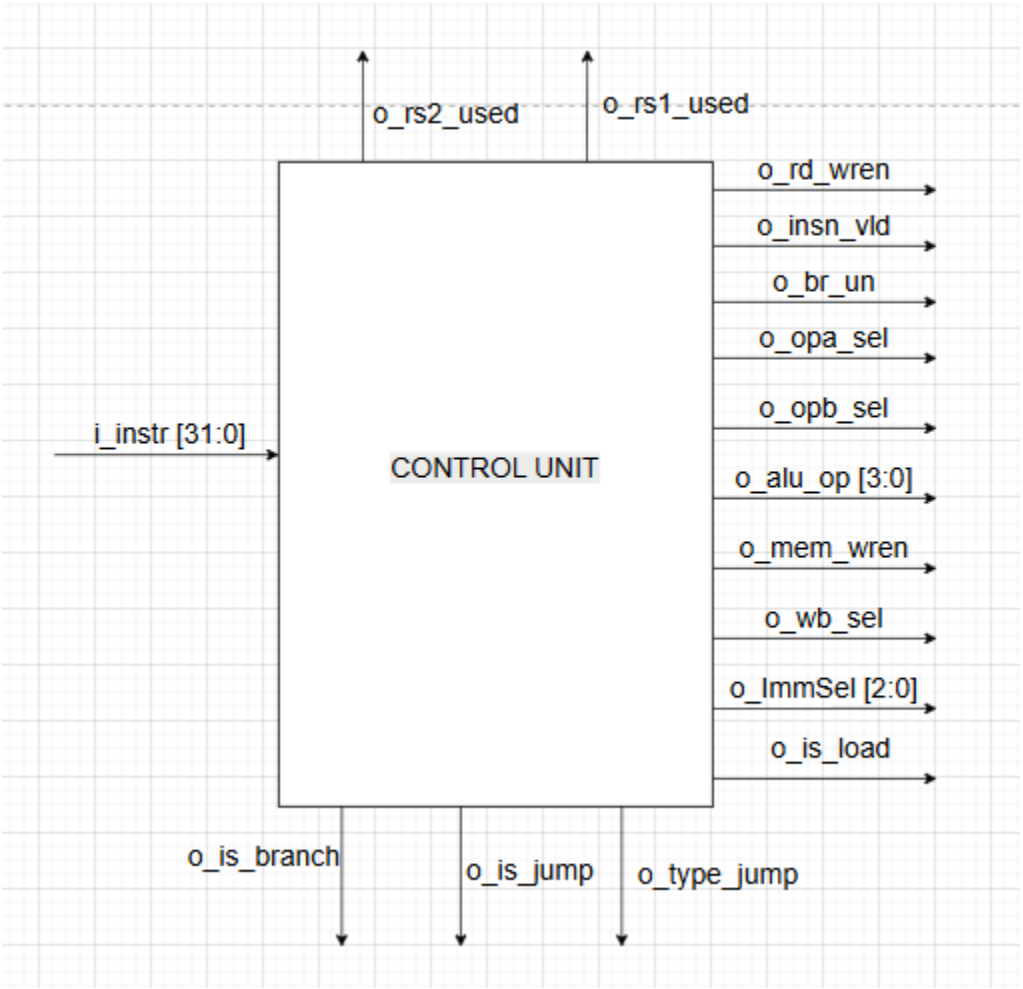
Module name: ControlUnit.sv

Signal	Width	Direction	Description
i_instr	32	Input	Instruction word received from imem
i_br_equal	1	Input	Flag indicates rs1 = rs2 from BRC
i_br_less	1	Input	Flag indicates rs1 < rs2 from BRC
o_pc_sel	1	Output	Next PC source select. (1 if select PC+4 , 0 if select branch/jump target)
o_rd_wren	1	Output	Register file write enable. (Asserted to write data back to the Regfile).
o_br_un	1	Output	Branch unsigned select. (1 if signed, 0 if unsigned).
o_opa_sel	1	Output	ALU operand A select. (1 if select rs1_data, 0 if select PC).
o_opb_sel	1	Output	ALU Operand B select. (1 if select rs2_data, 0 if select immediate value from ImmGen).
o_alu_op	4	Output	ALU operation code. (Tells the ALU

			which operation to perform)
o_mem_wren	1	Output	Memory write enable. (0 if select alu result, 1 if select load data, and 2 if select PC +4)
o_wb_sel	2	Output	Write back data select. (Selects the data to write to the Regfile
o_ImmSel	3	Output	Immediate generator select. (Tells the ImmGen block which immediate format to decode).
o_insn_vld	1	Output	Instruction valid. (Indicates that the current instruction is valid).
o_is_branch	1	Output	Nếu là lệnh nhóm B thì sẽ bật lên 1 để báo hiệu, đi vào các khối Branch Control Unit để điều khiển việc rẻ nhánh.
o_is_jump	1	Output	Nếu là lệnh nhóm jal hoặc jalr thì sẽ bật lên 1 để báo hiệu, đi vào các khối Branch Control Unit để điều khiển việc rẻ nhánh.
o_type_jump	1	Output	Tín liệu này lên 1 khi lệnh là jal, ngược lại là lệnh jalr. Sẽ được truyền vào Branch Control Unit để điều khiển việc rẻ nhánh.
o_rs1_used	1	Output	Báo hiệu lệnh có sử dụng đến thanh ghi ghi tại địa chỉ rs1, truyền đến bộ forwarding control unit để thực hiện tín hiệu điều khiển forwarding

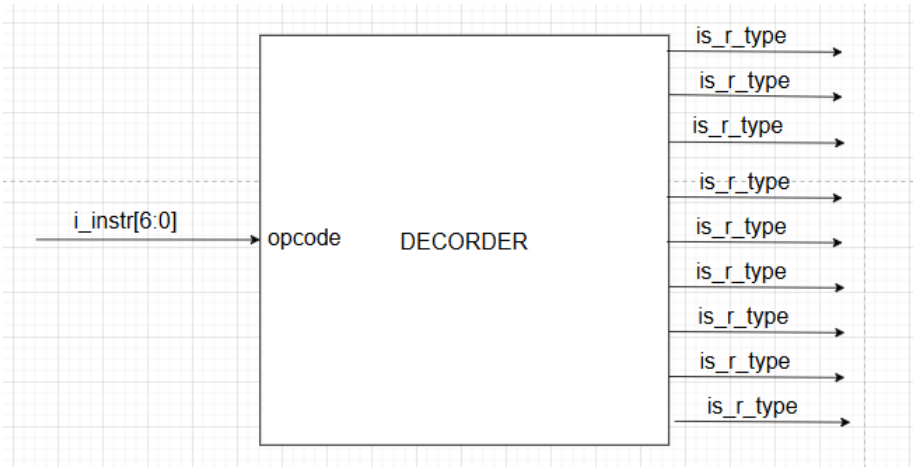
o_rs2_used	1	Output	Báo hiệu lệnh có sử dụng đến thanh ghi ghi tại địa chỉ rs2, truyền đến bộ forwarding control unit để thực hiện tín hiệu điều khiển forwarding

Sơ đồ chân:

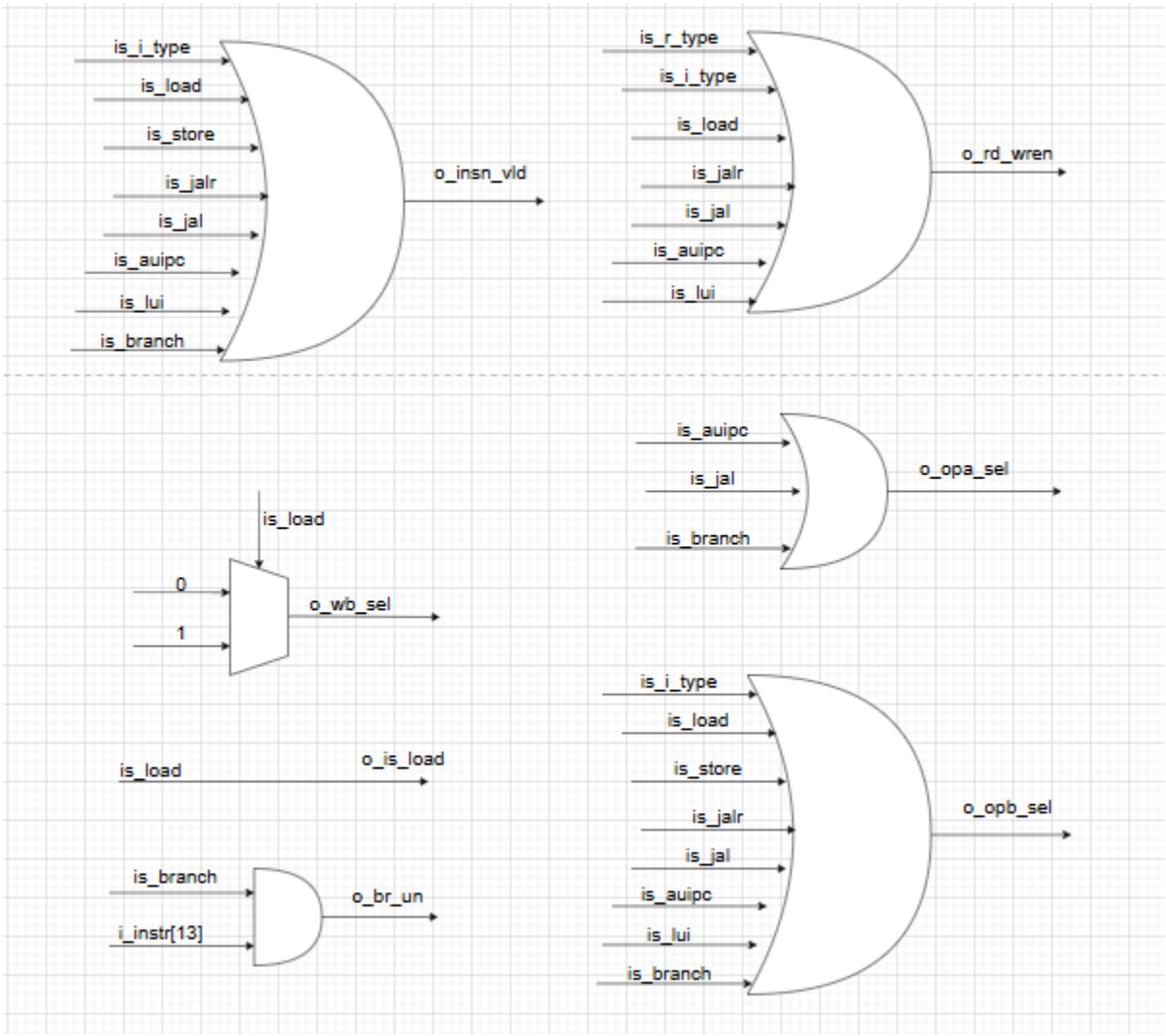


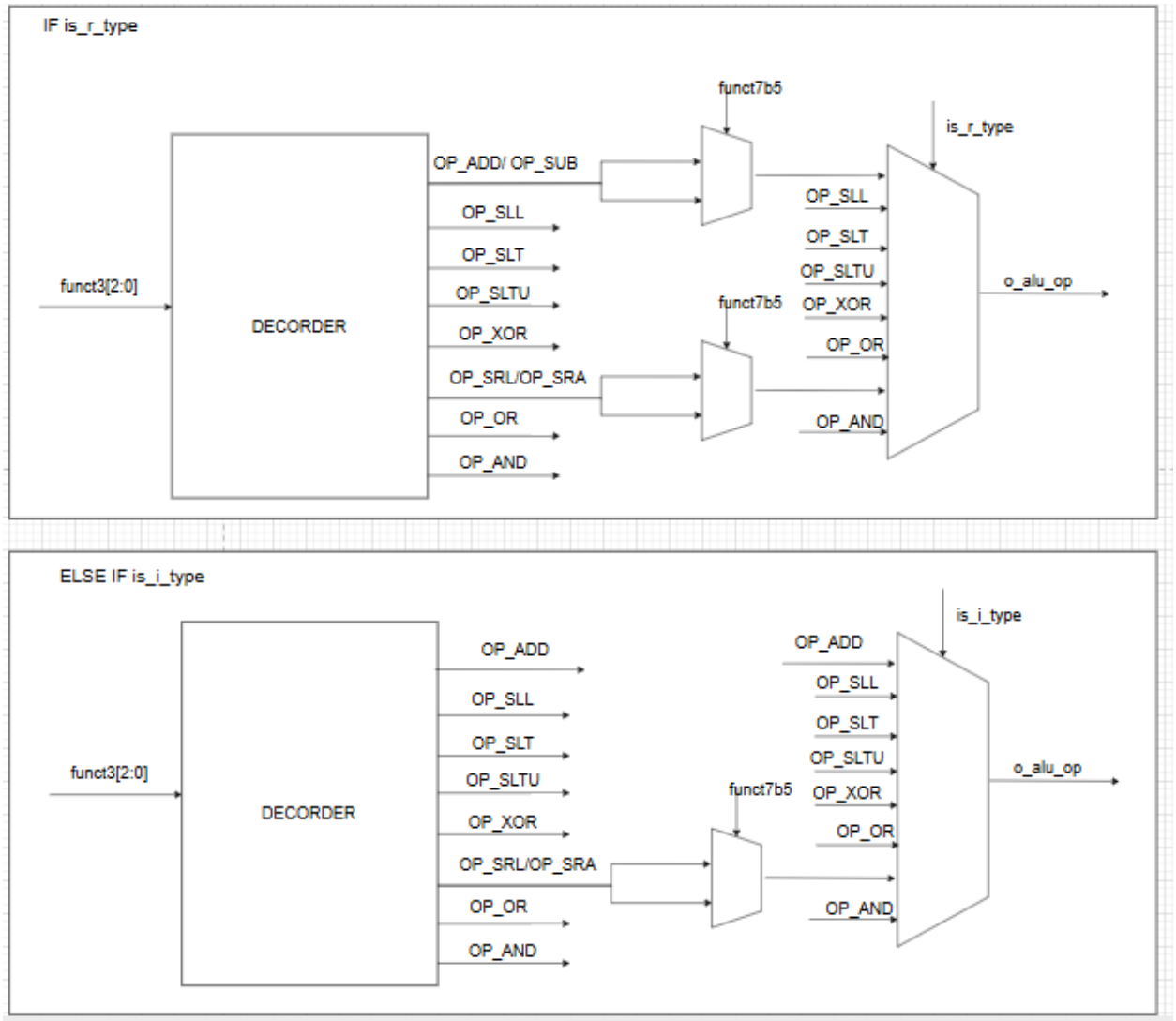
Chi tiết hoạt động:

Đi qua bộ giải mã lệnh:

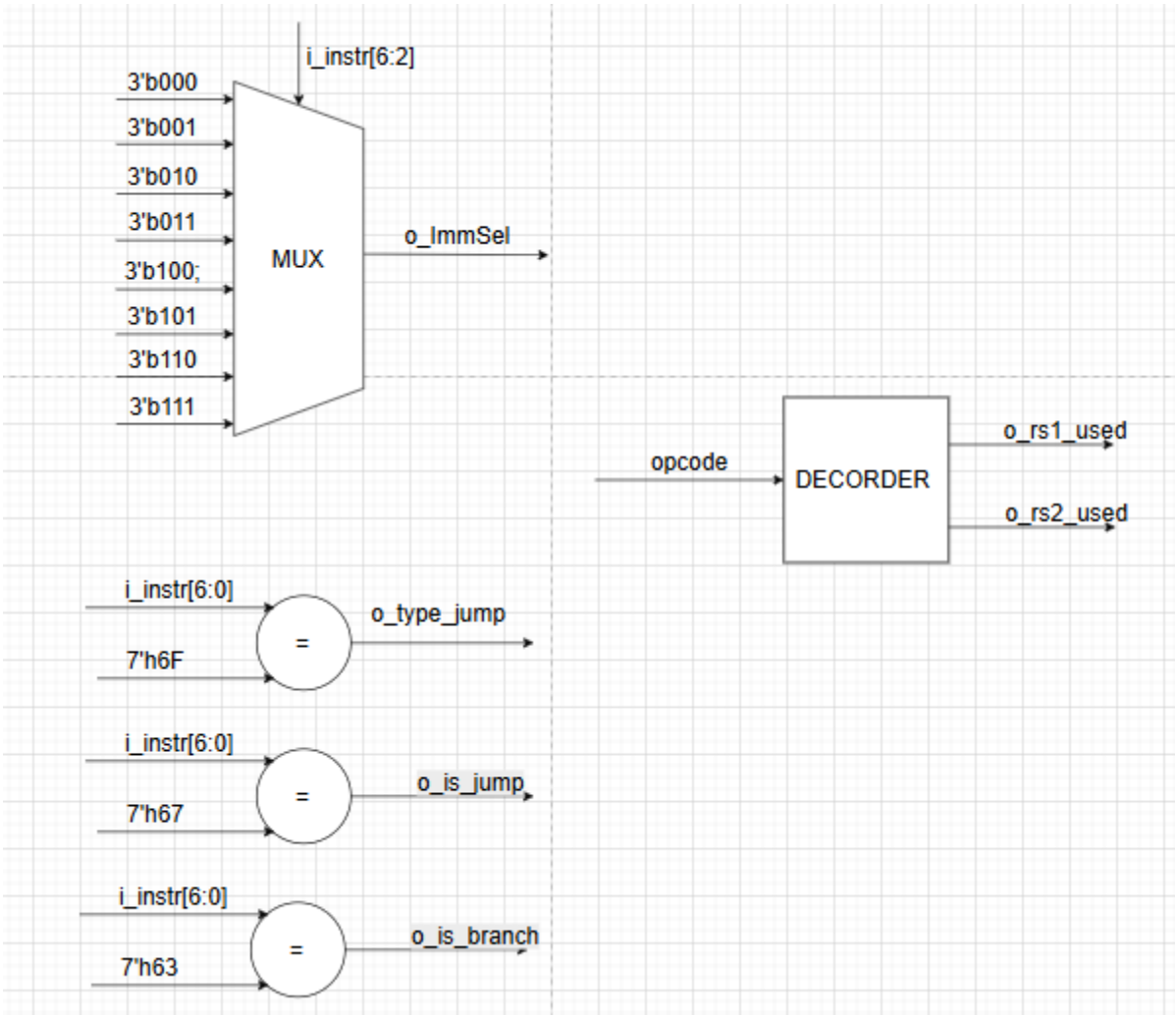


Tạo các tín hiệu điều khiển: Các tín hiệu này được lan truyền theo CPU pipelined





Các tín hiệu bổ sung cho hoạt động của CPU pipelined:



2.13. ALU:

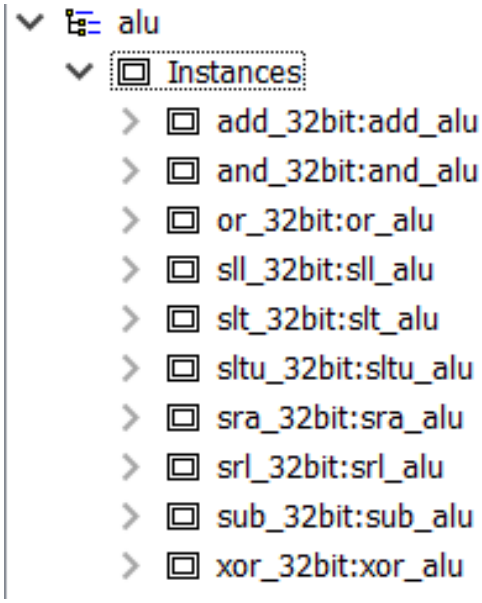
ALU là khối có khả năng thực hiện các phép toán số học và logic, khối này sẽ nhận hai đầu vào 32 bit, thực hiện phép toán và đưa ra kết quả 32 bit. Điều này được định nghĩa trong tập lệnh RV32I như hình bên dưới.

alu_op	Description (R-type)	Description (I-type)
ADD	$rd \leftarrow rs1 + rs2$	$rd \leftarrow rs1 + imm$
SUB	$rd \leftarrow rs1 - rs2$	n/a
SLT	$rd \leftarrow (rs1 < rs2)?\ 1 : 0$	$rd \leftarrow (rs1 < imm)?\ 1 : 0$
SLTU	$rd \leftarrow (rs1 < rs2)?\ 1 : 0$	$rd \leftarrow (rs1 < imm)?\ 1 : 0$
XOR	$rd \leftarrow rs1 \oplus rs2$	$rd \leftarrow rs1 \oplus imm$
OR	$rd \leftarrow rs1 \vee rs2$	$rd \leftarrow rs1 \vee imm$
AND	$rd \leftarrow rs1 \wedge rs2$	$rd \leftarrow rs1 \wedge imm$
SLL	$rd \leftarrow rs1 \ll rs2[4:0]$	$rd \leftarrow rs1 \ll imm[4:0]$
SRL	$rd \leftarrow rs1 \gg rs2[4:0]$	$rd \leftarrow rs1 \gg imm[4:0]$
SRA	$rd \leftarrow rs1 \ggg rs2[4:0]$	$rd \leftarrow rs1 \ggg imm[4:0]$

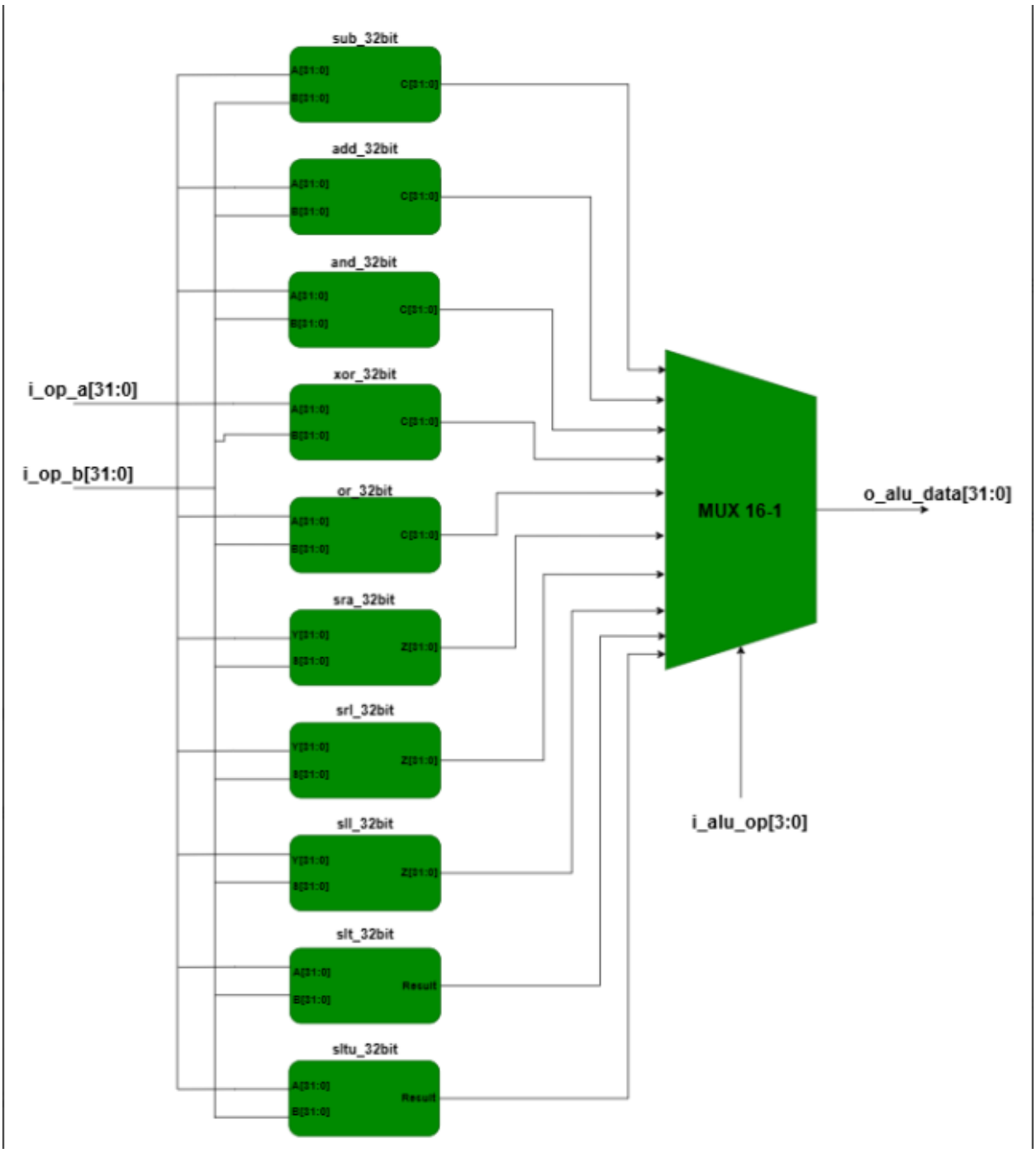
2.13.1.Design ideas

Nhóm sẽ thiết kế tuần tự từng khối chức năng trong ALU. các mô-đun như add_32bit.sv, sub_32bit.sv, xor_32bit.sv, or_32bit.sv, sra_32bit.sv, srl_32bit.sv, sll_32bit.sv, slt_32bit.sv, sltu_32bit.sv và nối chúng vào một bộ MUX lớn để tạo thành bộ alu.sv.

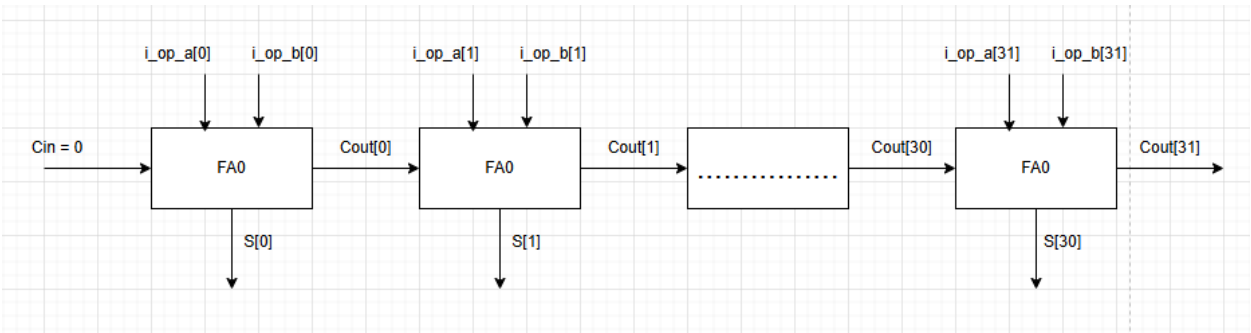
Ngoài ra, một vài module phụ khác cũng cần thiết để khởi tạo trong việc xây dựng các khối khác như full_adder.sv, mux_32_to_1.sv. Sẽ có một MUX 16-1 để chọn loại toán tử cần thực hiện. MUX này sẽ được lựa chọn bởi tín hiệu từ Bộ Điều khiển.



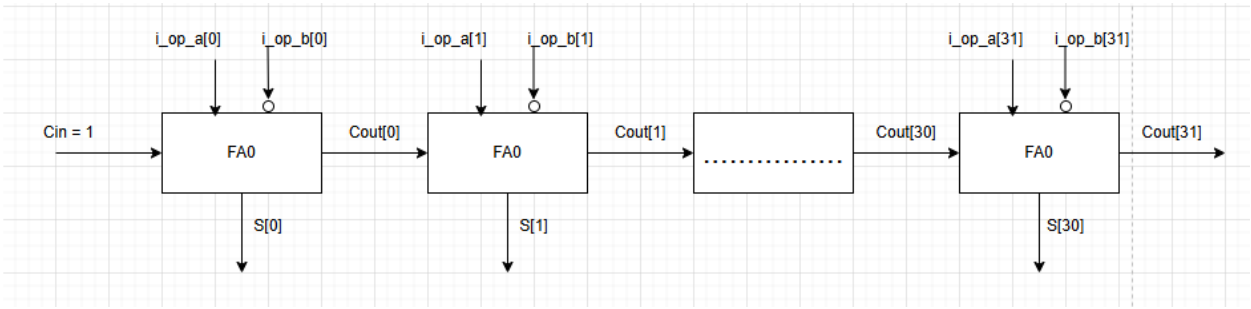
Design sketch: thiết kế của khối ALU được như hình ảnh bên dưới.



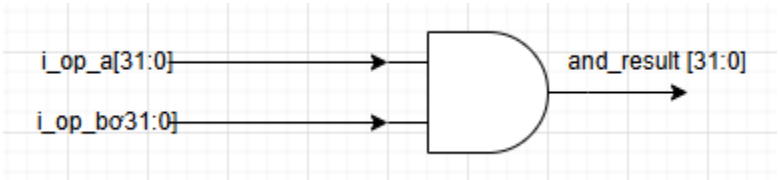
Thiết kế của bộ cộng: Theo phương pháp RCA.



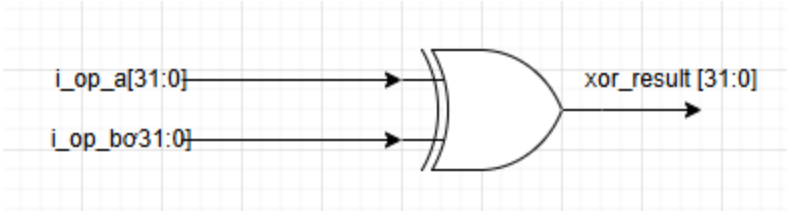
Thiết kế bộ trừ: Cũng theo phương pháp RCA.



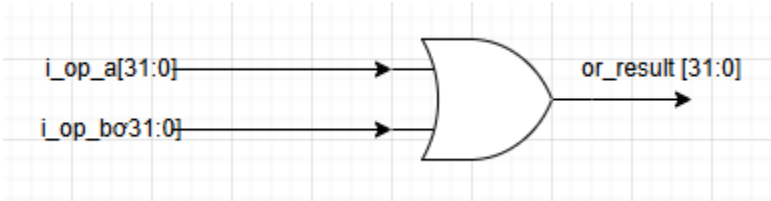
Thiết kế bộ AND:



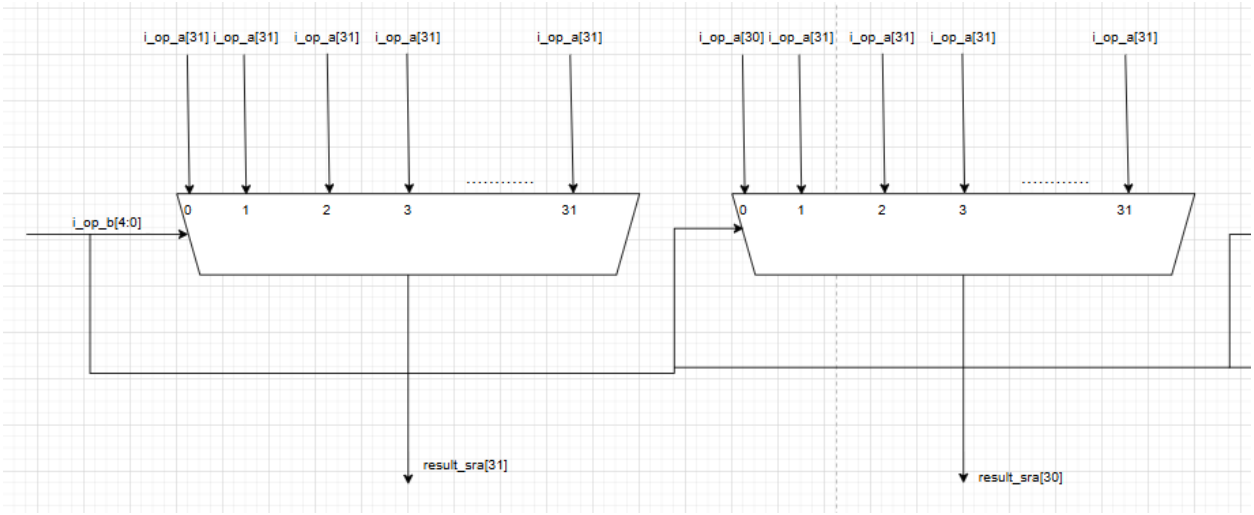
Thiết kế bộ XOR:

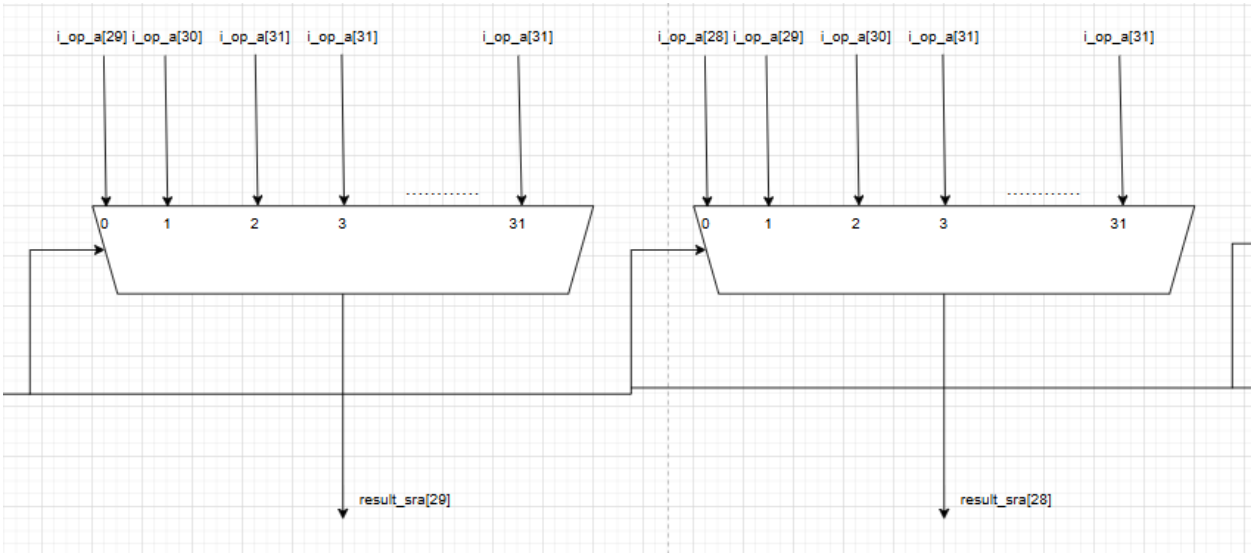


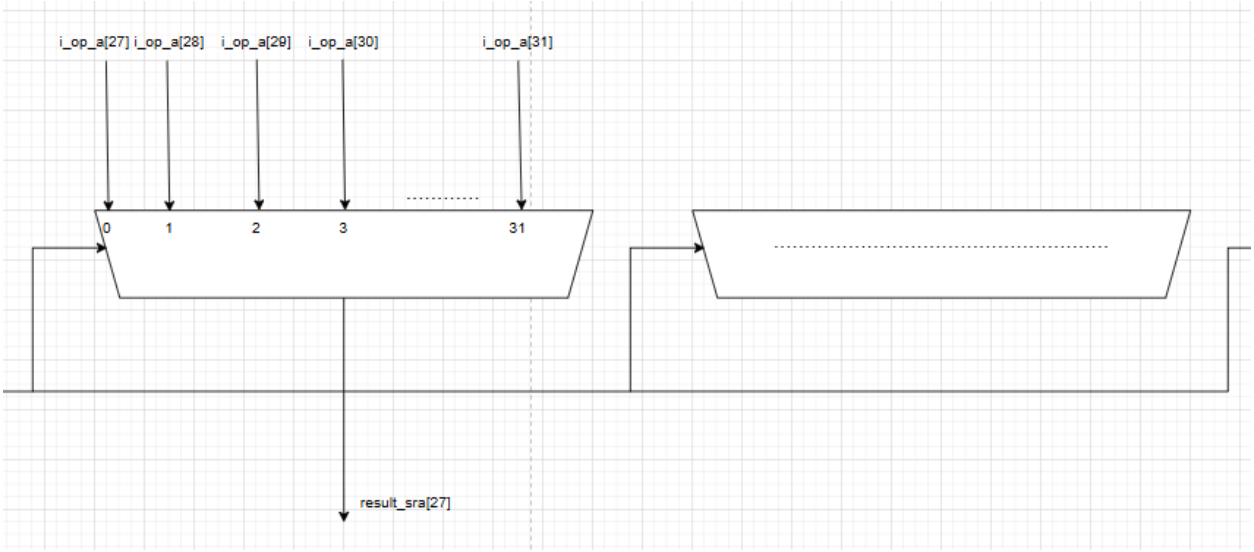
Thiết kế bộ OR:

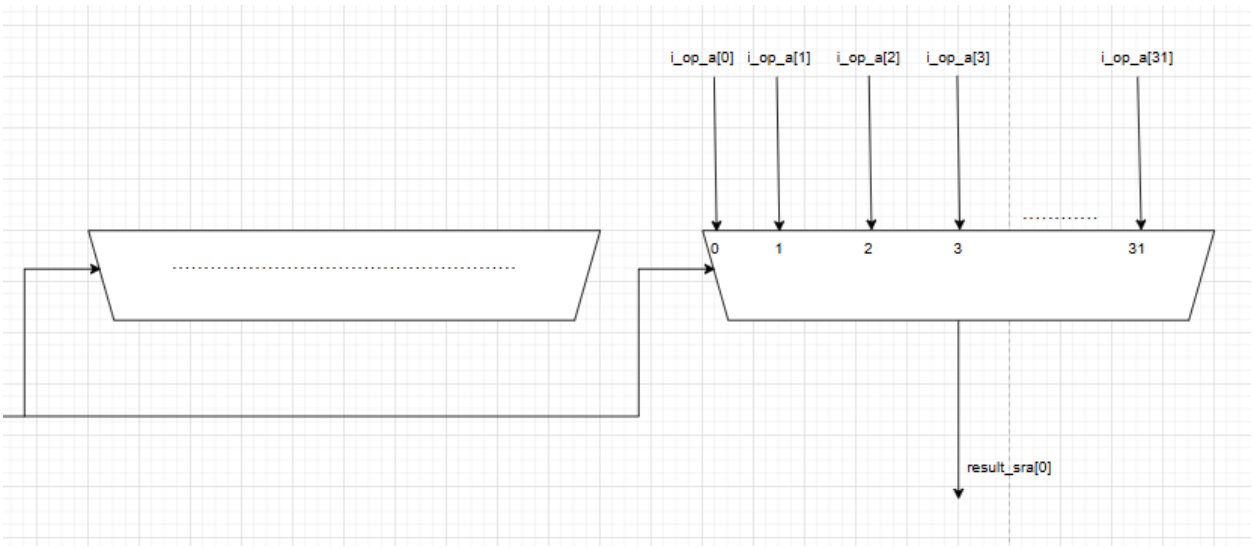


Thiết kế bộ SRA: Gồm các bộ MUX32-1 ghép song song nhau.


i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_a[30] i_op_a[31] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_b[4:0]
0 1 2 3 31
0 1 2 3 31
result_sra[31]
result_sra[30]

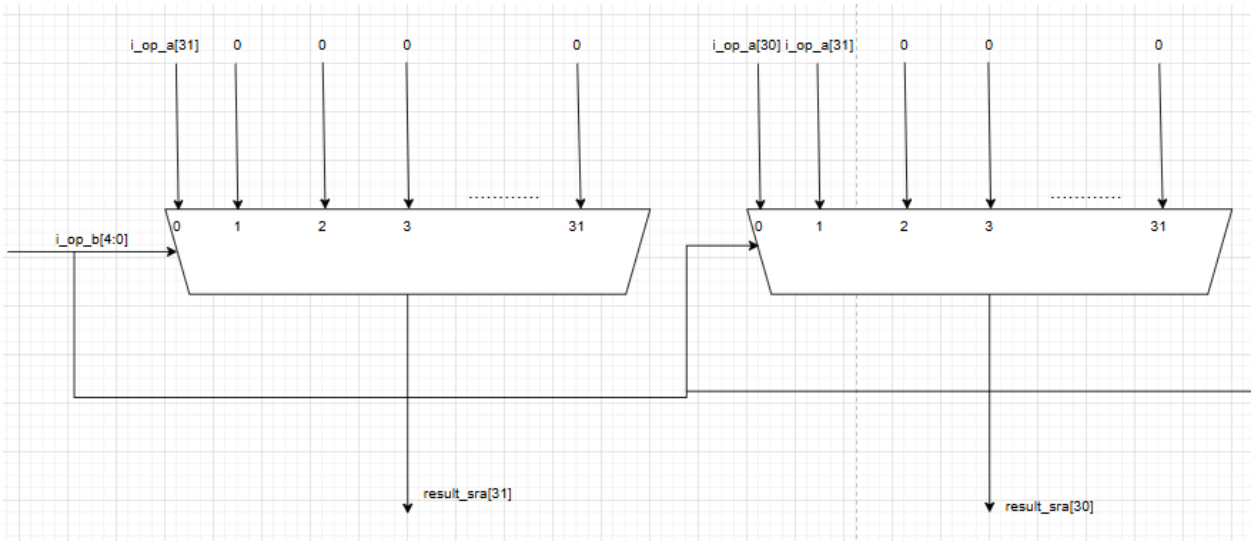

i_op_a[29] i_op_a[30] i_op_a[31] i_op_a[31] i_op_a[31]
i_op_a[28] i_op_a[29] i_op_a[30] i_op_a[31] i_op_a[31]
0 1 2 3 31
0 1 2 3 31
result_sra[29]
result_sra[28]

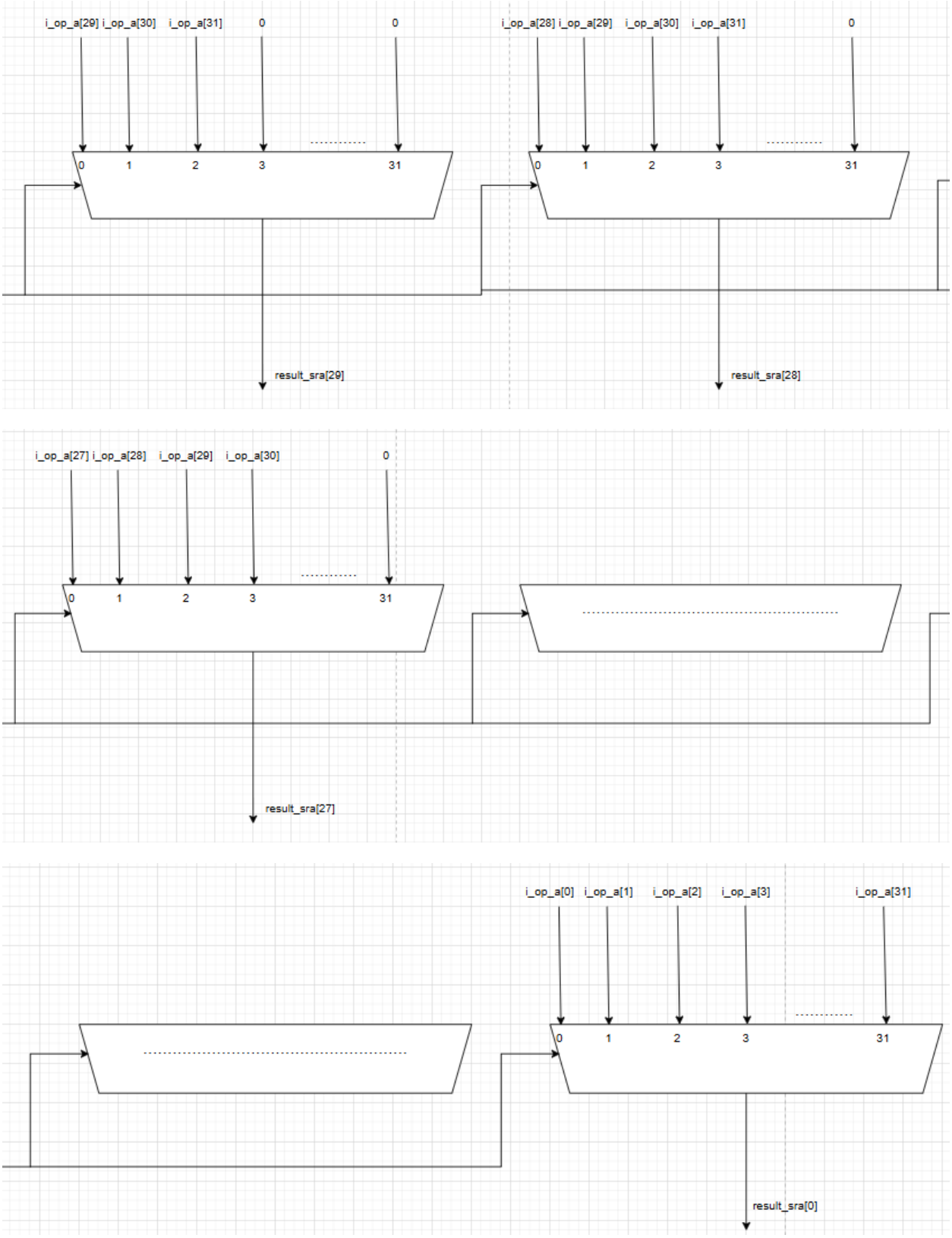

i_op_a[27] i_op_a[28] i_op_a[29] i_op_a[30] i_op_a[31]
0 1 2 3 31
result_sra[27]



*Chú thích: Dấu "…" thể hiện các ngõ vào hoặc các mux được ẩn đi để hình vẽ gọn hơn.

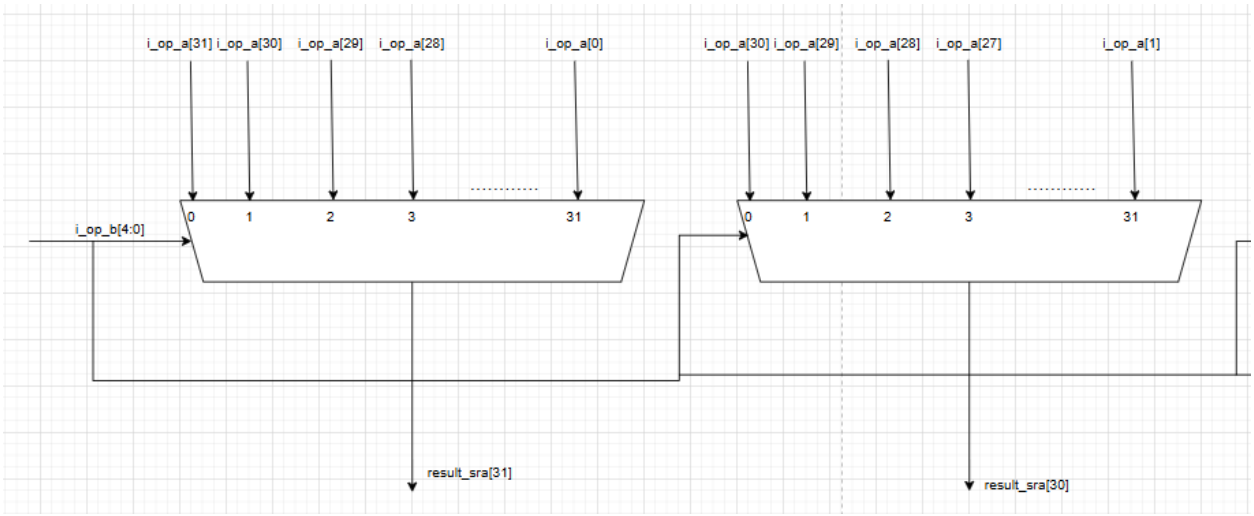
Thiết kế bộ SRL: Cấu trúc tương tự như SRA.

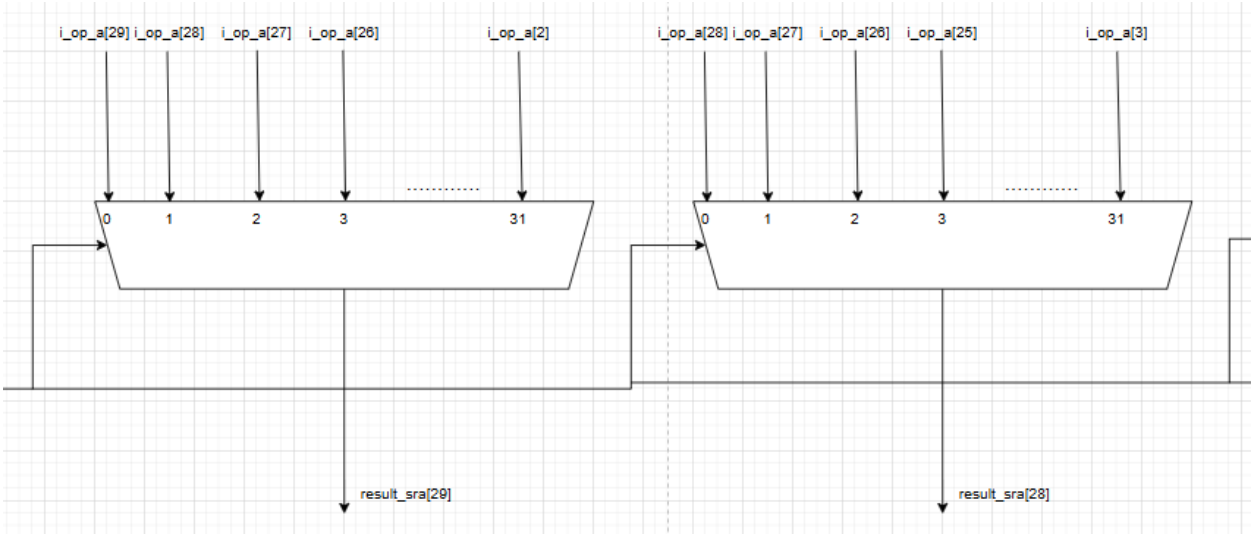


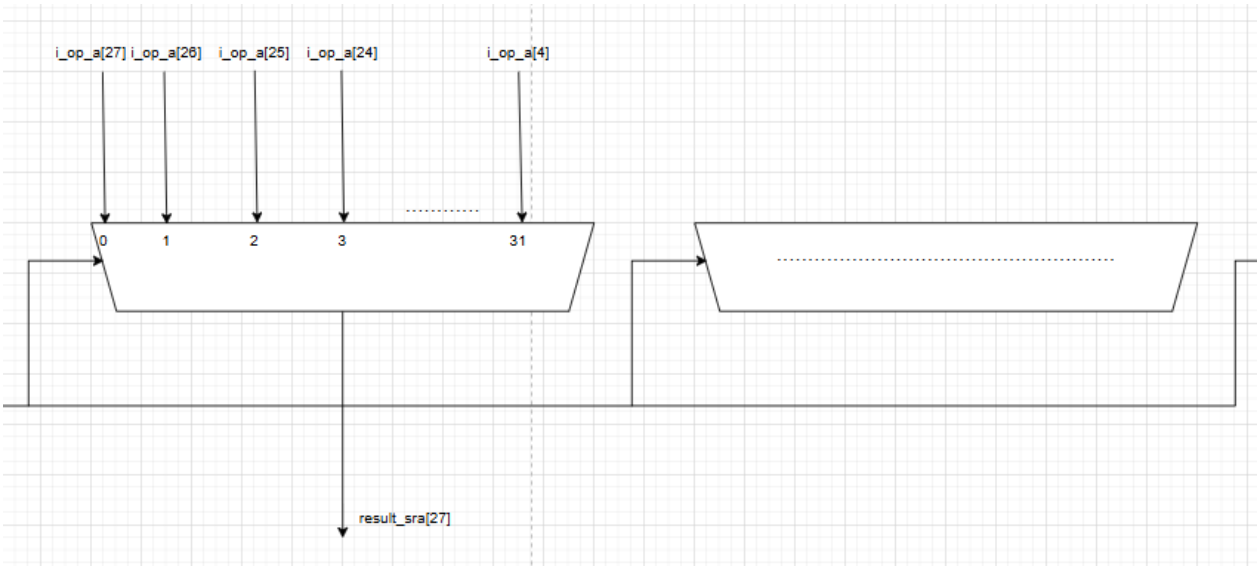


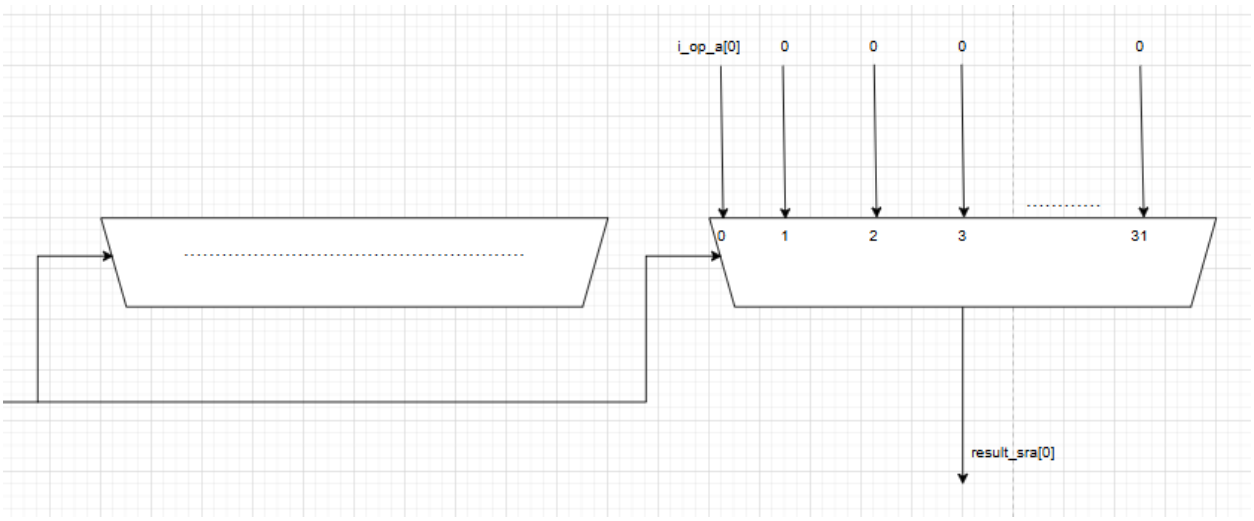
*Chú thích: Dấu "…" thể hiện các ngõ vào hoặc các mux được ẩn đi để hình vẽ gọn hơn.

Thiết kế bộ SLL: Cấu trúc tương tự như các bộ dịch phải.









Thiết kế bộ so sánh: slt và sltu được khởi tạo từ module BRC (sẽ được trình bày chi tiết ở mục sau) trong đó các ngõ vào i_op_a và i_op_b được nối lần lượt với các ngõ vào của module BRC là i_rs1_data và i_rs2_data. Ngõ vào i_br_un được điều khiển bởi Control Unit (kết hợp với tín hiệu lựa chọn operations), ví dụ, nếu lựa chọn toán tử slt thì i_br_un = 0, ngược lại, chọn sltu thì i_br_un = 1. Ngõ ra o_alu_data được nối với bộ MUX lớn để lựa chọn ngõ ra ALU.

2.13.2. Specification

Module name: alu.sv

Signal	Width	Direction	Description
i_op_a	32	Input	First operand for ALU operations.
i_op_b	32	Input	Second operand for ALU operations.
i_alu_op	4	Input	The operation to be performed.
o_alu_data	32	Output	Result of the ALU operation.

2.14. BRC:

Đơn vị BRC chịu trách nhiệm so sánh giá trị của hai thanh ghi để xác định kết quả của các lệnh rẽ nhánh. Đơn vị này cần có khả năng xử lý cả phép so sánh có dấu và không dấu, do đó cần thêm tín hiệu i_br_un để xác định đầu ra.

Thiết kế bộ BRC được trình bày như hình vẽ:

Specification

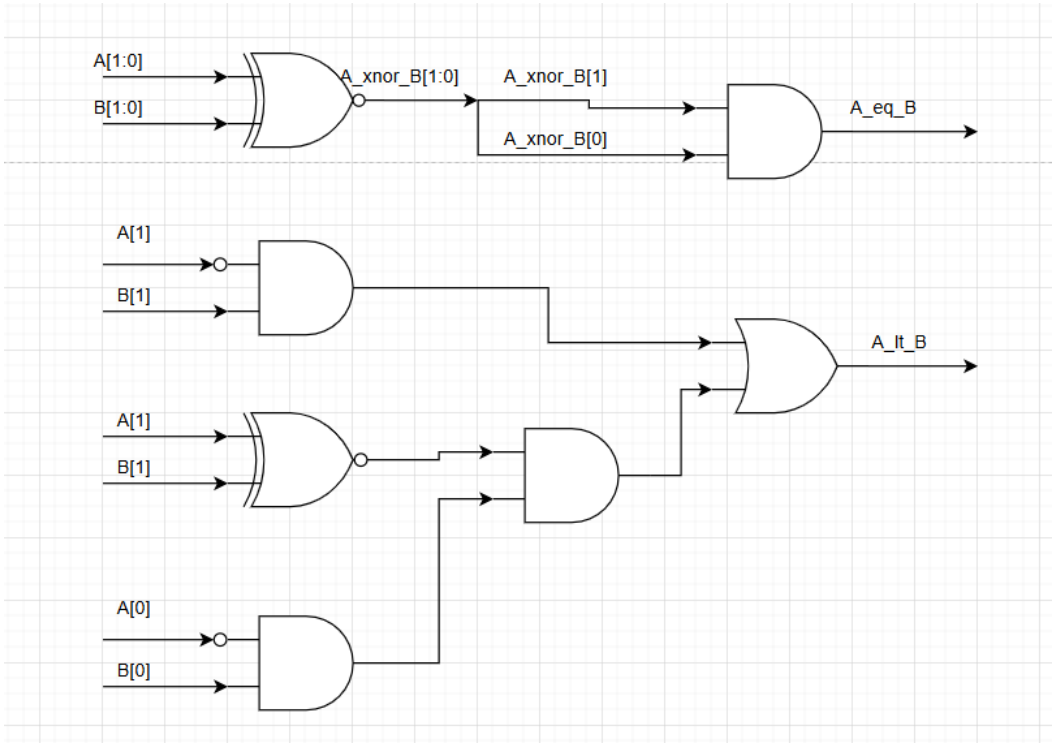
Module name: brc.sv

Signal	Width	Direction	Description
i_rs1_data	32	Input	Data from the first register
i_rs2_data	32	Input	Data from the second register
i_br_un	1	Input	Comparison mode (1 if signed, 0 if unsigned)
o_br_less	1	Output	Output is 1 if rs1 < rs2
o_br_equal	1	Output	Output is 1 if rs1 = rs2

Thiết kế bộ BRC được xây dựng từ việc thiết kế bộ so sánh hai không dấu số 2 bit, rồi sau đó thiết kế bộ so sánh không dấu 2 số 4 bit, 8 bit và 16 bit. Sử dụng hai bộ so sánh không dấu hai số 16 bit để thiết kế bộ BRC.

Được trình bày như sau:

- Thiết kế bộ so sánh không dấu hai số 2 bit:



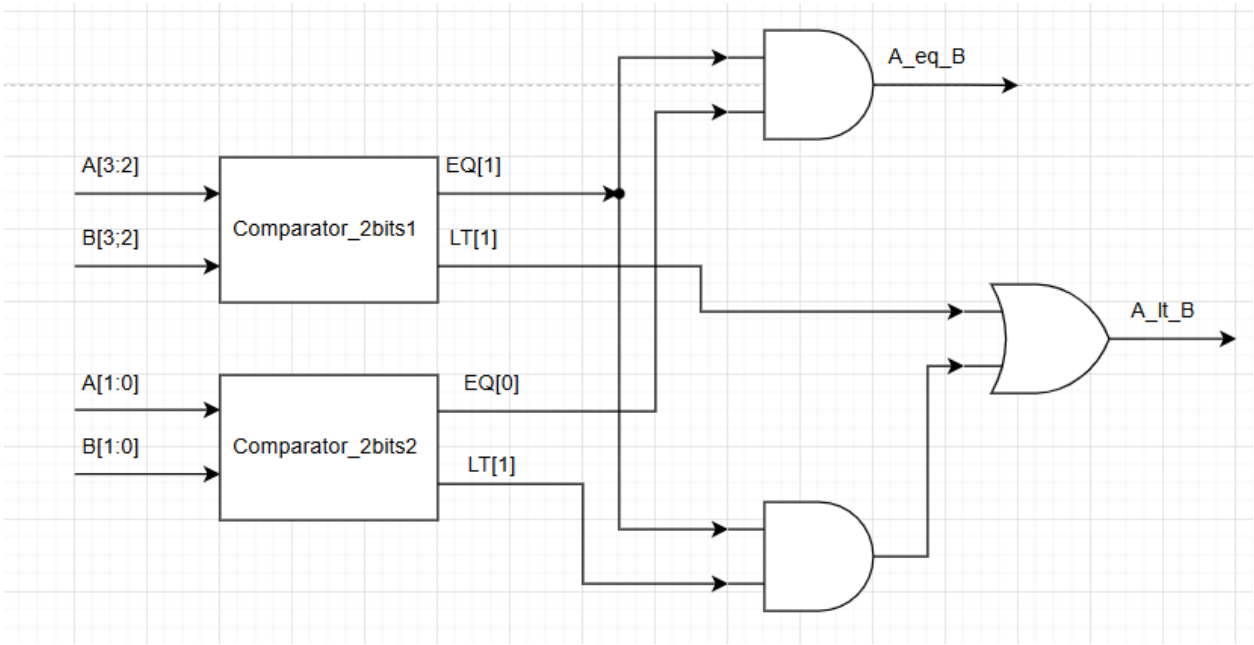
Sử dụng cổng so trong việc tạo tín hiệu so sánh bằng. XOR từng bit của 2

số để xét sự giống nhau của từng cặp bit và AND hai kết quả, nếu cả hai cặp bit đều giống nhau thì A_eq_B = 1.

A_lt_B = 1 khi A[1] = 0 và B[1] = 1 (lưu ý: so sánh không dấu). Nếu bit A[1] = B[1], thì ta xét nếu A[0] = 0 và B[0] = 1 thì A_lt_B = 1.

- Thiết kế bộ so sánh không dấu 4 bit:

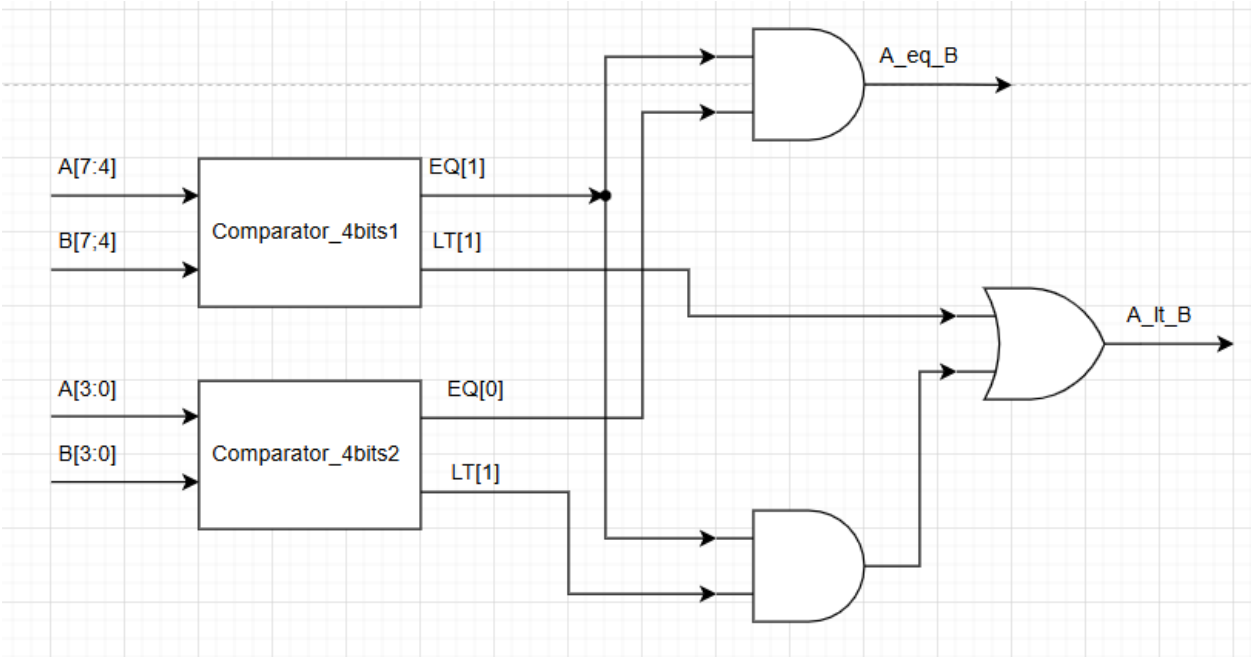
  Sử dụng 2 bộ so sánh số 2 bit vừa thiết kế:



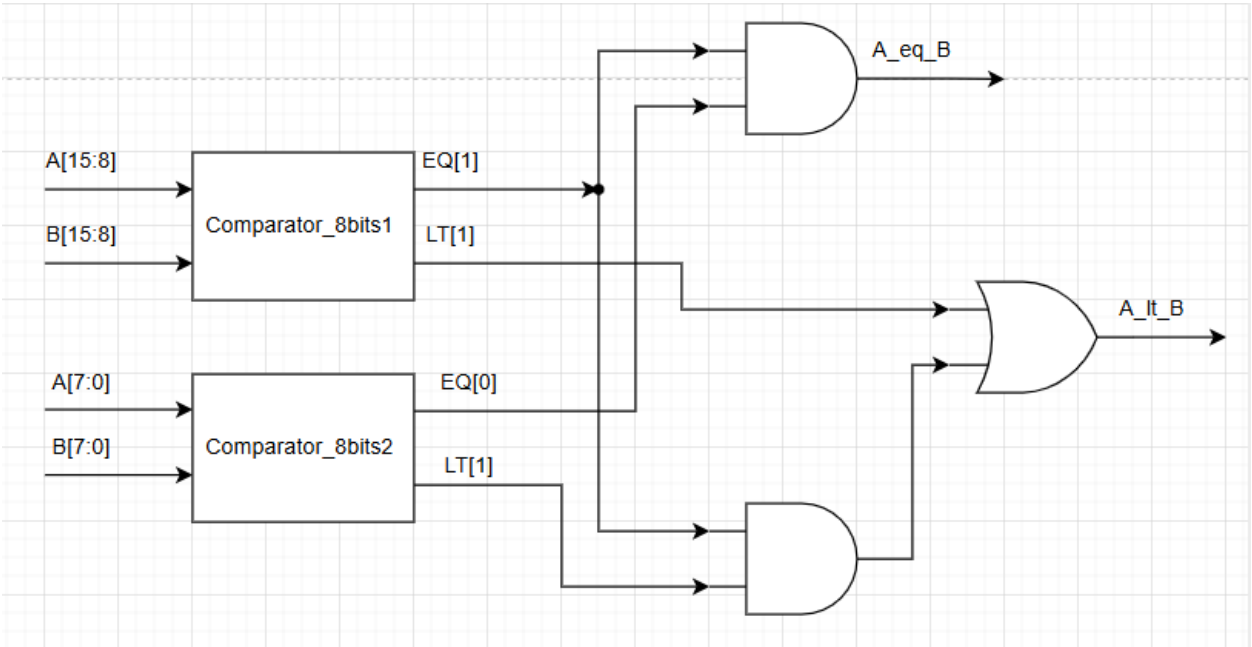
  A_eq_B = 1 khi A[3:2] = B[3:2] và A[1:0] = B[1:0]

  A_lt_B = 1 khi A[3:2] < B[3:2] hoặc A[3:2] = B[3:2] và A[1:0] < B[1:0]

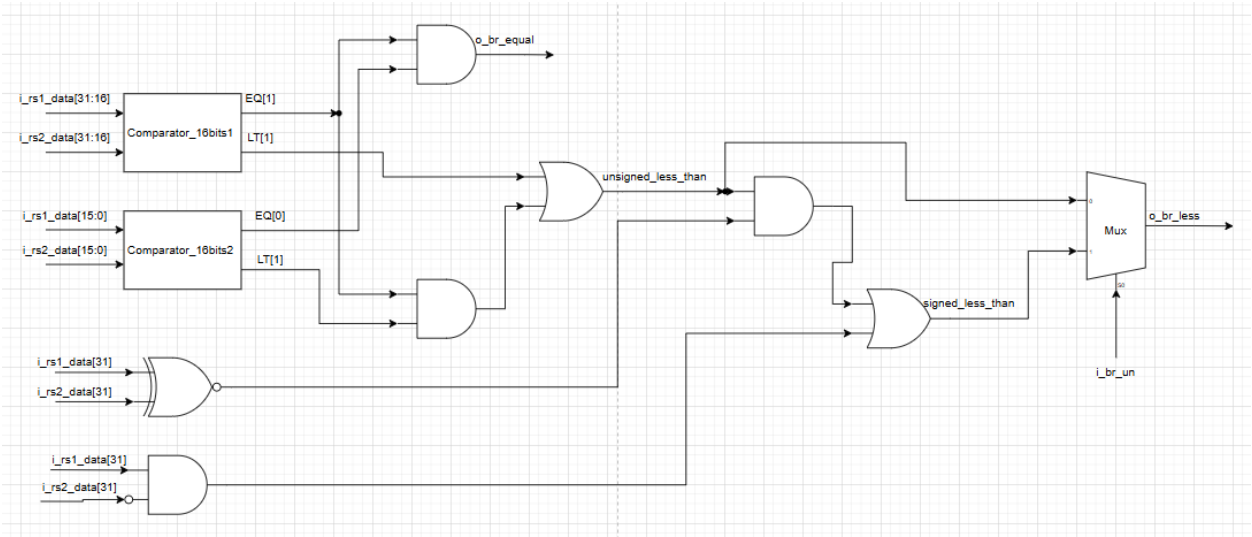
- Thiết kế bộ so sánh không dấu 2 số 8 bit: Tương tự khi vừa thiết kế bộ so sánh 4 bit.



- Thiết kế bộ so sánh không dấu 2 số 16 bit: Tương tự khi vừa thiết kế bộ so sánh 4 bit.



- Thiết kế toàn bộ BRC: Sử dụng hai bộ so sánh không dấu hai số 16bit vừa thiết kế:



o_br_equal = 1 khi A[31:16] = B[31:16] và A[15:0] = B[15:0]

Tín hiệu so sánh không dấu: unsigned_less_than = 1 khi i_rs1_data[31:16]<i_rs2_data[31:16] hoặc i_rs1_data [15:0] = i_rs2_data [15:0] và i_rs1_data [31:16] < i_rs2_data [31:16].

Tín hiệu so sánh có dấu: signed_less_than = 1 khi i_rs1_data[31] = 1 và i_rs2_data[31] = 0. Hoặc i_rs1_data[31] = i_rs2_data[31] và unsigned_less_than = 1.

Sau đó tín hiệu i_br_un sẽ chọn tín hiệu unsigned_less_than hay signed_less_than.

2.15. Immediate Generator:

Immediate Generator là một khối logic tổ hợp chịu trách nhiệm giải mã và tạo ra giá trị tức thời - imm 32 bit, mở rộng dấu cho imm chính xác từ một từ lệnh 32 bit. Hoạt động của nó rất quan trọng để xử lý các lệnh sử dụng toán hạng imm.

2.15.1. Design ideal

Input: Khối này lấy từ lệnh 32 bit (cụ thể là bit [31:7]) làm đầu vào dữ liệu chính. Khối này cũng nhận tín hiệu điều khiển (i_ImmSel) từ Control Unit để chọn định dạng lệnh ứng với giá trị imm.

Function: Khối này về cơ bản là một bộ MUX lớn. Dựa trên tín hiệu điều khiển

i_ImmSel, nó sẽ lựa chọn, sắp xếp lại và nối các bit thích hợp từ lệnh đến đầu ra imm 32 bit. Module này phải được thiết kế xuất kết quả chính xác cho tất cả các giá trị tức imm ứng với các Trường lệnh được quy định bởi RV32I ISA (kiểu I, kiểu S, kiểu B, kiểu U và kiểu J).

Mở rộng Dấu: Một chức năng quan trọng của khối này là mở rộng dấu. Khối này xác định bit dấu cho định dạng đã cho (hầu như luôn là inst[31]) và sao chép nó để lấp đầy tất cả các bit cao hơn của đầu ra 32 bit.

Output: Khối này tạo ra một giá trị tức thời 32 bit duy nhất, mở rộng dấu. Giá trị này sau đó thường được gửi làm một trong các đầu vào cho ALU (thông qua việc lựa chọn cho i_op_b của ALU).

2.15.2. Specification

Module name: immGen.sv

Signal	Width	Direction	Description
i_instr	32	Input	Instruction word received from imem
i_ImmSel	3	Input	Immediate select. (Control signal from ControlUnit select which format to decode
o_imm	32	Output	Sign-extended immediate value

Cấu trúc của immGen.sv là một bộ MUX8-1 với các ngõ vào được mô tả như code sau:

```
module immGen(
  input logic [31:0] i_instr,              // mã lệnh nhận được từ imem
  input  logic [2:0]  i_ImmSel,
  output logic [31:0] o_imm                            // giá trị imm tính được
);
// Mặc định luôn gán o_imm = 0 để tránh latch
logic [31:0] imm_val;                      // Ta sẽ dùng 1 tín hiệu trung gian
  always_comb begin
    case (i_ImmSel)
          3'b000: imm_val = {{20{i_instr[31]}}, i_instr[31:20]};//dạng I format, mở rộng bit dấu ra cho đủ 32
bit //

          3'b001: imm_val  = {{20{i_instr[31]}}, i_instr[31:20]};           //// load-I format ////
```

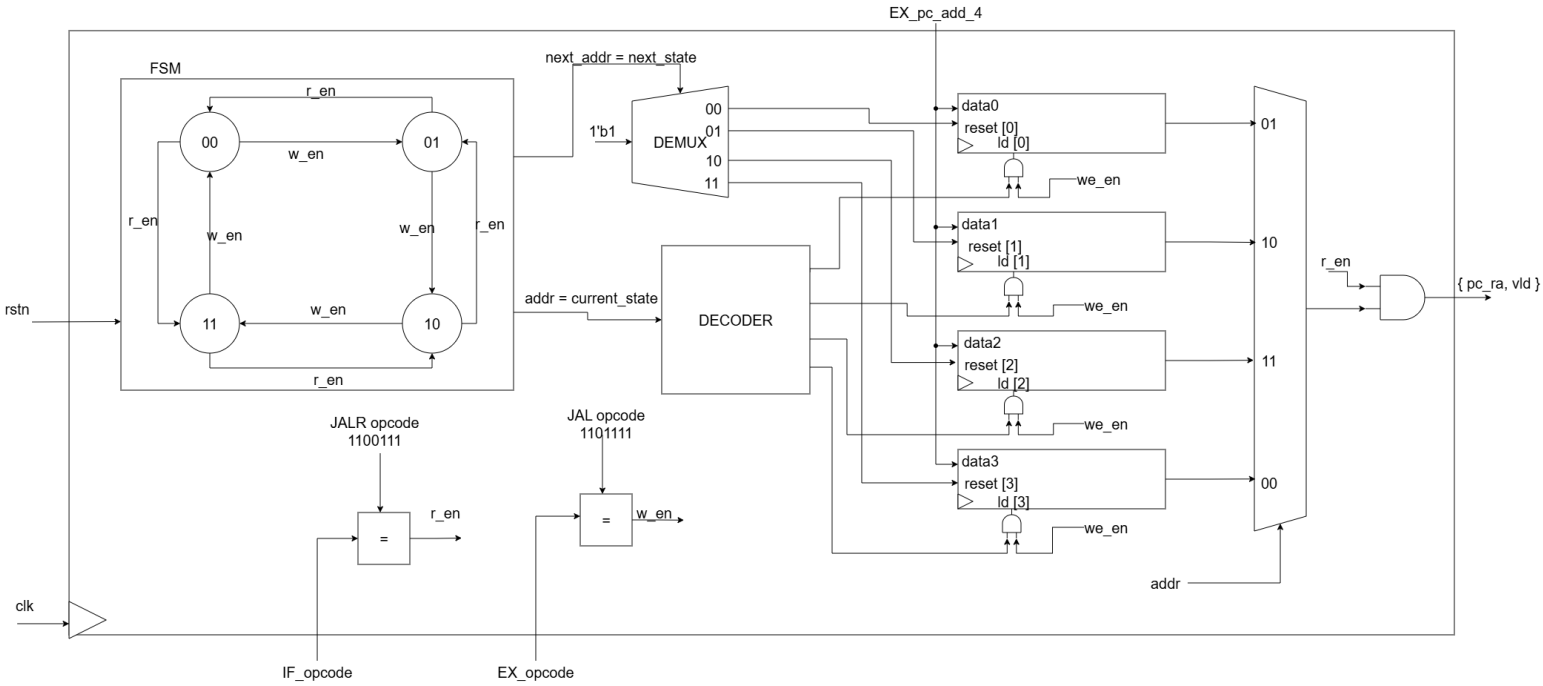
```
            3'b010: imm_val = {{20{i_instr[31]}}, i_instr[31:25], i_instr[11:7]};    //// S format: mở rộng bit
dấu và phải ghép bit  ////

            3'b011: imm_val= {{19{i_instr[31]}}, i_instr[31], i_instr[7],i_instr[30:25],i_instr[11:8], 1'b0};
//// B format ////
            3'b100: imm_val = {{11{i_instr[31]}},i_instr[31], i_instr[19:12],i_instr[20],i_instr[30:21],1'b0};
//// J format ////
            3'b101: imm_val = {i_instr [31:12], 12'b0};      //// lui-U format /////

            3'b110: imm_val = { i_instr [31:12], 12'b0 };      //// auipc-U format ////

            3'b111: imm_val = {{20{i_instr[31]}}, i_instr[31:20]};   //// jalr-I format ////

      default : imm_val = 32'd0;
    endcase
  end
  // Gán giá trị ra output
    assign o_imm = imm_val;
endmodule
```

2.16. Khối Stack_ra_unit:

Ý Tưởng chính:

JAL (Call): Ghi $PC+4$ vào Stack (Push).
JALR (Return): Lấy $PC+4$ từ Stack ra (Pop) để làm Next PC dự đoán.

Tên Chân	Chiều	Độ rộng (Bit)	Mô tả
clk	Input	1	Tín hiệu xung nhịp đồng bộ hệ thống.
rstn	Input	1	Tín hiệu Reset tích cực mức thấp (Active Low).
IF_opcode	Input	7	Mã Opcode của lệnh đang ở tầng Fetch (IF). Dùng để phát hiện sớm lệnh JALR.
EX_opcode	Input	7	Mã Opcode của lệnh đang ở tầng Execute (EX). Dùng để phát hiện lệnh JAL.
EX_pc_add_4	Input	32	Địa chỉ của lệnh tiếp theo (PC+4) từ tầng EX. Đây là địa chỉ cần lưu vào Stack để sau này quay về.
pc_ra	Output	32	Địa chỉ trả về được dự đoán (lấy ra từ đỉnh Stack).
vld	Output	1	Tín hiệu báo dữ liệu pc_ra hợp lệ (Valid). 1 nghĩa là có dự đoán, 0 là không.



Cách hoạt động:

Khối này hoạt động hình dung như một bộ nhớ Stack vòng tròn với độ sâu là 4 ngăn nhớ (data0 đến data3).

FSM:

Trung tâm của khối là một FSM quản lý con trỏ ngăn xếp (Stack Pointer).
Có 4 trạng thái: 00, 01, 10, 11 tương ứng với địa chỉ của 4 thanh ghi lưu trữ.
Hoạt động theo kiểu vòng tròn**:** Khi Stack đầy (11) mà nạp thêm, nó sẽ quay vòng về (00) và ghi đè.

Quá trình PUSH - Lưu địa chỉ trả về

Quá trình này xảy ra khi CPU thực hiện một lệnh gọi hàm (JAL).

Phát hiện**:** Khối so sánh EX_opcode với mã của lệnh **JAL** (1101111). Nếu khớp, tín hiệu w_en (Write Enable) được bật lên 1.
Cập nhật con trỏ**:** FSM nhận w_en, chuyển sang trạng thái kế tiếp (next_state).
Lưu dữ liệu**:**

- Khối **DEMUX** sử dụng địa chỉ next_addr (tương ứng next_state) để kích hoạt chân load/reset của thanh ghi đích.
- Giá trị EX_pc_add_4 (địa chỉ dòng lệnh ngay sau lệnh JAL) được ghi vào thanh ghi đó.
- *Ví dụ:* Nếu đang ở 00, w_en tích cực, FSM trỏ sang 01, dữ liệu được ghi vào thanh ghi data1.

Quá trình ĐỌC (POP) - Dự đoán địa chỉ nhảy

Quá trình này xảy ra khi CPU chuẩn bị nạp một lệnh return (JALR). Để tránh Stall pipeline, ta cần dự đoán ngay địa chỉ nhảy tại tầng Fetch (IF).

Phát hiện**:** Khối so sánh IF_opcode với mã của lệnh **JALR** (1100111). Nếu khớp, tín hiệu r_en (Read Enable) được bật lên 1.

Lấy dữ liệu**:**

Khối DECODER và MUX sử dụng địa chỉ addr (tương ứng current_state - đỉnh Stack hiện tại) để chọn dữ liệu từ một trong 4 thanh ghi (data0 - data3). Dữ liệu này chính là pc_ra (địa chỉ trả về đã lưu trước đó).

Cập nhật con trỏ**:** FSM nhận r_en, con trỏ sẽ quay lùi lại trạng thái trước đó (Pop stack).

Output**:** Tín hiệu r_en đi qua cổng AND để tạo bit vld (valid), báo hiệu gửi cho Branch Control Unit để thực hiện việc nhảy.

Nhận xét: Do chỉ có độ sâu 4 word, nên đối với chương trình con có độ sâu vượt quá 4 hoặc các chương trình đệ quy có số lần tự gọi lớn thì sẽ dễ bị dự đoán sai. Do đó, module này được thiết kế trên cơ sở nhóm em đã khảo sát toàn bộ và thống kê tất cả chương trình con trong chương trình asembly mà bộ môn cung cấp, và không có chương trình con nào có độ sâu quá 4 lệnh jal cũng như không tồn tại chương trình đệ quy nào, do đó, việc chọn độ sâu 4 word nhằm mục đích pass test bench một cách hiệu quả nhất.

3. Các model CPU:

Các khối đã xây dựng bên trên là tổng quát cho CPU always taken, do đó, để sử dụng các mô hình khác, ta chỉ cần "không nối dây" (loại bỏ) với các khối cần thiết.

3.1. Model 2: Forwarding always not taken

Model này ta chỉ cần loại bỏ các module: BTB, Stack_ra_unit

3.2. Model 3: Forwarding always taken

Model này là toàn bộ những quá trình xây dựng các khối bên trên.

4. Test bench:

Trong phần này, nhóm sẽ sử dụng một chương trình testbench riêng để kiểm tra chức năng của các khối.

4.1. ALU

Code Testbench:

```
`timescale 1ns/1ps
```

```
module alu_tb ();
// stimulus input //
  logic [31:0] i_op_a, i_op_b;
  logic [3:0]  i_alu_op;

// expected output //
  logic [31:0] o_alu_data;

// design under test //
alu DUT (
  .i_op_a     ( i_op_a ),
  .i_op_b     ( i_op_b ),
  .i_alu_op   ( i_alu_op ),
  .o_alu_data ( o_alu_data )
);

// Definitions for verification logic
  logic [31:0] expected_result;

  // ALU Opcode Parameters (phải khớp với DUT)
  localparam OP_ADD    = 4'b0000;
  localparam OP_SUB    = 4'b0001;
  localparam OP_SLT    = 4'b0010;
  localparam OP_SLTU   = 4'b0011;
  localparam OP_XOR    = 4'b0100;
  localparam OP_OR     = 4'b0101;
  localparam OP_AND    = 4'b0110;
  localparam OP_SLL    = 4'b0111;
  localparam OP_SRL    = 4'b1000;
  localparam OP_SRA    = 4'b1001;
  localparam OP_PASS_B = 4'b1111;

//// stimulus simulator ////
int seed = 14159;

initial begin
  int count = 0;  //// the corect case counter ////
  $display ("----------------------------------------");
  $display (" Starting ALU Verification Loop ");
  $display ("----------------------------------------");

  // random stimulus loop //
  // Tăng số lần lặp lên để bao phủ đủ các trường hợp ngẫu nhiên của 11 phép toán
  for ( int i=1; i < 101; i++ ) begin
    i_op_a   = $random ( seed );
    i_op_b   = $random ( seed );
    // Random opcode trong khoảng hợp lệ (0-9 hoặc 15), dùng %16 để ra 4 bit
    i_alu_op = $random ( seed ) % 16;

    #5; // Chờ DUT xử lý

    // Tính toán kết quả mong đợi (Golden Model)
```

```
    case (i_alu_op)
        OP_ADD:    expected_result = i_op_a + i_op_b;
        OP_SUB:    expected_result = i_op_a - i_op_b;
        OP_SLT:    expected_result = ($signed(i_op_a) < $signed(i_op_b)) ? 32'd1 : 32'd0;
        OP_SLTU:   expected_result = (i_op_a < i_op_b) ? 32'd1 : 32'd0;
        OP_XOR:    expected_result = i_op_a ^ i_op_b;
        OP_OR:     expected_result = i_op_a | i_op_b;
        OP_AND:    expected_result = i_op_a & i_op_b;
        OP_SLL:    expected_result = i_op_a << i_op_b[4:0];
        OP_SRL:    expected_result = i_op_a >> i_op_b[4:0];
        OP_SRA:    expected_result = $signed(i_op_a) >>> i_op_b[4:0];
        OP_PASS_B: expected_result = i_op_b;
        default:   expected_result = 32'b0; // Các opcode không định nghĩa
    endcase

    // Kiểm tra kết quả
    if ( o_alu_data == expected_result ) begin
      count++;
      // In ít thông tin hơn để dễ nhìn, hoặc in chi tiết nếu muốn giống mẫu
       $display(" #Case %0d ||| Op: %b | A: %h | B: %h ||| Out: %h | Exp: %h ||| Correct", i,
i_alu_op, i_op_a, i_op_b, o_alu_data, expected_result);
    end
    else begin
      $display(" #Case %0d ||| Op: %b | A: %h | B: %h ||| Out: %h | Exp: %h ||| Error ", i,
i_alu_op, i_op_a, i_op_b, o_alu_data, expected_result);
    end
  end

  $display ( " Number of correct cases: %0d / 100 ", count );
  $finish;
end
endmodule : alu_tb
```

```
 #Case 78 ||| Op: 0100 | A: 5d887fbb | B: 52551ba4 ||| Out: 0fdd641f | Exp: 0fdd641f ||| Correct
 #Case 79 ||| Op: 1011 | A: d21ed6a4 | B: a86a6250 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 80 ||| Op: 0100 | A: 4ff64b9f | B: d291c6a5 ||| Out: 9d678d3a | Exp: 9d678d3a ||| Correct
 #Case 81 ||| Op: 0010 | A: cf69249e | B: 91940223 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 82 ||| Op: 1001 | A: 9b209c36 | B: 7b09c9f6 ||| Out: fffffe6c | Exp: fffffe6c ||| Correct
 #Case 83 ||| Op: 0100 | A: fd1f9efa | B: eabda8d5 ||| Out: 17a2362f | Exp: 17a2362f ||| Correct
 #Case 84 ||| Op: 0010 | A: f74d6cee | B: 61e3a1c3 ||| Out: 00000001 | Exp: 00000001 ||| Correct
 #Case 85 ||| Op: 0111 | A: 21bb8743 | B: 0de2151b ||| Out: 18000000 | Exp: 18000000 ||| Correct
 #Case 86 ||| Op: 1100 | A: 8e60901c | B: 78dafbf1 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 87 ||| Op: 0000 | A: 10e22b21 | B: 29dda353 ||| Out: 3abfce74 | Exp: 3abfce74 ||| Correct
 #Case 88 ||| Op: 0100 | A: 4047d980 | B: f5764aea ||| Out: b531936a | Exp: b531936a ||| Correct
 #Case 89 ||| Op: 0010 | A: 13bb8527 | B: d5ec6aab ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 90 ||| Op: 1101 | A: 64ee31c9 | B: 19577532 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 91 ||| Op: 1010 | A: 279bdd4f | B: 7c9f63f9 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 92 ||| Op: 0001 | A: ea3dc0d4 | B: 758f67eb ||| Out: 74ae58e9 | Exp: 74ae58e9 ||| Correct
 #Case 93 ||| Op: 1011 | A: 522865a4 | B: 39507d72 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 94 ||| Op: 0101 | A: 52790fa4 | B: 3e9cbb7d ||| Out: 7efdbffd | Exp: 7efdbffd ||| Correct
 #Case 95 ||| Op: 1101 | A: 72ded9e5 | B: 24ddc149 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 96 ||| Op: 0001 | A: 3ef31d7d | B: db7d58b6 ||| Out: 6375c4c7 | Exp: 6375c4c7 ||| Correct
 #Case 97 ||| Op: 0001 | A: 4ca82799 | B: 12337b24 ||| Out: 3a74ac75 | Exp: 3a74ac75 ||| Correct
 #Case 98 ||| Op: 0101 | A: 4cee8399 | B: 38fb1371 ||| Out: 7cff93f9 | Exp: 7cff93f9 ||| Correct
 #Case 99 ||| Op: 1100 | A: 2ede055d | B: d0ed4ca1 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 #Case 100 ||| Op: 1010 | A: 062b9d0c | B: c31b4686 ||| Out: 00000000 | Exp: 00000000 ||| Correct
 Number of correct cases: 100 / 100
```

4.2. BRC

```
 `timescale 1ns/1ps
module tb ();
// stimulus input //
  logic [31:0] i_rs1_data, i_rs2_data;
  logic        i_br_un;
// expected output //
  logic        o_br_less;
  logic        o_br_equal;
// design under test //
brc_32bit DUT (
  .i_rs1_data ( i_rs1_data ),
  .i_rs2_data ( i_rs2_data ),
  .i_br_un    ( i_br_un ),
  .o_br_less  ( o_br_less ),
  .o_br_equal ( o_br_equal )
);
//// stimulus simulator ////
int seed = 14159;
initial begin
  int count = 0;  //// the corect case counter ////
  $display ("----------------------------------------");
  // random stimulus loop //
  for ( int i=0; i < 100; i++ ) begin
    i_rs1_data = $random ( seed );
    i_rs2_data = $random ( seed );
    i_br_un    = $random ( seed ); // L?y bit th?p nh?t làm giá tr? 0 ho?c 1
    #5
    // So sánh output c?a DUT v?i Logic chu?n (Golden Model)
    // Logic chu?n:
    // Less: N?u i_br_un=1 thì so sánh không d?u, ng??c l?i so sánh có d?u ($signed)
    // Equal: So sánh b?ng
    if ( { o_br_less, o_br_equal } == { (i_br_un ? (i_rs1_data < i_rs2_data) :
($signed(i_rs1_data) < $signed(i_rs2_data))), (i_rs1_data == i_rs2_data) } ) begin
      count++;
      $display(" #Case %0d ||| Simulus input: rs1 = %0d; rs2 = %0d; un = %0d ||| output: less=
%0d; equal = %0d ||| Correct ||| Number of correct cases: %0d  ", i, i_rs1_data, i_rs2_data,
i_br_un, o_br_less, o_br_equal, count);
    end
    else
      $display(" #Case %0d ||| Simulus input: rs1 = %0d; rs2 = %0d; un = %0d ||| output: less=
%0d; equal = %0d ||| Error ", i, i_rs1_data, i_rs2_data, i_br_un, o_br_less, o_br_equal);
  end
  $display ( " Number of correct cases: %0d / 100 ", count );
end
endmodule
```

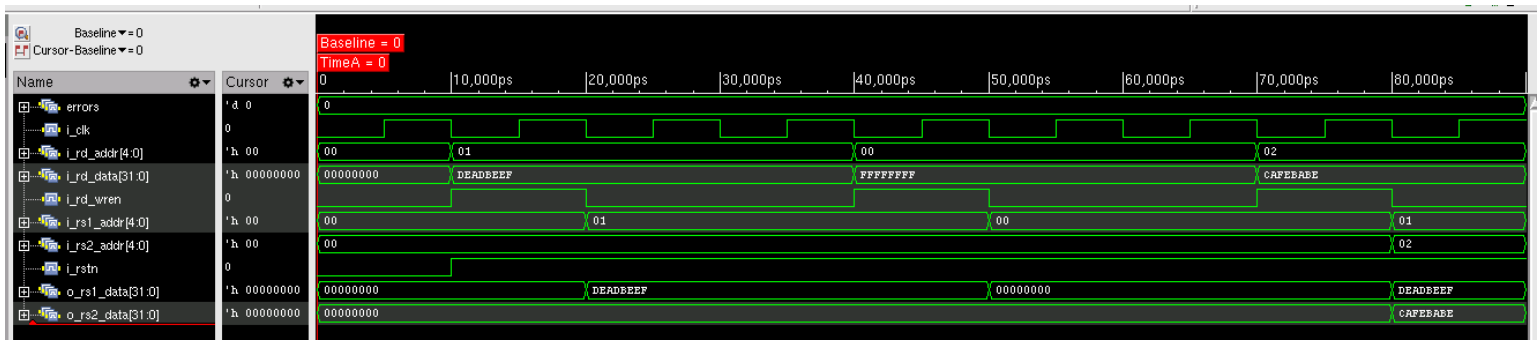
```
#Case 71 ||| Simulus input: rs1 = 4155757807; rs2 = 1324199325; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 72
#Case 72 ||| Simulus input: rs1 = 2243553291; rs2 = 1839571931; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 73
#Case 73 ||| Simulus input: rs1 = 1233659795; rs2 = 4040748769; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 74
#Case 74 ||| Simulus input: rs1 = 3699584185; rs2 = 1767931346; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 75
#Case 75 ||| Simulus input: rs1 = 3951165143; rs2 = 763352922; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 76
#Case 76 ||| Simulus input: rs1 = 696769875; rs2 = 47744261; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 77
#Case 77 ||| Simulus input: rs1 = 1569226683; rs2 = 1381309348; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 78
#Case 78 ||| Simulus input: rs1 = 3525236388; rs2 = 2825544272; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 79
#Case 79 ||| Simulus input: rs1 = 1341541279; rs2 = 3532768933; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 80
#Case 80 ||| Simulus input: rs1 = 3479774366; rs2 = 2442396195; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 81
#Case 81 ||| Simulus input: rs1 = 2602605622; rs2 = 2064239094; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 82
#Case 82 ||| Simulus input: rs1 = 4246707962; rs2 = 3938298069; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 83
#Case 83 ||| Simulus input: rs1 = 4149046510; rs2 = 1642308035; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 84
#Case 84 ||| Simulus input: rs1 = 565937987; rs2 = 232920347; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 85
#Case 85 ||| Simulus input: rs1 = 2388693020; rs2 = 2027617265; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 86
#Case 86 ||| Simulus input: rs1 = 283257633; rs2 = 702391123; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 87
#Case 87 ||| Simulus input: rs1 = 1078450560; rs2 = 4118170346; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 88
#Case 88 ||| Simulus input: rs1 = 331056423; rs2 = 3589040811; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 89
#Case 89 ||| Simulus input: rs1 = 1693331913; rs2 = 425162034; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 90
#Case 90 ||| Simulus input: rs1 = 664526159; rs2 = 2090820601; un = 0 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 91
#Case 91 ||| Simulus input: rs1 = 3929915604; rs2 = 1972332523; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 92
#Case 92 ||| Simulus input: rs1 = 1378379172; rs2 = 961576306; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 93
#Case 93 ||| Simulus input: rs1 = 1383665572; rs2 = 1050459005; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 94
#Case 94 ||| Simulus input: rs1 = 1927207397; rs2 = 618512713; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 95
#Case 95 ||| Simulus input: rs1 = 1056120189; rs2 = 3682425014; un = 1 ||| output: less= 1; equal = 0 ||| Correct ||| Number of correct cases: 96
#Case 96 ||| Simulus input: rs1 = 1286088601; rs2 = 305363748; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 97
#Case 97 ||| Simulus input: rs1 = 1290699673; rs2 = 955978609; un = 1 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 98
#Case 98 ||| Simulus input: rs1 = 786302301; rs2 = 3505212577; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 99
#Case 99 ||| Simulus input: rs1 = 103521548; rs2 = 3273344646; un = 0 ||| output: less= 0; equal = 0 ||| Correct ||| Number of correct cases: 100
Number of correct cases: 100 / 100
```

3.3. REGFILE

```
`timescale 1ns/1ps

module tb();

  // 1. Khai báo tín hiệu kết nối với DUT
  logic        i_clk;
  logic        i_rstn;

  // Write Port
  logic        i_rd_wren;
  logic [4:0]  i_rd_addr;
  logic [31:0] i_rd_data;

  // Read Port 1
  logic [4:0]  i_rs1_addr;
  logic [31:0] o_rs1_data;

  // Read Port 2
  logic [4:0]  i_rs2_addr;
  logic [31:0] o_rs2_data;

  // 2. Khởi tạo DUT (Device Under Test)
  regfile DUT (
    .i_clk      ( i_clk ),
    .i_rstn     ( i_rstn ),
    .i_rd_wren  ( i_rd_wren ),
    .i_rd_addr  ( i_rd_addr ),
    .i_rd_data  ( i_rd_data ),
    .i_rs1_addr ( i_rs1_addr ),
    .o_rs1_data ( o_rs1_data ),
    .i_rs2_addr ( i_rs2_addr ),
    .o_rs2_data ( o_rs2_data )
  );
```

```
initial begin
  $dumpfile("tb.vcd");
  $dumpvars(1,tb );
end
  // 3. Tạo xung Clock (Chu kỳ 10ns)
    always #5 i_clk = ~i_clk;
  // 4. Biến đếm lỗi
  int errors = 0;

  // 5. Kịch bản kiểm tra (Test Scenario)
  initial begin
    // --- Khởi tạo ban đầu ---
    i_rstn      = 0;
    i_rd_wren   = 0;
    i_rd_addr   = 0;
    i_rd_data   = 0;
    i_rs1_addr  = 0;
    i_rs2_addr  = 0;
    i_clk =0 ;
    #10;
    // ============================================================
    // CASE 1: Ghi và Đọc chuẩn (Standard Write/Read Test)
    // ============================================================
    i_rstn = 1;
    // Ghi 0xDEADBEEF vào thanh ghi R1
    i_rd_wren = 1;
    i_rd_addr = 5'd1;
    i_rd_data = 32'hDEADBEEF;

    #10;
    i_rd_wren = 0;       // Tắt ghi

    i_rs1_addr = 5'd1;
    #10;


    // ============================================================
    // CASE 2: Bảo vệ ghi vào R0 (Write to R0 Protection Test)
    // ============================================================

    // Cố tình ghi giá trị khác 0 vào R0
    #10;
    i_rd_wren = 1;
    i_rd_addr = 5'd0;
    i_rd_data = 32'hFFFFFFFF; // Giá trị toàn 1


    #10;
    i_rd_wren = 0;

    // Đọc lại R0
    i_rs1_addr = 5'd0;
    #10;
```

```
    // ============================================================
    // CASE 3: Đọc đồng thời 2 cổng (Dual Port Read Test)
    // ============================================================

    // Chuẩn bị dữ liệu: Ghi vào R2 trước (R1 đã có dữ liệu từ Case 2)
    #10;
    i_rd_wren = 1;
    i_rd_addr = 5'd2;
    i_rd_data = 32'hCAFEBABE;

    #10;
    i_rd_wren = 0;

    // Thực hiện đọc đồng thời: Port 1 đọc R1, Port 2 đọc R2
    i_rs1_addr = 5'd1;
    i_rs2_addr = 5'd2;
    #10;
    $finish;
  end
endmodule
```



Case 1: Ta thực hiện ghi giá trị 32'hDEADBEEF vào địa chỉ x1, sau đó tiền hành đọc lại giá trị này cũng từ địa chỉ x1.

Case 2: Ta cố tính ghi giá trị vào x0, dựa vào dạng sóng, x0 luôn load ra giá trị 32’d0.

Case 3: Ta ghi vào x2 giá trị 32'hCAFEBABE và sau đó cùng lúc đọc cả hai thanh ghi x1 và x2.

3.4. LSU

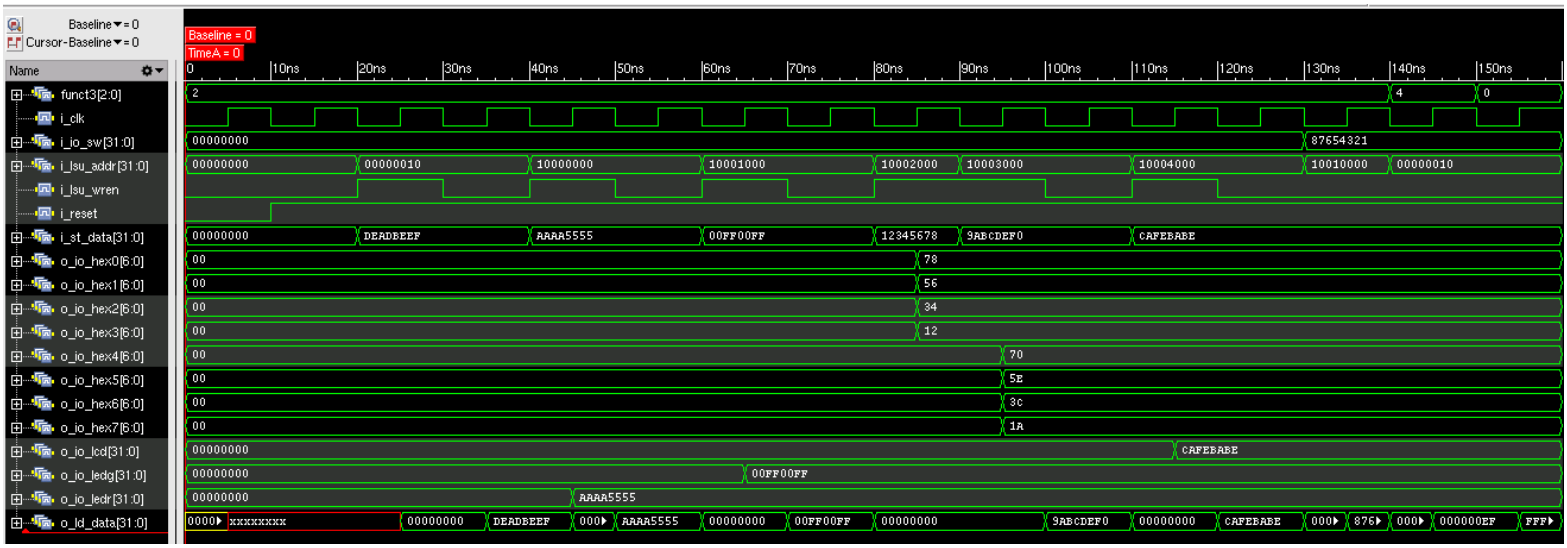
```
module tb;

  // 1. Khai báo tín hiệu kết nối với DUT
  logic        i_clk;
```

```
logic        i_reset;
logic [31:0] i_lsu_addr;
logic [31:0] i_st_data;
logic        i_lsu_wren;
logic [2:0]  funct3;
logic [31:0] i_io_sw;

logic [31:0] o_ld_data;
logic [31:0] o_io_ledr;
logic [31:0] o_io_ledg;
logic [6:0]  o_io_hex0, o_io_hex1, o_io_hex2, o_io_hex3,
             o_io_hex4, o_io_hex5, o_io_hex6, o_io_hex7;
logic [31:0] o_io_lcd;

// 2. Khởi tạo DUT (Device Under Test)
// Lưu ý: Tên port phải khớp chính xác với module lsu
lsu DUT (
  .i_clk      ( i_clk ),
  .i_rstn     ( i_reset ),     // Kết nối tín hiệu reset (TB: i_reset -> DUT: i_rstn)
  .i_lsu_addr ( i_lsu_addr ),
  .i_st_data  ( i_st_data ),
  .i_lsu_wren ( i_lsu_wren ),
  .funct3     ( funct3 ),
  .i_io_sw    ( i_io_sw ),
  .o_ld_data  ( o_ld_data ),
  .o_io_ledr  ( o_io_ledr ),
  .o_io_ledg  ( o_io_ledg ),
  .o_io_hex0  ( o_io_hex0 ), .o_io_hex1 ( o_io_hex1 ),
  .o_io_hex2  ( o_io_hex2 ), .o_io_hex3 ( o_io_hex3 ),
  .o_io_hex4  ( o_io_hex4 ), .o_io_hex5 ( o_io_hex5 ),
  .o_io_hex6  ( o_io_hex6 ), .o_io_hex7 ( o_io_hex7 ),
  .o_io_lcd   ( o_io_lcd )
);

// 3. Tạo xung Clock (Chu kỳ 10ns)
initial begin
  $dumpfile("tb.vcd");
  $dumpvars(0, tb );
end
  always #5 i_clk = ~i_clk;

// 4. Kịch bản kiểm tra (Test Scenario)
initial begin
  // --- Khởi tạo ban đầu ---
  i_reset    = 0; // Active low reset (i_rstn = 0)
  i_lsu_addr = 0;
  i_st_data  = 0;
  i_lsu_wren = 0;
  funct3     = 3'b010; // Word access
  i_io_sw    = 0;
  i_clk      = 0;

  #10;
```

```
i_reset    = 1; // Release reset (i_rstn = 1)
#10;

// ============================================================
// CASE 1: Ghi và Đọc DMEM (Word Access)
// ============================================================
// DMEM Base: 0x0000_0000

// Ghi vào DMEM
i_lsu_addr = 32'h0000_0010;
i_st_data  = 32'hDEADBEEF;
i_lsu_wren = 1;             // Write
funct3     = 3'b010;        // Word

#10;
i_lsu_wren = 0;   // Tắt ghi

// Đọc lại (Cần đợi 1 chu kỳ pipeline)
#10;

// ============================================================
// CASE 2: Ghi LEDR (Red LEDs)
// ============================================================
// Base: 0x1000_0000

i_lsu_addr = 32'h1000_0000;
i_st_data  = 32'hAAAA_5555;
i_lsu_wren = 1;
funct3     = 3'b010;

#10;
i_lsu_wren = 0;
#10;

// ============================================================
// CASE 3: Ghi LEDG (Green LEDs)
// ============================================================
// Base: 0x1000_1000

i_lsu_addr = 32'h1000_1000;
i_st_data  = 32'h00FF_00FF;
i_lsu_wren = 1;

#10;
i_lsu_wren = 0;
#10;

// ============================================================
// CASE 4: Ghi 7-Segment LEDs (HEX)
// ============================================================
// HEX3-0 Base: 0x1000_2000
i_lsu_addr = 32'h1000_2000;
i_st_data  = 32'h1234_5678; // Data cho HEX 3,2,1,0
```

```
i_lsu_wren = 1;
#10;

// HEX7-4 Base: 0x1000_3000
i_lsu_addr = 32'h1000_3000;
i_st_data  = 32'h9ABC_DEF0; // Data cho HEX 7,6,5,4
i_lsu_wren = 1;
#10;

i_lsu_wren = 0;
#10;

// ============================================================
// CASE 5: Ghi LCD
// ============================================================
// Base: 0x1000_4000

i_lsu_addr = 32'h1000_4000;
i_st_data  = 32'hCAFE_BABE;
i_lsu_wren = 1;

#10;
i_lsu_wren = 0;
#10;

// ============================================================
// CASE 6: Đọc Switches (Input Peripheral)
// ============================================================
// Base: 0x1001_0000

// Giả lập tác động vào công tắc
i_io_sw = 32'h8765_4321;

i_lsu_addr = 32'h1001_0000;
i_lsu_wren = 0; // Read
funct3     = 3'b010;

#10;
// Kiểm tra waveform o_ld_data tại thời điểm này

// ============================================================
// CASE 7: Byte Access (Load Byte Signed/Unsigned)
// ============================================================
// Sử dụng lại dữ liệu DEADBEEF tại 0x0000_0010 trong DMEM
// Byte 0 (LSB) là EF (1110_1111) -> Số âm nếu signed

i_lsu_addr = 32'h0000_0010;
i_lsu_wren = 0;

// Test LBU (Load Byte Unsigned) - funct3 = 100
funct3 = 3'b100;
#10;
// Kỳ vọng: 000000EF
```

```
    // Test LB (Load Byte Signed) - funct3 = 000
    funct3 = 3'b000;
    #10;
    // Kỳ vọng: FFFFFFEF (Sign extension)

    $finish;
  end

endmodule
```



Biểu diễn qua 4 case:

CASE 1: Truy cập Bộ nhớ Dữ liệu (DMEM) - Word Access

Mục đích: Kiểm tra khả năng Ghi và Đọc dữ liệu 32-bit (Word) vào vùng nhớ chính (Data Memory).

1. Ghi (Store):

Tín hiệu vào:

- i_lsu_addr = 0x0000_0010 (Địa chỉ thuộc vùng DMEM).
- i_st_data = 0xDEADBEEF (Giá trị cần ghi).
- i_lsu_wren = 1 (Ra lệnh ghi).
- funct3 = 010 (Word access).

Sóng: Tại cạnh lên của Clock, LSU giải mã địa chỉ, thấy nằm trong vùng DMEM và kích hoạt tín hiệu ghi vào module dmem.

Kết quả: Tại địa chỉ 0x10 trong dmem sẽ lưu giá trị 0xDEADBEEF.

2. Đọc (Load):

Tín hiệu vào:

i_lsu_wren chuyển xuống 0 (Ra lệnh đọc).
i_lsu_addr vẫn giữ 0x0000_0010.

Dạng sóng:

Chu kỳ T: Địa chỉ được gửi vào.
Chu kỳ T+1: Dữ liệu từ dmem được lấy ra, đi qua bộ MUX chọn nguồn, rồi đi ra o_ld_data.

Tín hiệu ra: o_ld_data sẽ chuyển thành 0xDEADBEEF.

CASE 2 & 3: Ghi ra đèn LED (Output Peripherals)

Mục đích: Kiểm tra cơ chế Memory Mapped I/O (Ánh xạ ngoại vi vào bộ nhớ). Ghi vào địa chỉ cụ thể sẽ làm sáng đèn.

Case 2: LEDR (Đèn đỏ)

Tín hiệu vào: i_lsu_addr = 0x1000_0000, i_st_data = 0xAAAA_5555, wren = 1.
Dạng song: Bộ giải mã địa chỉ phát hiện dải 0x1000_0000 thuộc về io_ledr.
Tín hiệu ra: Ngõ ra o_io_ledr của LSU sẽ thay đổi thành 0xAAAA_5555.

Case 3: LEDG (Đèn xanh)

Tín hiệu vào: i_lsu_addr = 0x1000_1000, i_st_data = 0x00FF_00FF.
Tín hiệu ra: Ngõ ra o_io_ledg sẽ thay đổi thành 0x00FF_00FF.

CASE 4: Ghi ra LED 7 thanh (HEX)

Mục đích: Kiểm tra việc ghi dữ liệu vào các thanh ghi điều khiển LED 7 đoạn.

Lần 1: Ghi vào 0x1000_2000 (HEX 3-0) giá trị 0x1234_5678.
Tín hiệu ra: Các ngõ ra o_io_hex0 đến o_io_hex3 sẽ cập nhật giá trị tương ứng với các byte của dữ liệu vào.

Lần 2: Ghi vào 0x1000_3000 (HEX 7-4) giá trị 0x9ABC_DEF0.
Tín hiệu ra: Các ngõ ra o_io_hex4 đến o_io_hex7 sẽ cập nhật.

CASE 5: Ghi ra màn hình LCD

Mục đích: Tương tự như LED, kiểm tra vùng nhớ LCD.

Tín hiệu vào: i_lsu_addr = 0x1000_4000, data = 0xCAFE_BABE.
Tín hiệu ra: Ngõ ra o_io_lcd sẽ giữ giá trị 0xCAFE_BABE.

CASE 6: Đọc Công tắc (Input Peripheral - Switches)

Mục đích: Kiểm tra khả năng đọc dữ liệu từ môi trường bên ngoài vào CPU.

Tín hiệu vào:

i_io_sw = 0x8765_4321 (Giả lập người dùng gạt công tắc).
i_lsu_addr = 0x1001_0000 (Địa chỉ của Switch).

i_lsu_wren = 0 (Đọc).

Dạng sóng:

LSU giải mã địa chỉ 0x1001_0000, MUX trong LSU sẽ chọn kênh đầu vào là i_io_sw thay vì dmem hay các nguồn khác.

Tín hiệu ra: Sau 1 chu kỳ clock, o_ld_data sẽ bằng 0x8765_4321.

CASE 7: Truy cập Byte và Mở rộng dấu (Quan trọng)

Mục đích: Kiểm tra funct3 hoạt động có đúng không khi đọc dữ liệu nhỏ hơn 32-bit (Byte/Half-word) và xử lý dấu (Sign Extension).

Dữ liệu có sẵn: Tại địa chỉ 0x10 đang chứa 0xDEADBEEF.
Byte thấp nhất (LSB): Tại địa chỉ 0x10 chính là byte 0xEF (Nhị phân: 1110_1111).
Lưu ý: Bit cao nhất của byte này là 1 (bit dấu âm).

1. Test LBU (Load Byte Unsigned - funct3 = 100):

Yêu cầu: Đọc 1 byte, phần còn lại điền số 0.
Dạng sóng: LSU lấy 0xEF, các bit từ 8-31 được gán bằng 0.
Kết quả: o_ld_data = 0x0000_00EF.

2. Test LB (Load Byte Signed - funct3 = 000):

Yêu cầu: Đọc 1 byte, phần còn lại mở rộng theo bit dấu (bit 7 của byte đó).
Dạng sóng: LSU lấy 0xEF. Vì bit 7 là 1, nó sẽ điền toàn bộ các bit cao (8-31) bằng 1.
Kết quả: o_ld_data = 0xFFFF_FFEF. (Đây là giá trị số âm tương đương trong hệ 32-bit bù 2).

3.5. CONTROl UNIT

```
`timescale 1ns/1ps

module tb();

  // 1. Khai báo tín hiệu kết nối với DUT
  logic [31:0] i_instr;
  logic        i_br_equal;
  logic        i_br_less;

  logic        o_pc_sel;
  logic        o_rd_wren;
  logic        o_insn_vld;
```

```
logic        o_br_un;
logic        o_opa_sel;
logic        o_opb_sel;
logic [3:0]  o_alu_op;
logic        o_mem_wren;
logic [1:0]  o_wb_sel;
logic [2:0]  o_ImmSel;

// 2. Khởi tạo DUT (Device Under Test)
ControlUnit DUT (
  .i_instr    ( i_instr ),
  .i_br_equal ( i_br_equal ),
  .i_br_less  ( i_br_less ),
  .o_pc_sel   ( o_pc_sel ),
  .o_rd_wren  ( o_rd_wren ),
  .o_insn_vld ( o_insn_vld ),
  .o_br_un    ( o_br_un ),
  .o_opa_sel  ( o_opa_sel ),
  .o_opb_sel  ( o_opb_sel ),
  .o_alu_op   ( o_alu_op ),
  .o_mem_wren ( o_mem_wren ),
  .o_wb_sel   ( o_wb_sel ),
  .o_ImmSel   ( o_ImmSel )
);

// 3. Các biến cho Golden Model (Kết quả mong đợi)
logic        exp_pc_sel;
logic        exp_rd_wren;
logic        exp_insn_vld;
logic        exp_br_un;
logic        exp_opa_sel;
logic        exp_opb_sel;
logic [3:0]  exp_alu_op;
logic        exp_mem_wren;
logic [1:0]  exp_wb_sel;
logic [2:0]  exp_ImmSel;

// Các trường trích xuất từ instruction để tính toán
logic [6:0] opcode;
logic [2:0] funct3;
logic       funct7b5;

// ALU Opcode Parameters (Phải khớp với DUT)
localparam OP_ADD    = 4'b0000;
localparam OP_SUB    = 4'b0001;
localparam OP_SLT    = 4'b0010;
localparam OP_SLTU   = 4'b0011;
localparam OP_XOR    = 4'b0100;
localparam OP_OR     = 4'b0101;
localparam OP_AND    = 4'b0110;
localparam OP_SLL    = 4'b0111;
localparam OP_SRL    = 4'b1000;
localparam OP_SRA    = 4'b1001;
```

```
localparam OP_PASS_B = 4'b1111;

// Opcodes
localparam OPCODE_R_TYPE = 7'b0110011;
localparam OPCODE_I_TYPE = 7'b0010011;
localparam OPCODE_LOAD   = 7'b0000011;
localparam OPCODE_STORE  = 7'b0100011;
localparam OPCODE_BRANCH = 7'b1100011;
localparam OPCODE_JALR   = 7'b1100111;
localparam OPCODE_JAL    = 7'b1101111;
localparam OPCODE_AUIPC  = 7'b0010111;
localparam OPCODE_LUI    = 7'b0110111;

//// stimulus simulator ////
int seed = 14159;

// Task để tạo Instruction ngẫu nhiên hợp lệ
task generate_random_instr;
  logic [6:0] rand_opcode;
  logic [2:0] rand_funct3;
  logic       rand_funct7b5;
  int         type_selector;

  type_selector = $random(seed) % 9; // Chọn 1 trong 9 loại lệnh

  case(type_selector)
    0: rand_opcode = OPCODE_R_TYPE;
    1: rand_opcode = OPCODE_I_TYPE;
    2: rand_opcode = OPCODE_LOAD;
    3: rand_opcode = OPCODE_STORE;
    4: rand_opcode = OPCODE_BRANCH;
    5: rand_opcode = OPCODE_JALR;
    6: rand_opcode = OPCODE_JAL;
    7: rand_opcode = OPCODE_AUIPC;
    8: rand_opcode = OPCODE_LUI;
    default: rand_opcode = OPCODE_R_TYPE;
  endcase

  rand_funct3   = $random(seed);
  rand_funct7b5 = $random(seed);

  // Ghép thành lệnh 32-bit (các phần khác random)
  i_instr = {1'b0, rand_funct7b5, 5'b0, 5'b0, 5'b0, rand_funct3, 5'b0, rand_opcode};
  // Lưu ý: Chỉ quan tâm opcode, funct3, funct7[5] cho Control Unit
  // Các trường rs1, rs2, rd, imm có thể random nhưng không ảnh hưởng logic này
  i_instr[31:25] = {1'b0, rand_funct7b5, 5'b0}; // Set funct7
  i_instr[24:20] = $random(seed); // rs2
  i_instr[19:15] = $random(seed); // rs1
  i_instr[11:7]  = $random(seed); // rd
endtask


initial begin
```

```
    int count = 0;  //// the corect case counter ////
    $display ("----------------------------------------");
    $display (" Starting ControlUnit Verification Loop ");
    $display ("----------------------------------------");

    // random stimulus loop //
    for ( int i=0; i < 100; i++ ) begin

      // 1. Tạo Stimulus
      generate_random_instr();
      i_br_equal = $random(seed);
      i_br_less  = $random(seed);

      #5; // Chờ DUT xử lý

      // 2. Tính toán kết quả mong đợi (Golden Model - Logic y hệt DUT)
      opcode   = i_instr[6:0];
      funct3   = i_instr[14:12];
      funct7b5 = i_instr[30];

      // Reset expected values
      exp_pc_sel = 0; exp_rd_wren = 0; exp_insn_vld = 0; exp_br_un = 0;
      exp_opa_sel = 0; exp_opb_sel = 0; exp_mem_wren = 0; exp_wb_sel = 0;
      exp_ImmSel = 0; exp_alu_op = OP_ADD;

      // Logic giải mã
      case (opcode)
        OPCODE_R_TYPE: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 0; exp_wb_sel = 0;
          case (funct3)
            3'b000: exp_alu_op = (funct7b5) ? OP_SUB : OP_ADD;
            3'b001: exp_alu_op = OP_SLL;
            3'b010: exp_alu_op = OP_SLT;
            3'b011: exp_alu_op = OP_SLTU;
            3'b100: exp_alu_op = OP_XOR;
            3'b101: exp_alu_op = (funct7b5) ? OP_SRA : OP_SRL;
            3'b110: exp_alu_op = OP_OR;
            3'b111: exp_alu_op = OP_AND;
            default: exp_alu_op = OP_ADD;
          endcase
        end
        OPCODE_I_TYPE: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b000;
          case (funct3)
            3'b000: exp_alu_op = OP_ADD;
            3'b010: exp_alu_op = OP_SLT;
            3'b011: exp_alu_op = OP_SLTU;
            3'b100: exp_alu_op = OP_XOR;
            3'b110: exp_alu_op = OP_OR;
            3'b111: exp_alu_op = OP_AND;
            3'b001: exp_alu_op = OP_SLL;
            3'b101: exp_alu_op = (funct7b5) ? OP_SRA : OP_SRL;
```

```
            default: exp_alu_op = OP_ADD;
          endcase
        end
        OPCODE_LOAD: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 1; exp_ImmSel =
3'b001;
          exp_alu_op = OP_ADD;
        end
        OPCODE_STORE: begin
          exp_mem_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_ImmSel = 3'b010;
          exp_alu_op = OP_ADD;
        end
        OPCODE_BRANCH: begin
          exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b011;
          exp_br_un = funct3[1]; // Bit 13 is bit 1 of funct3
          exp_alu_op = OP_ADD;
          // Branch Logic
          case (funct3)
            3'b000: exp_pc_sel = i_br_equal;
            3'b001: exp_pc_sel = !i_br_equal;
            3'b100: exp_pc_sel = i_br_less;
            3'b101: exp_pc_sel = i_br_equal | !i_br_less;
            3'b110: exp_pc_sel = i_br_less;
            3'b111: exp_pc_sel = i_br_equal | !i_br_less;
            default: exp_pc_sel = 0;
          endcase
        end
        OPCODE_JAL: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 2;
exp_ImmSel = 3'b100;
          exp_pc_sel = 1; exp_alu_op = OP_ADD;
        end
        OPCODE_JALR: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 2; exp_ImmSel =
3'b111;
          exp_pc_sel = 1; exp_alu_op = OP_ADD;
        end
        OPCODE_LUI: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opb_sel = 1; exp_wb_sel = 0; exp_ImmSel =
3'b101;
          exp_alu_op = OP_PASS_B;
        end
        OPCODE_AUIPC: begin
          exp_rd_wren = 1; exp_insn_vld = 1; exp_opa_sel = 1; exp_opb_sel = 1; exp_wb_sel = 0;
exp_ImmSel = 3'b110;
          exp_alu_op = OP_ADD;
        end
        default: ;
      endcase


      // 3. So sánh kết quả
```

```
      if ( {o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op, o_mem_wren,
o_wb_sel, o_ImmSel} ==
           {exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel} )
      begin
        count++;
        $display(" #Case %0d ||| Opcode: %b | ALU_OP: %b ||| Correct", i, opcode, o_alu_op);
         $display("     Expected: PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel);
        $display("     Actual:   PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op,
o_mem_wren, o_wb_sel, o_ImmSel);

      end
      else begin
        $display(" #Case %0d ||| Opcode: %b | F3: %b | F7: %b ||| ERROR", i, opcode, funct3,
funct7b5);
        $display("     Expected: PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  exp_pc_sel, exp_rd_wren, exp_insn_vld, exp_br_un, exp_opa_sel, exp_opb_sel,
exp_alu_op, exp_mem_wren, exp_wb_sel, exp_ImmSel);
        $display("     Actual:   PC=%b RD=%b VLD=%b BrU=%b OpA=%b OpB=%b ALU=%b Mem=%b WB=%b
Imm=%b",
                  o_pc_sel, o_rd_wren, o_insn_vld, o_br_un, o_opa_sel, o_opb_sel, o_alu_op,
o_mem_wren, o_wb_sel, o_ImmSel);
      end
    end

    $display ( " Number of correct cases: %0d / 100 ", count );
    $finish;
  end

endmodule
```

```
#       Actual:   PC=1 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=10 Imm=100
#   #Case 92 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   #Case 93 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   #Case 94 ||| Opcode: 0000011 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=0 WB=01 Imm=001
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=0 WB=01 Imm=001
#   #Case 95 ||| Opcode: 0110011 | ALU_OP: 0100 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#   #Case 96 ||| Opcode: 0100011 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=0 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=1 WB=00 Imm=010
#       Actual:   PC=0 RD=0 VLD=1 BrU=0 OpA=0 OpB=1 ALU=0000 Mem=1 WB=00 Imm=010
#   #Case 97 ||| Opcode: 0110011 | ALU_OP: 0100 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0100 Mem=0 WB=00 Imm=000
#   #Case 98 ||| Opcode: 0010111 | ALU_OP: 0000 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=00 Imm=110
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=1 OpB=1 ALU=0000 Mem=0 WB=00 Imm=110
#   #Case 99 ||| Opcode: 0110011 | ALU_OP: 0011 ||| Correct
#       Expected: PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#       Actual:   PC=0 RD=1 VLD=1 BrU=0 OpA=0 OpB=0 ALU=0011 Mem=0 WB=00 Imm=000
#   Number of correct cases: 100 / 100
```

3.6. IMMEDIATE GENERATION

```
`timescale 1ns/1ps

module tb();

  // 1. Khai báo tín hi?u k?t n?i v?i DUT
  logic [31:0] i_instr;
  logic [2:0]  i_ImmSel;
  logic [31:0] o_imm;

  // 2. Kh?i t?o DUT (Device Under Test)
  immGen DUT (
    .i_instr  ( i_instr ),
    .i_ImmSel ( i_ImmSel ),
    .o_imm    ( o_imm )
  );

  // 3. Bi?n cho Golden Model (K?t qu? mong ??i)
  logic [31:0] exp_imm;

  // ??nh ngh?a các h?ng s? cho ImmSel ?? code d? ??c (tùy ch?n)
  localparam IMM_I     = 3'b000;
  localparam IMM_LOAD  = 3'b001;
  localparam IMM_S     = 3'b010;
  localparam IMM_B     = 3'b011;
  localparam IMM_J     = 3'b100;
  localparam IMM_LUI   = 3'b101;
```

```
  localparam IMM_AUIPC = 3'b110;
  localparam IMM_JALR  = 3'b111;

  //// Stimulus simulator ////
  int seed = 14159;

  initial begin
    int count = 0;  //// Bi?n ??m s? case ?úng ////
    $display ("----------------------------------------");
    $display (" Starting ImmGen Verification Loop ");
    $display ("----------------------------------------");

    // Vòng l?p t?o stimulus ng?u nhiên
    for ( int i = 0; i < 100; i++ ) begin

      // 1. T?o Input Ng?u nhiên
      i_instr  = $random(seed);      // T?o l?nh 32-bit ng?u nhiên
      i_ImmSel = $random(seed) % 8;  // T?o ImmSel ng?u nhiên t? 0-7

      #5; // Ch? DUT x? lý (logic t? h?p nên r?t nhanh)

      // 2. Tính toán k?t qu? mong ??i (Golden Model)
      // Sao chép logic gi?i mã t? thi?t k? ?? ki?m tra
      case (i_ImmSel)
        IMM_I:     exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        IMM_LOAD:  exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        IMM_S:     exp_imm = {{20{i_instr[31]}}, i_instr[31:25], i_instr[11:7]};
        // B-type: Bit 0 luôn là 0, các bit khác ???c s?p x?p l?i t? instruction
        IMM_B:     exp_imm = {{19{i_instr[31]}}, i_instr[31], i_instr[7], i_instr[30:25],
i_instr[11:8], 1'b0};
        // J-type: Bit 0 luôn là 0
        IMM_J:     exp_imm = {{11{i_instr[31]}}, i_instr[31], i_instr[19:12], i_instr[20],
i_instr[30:21], 1'b0};
        IMM_LUI:   exp_imm = {i_instr[31:12], 12'b0};
        IMM_AUIPC: exp_imm = {i_instr[31:12], 12'b0};
        IMM_JALR:  exp_imm = {{20{i_instr[31]}}, i_instr[31:20]};
        default:   exp_imm = 32'd0;
      endcase

      // 3. So sánh k?t qu?
      if (o_imm == exp_imm) begin
        count++;
        // Có th? uncomment dòng d??i n?u mu?n xem chi ti?t các case ?úng
         $display(" #Case %0d ||| Sel: %b | Instr: %h ||| Correct: %h", i, i_ImmSel, i_instr,
o_imm);
        $display("     Expected: %h", exp_imm);
        $display("     Actual:   %h", o_imm);
      end else begin
        $display(" #Case %0d ||| Sel: %b | Instr: %h ||| ERROR", i, i_ImmSel, i_instr);
        $display("     Expected: %h", exp_imm);
        $display("     Actual:   %h", o_imm);
      end
```

```
    end

    $display ( " Number of correct cases: %0d / 100 ", count );
    $finish;
  end

endmodule
```

```
#       Expected: 00000016
#       Actual:   00000016
#  #Case 92 ||| Sel: 010 | Instr: e61c86cc ||| Correct: fffffe6d
#       Expected: fffffe6d
#       Actual:   fffffe6d
#  #Case 93 ||| Sel: 011 | Instr: c34fc086 ||| Correct: fffffc20
#       Expected: fffffc20
#       Actual:   fffffc20
#  #Case 94 ||| Sel: 110 | Instr: 507409a0 ||| Correct: 50740000
#       Expected: 50740000
#       Actual:   50740000
#  #Case 95 ||| Sel: 110 | Instr: a0b3de41 ||| Correct: a0b3d000
#       Expected: a0b3d000
#       Actual:   a0b3d000
#  #Case 96 ||| Sel: 101 | Instr: 30a2af61 ||| Correct: 30a2a000
#       Expected: 30a2a000
#       Actual:   30a2a000
#  #Case 97 ||| Sel: 011 | Instr: db69b0b6 ||| Correct: fffffda0
#       Expected: fffffda0
#       Actual:   fffffda0
#  #Case 98 ||| Sel: 110 | Instr: ce37c89c ||| Correct: ce37c000
#       Expected: ce37c000
#       Actual:   ce37c000
#  #Case 99 ||| Sel: 011 | Instr: 166c652c ||| Correct: 0000016a
#       Expected: 0000016a
#       Actual:   0000016a
#  Number of correct cases: 100 / 100
```

3.7. Branch Control Unit:

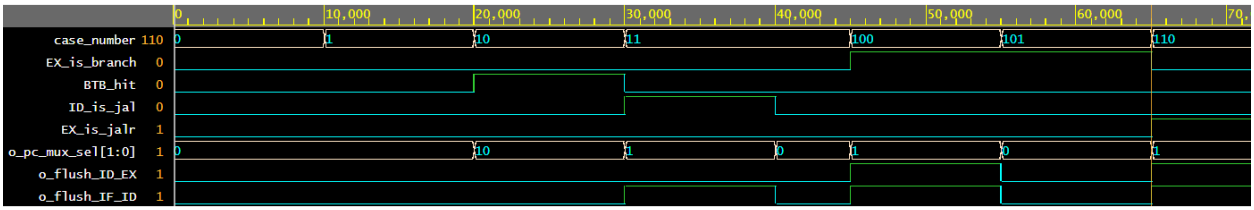
```
. `timescale 1ns / 1ps
module tb_branch_ctr_unit;

    // 1. KHAI BÁO TÍN HIỆU
    // Inputs từ EX Stage
    logic        EX_is_branch;
    logic        EX_is_jal;
    logic        EX_is_jalr;
    logic [2:0]  EX_funct3;
    logic        EX_br_equal;
    logic        EX_br_less;
    logic [31:0] EX_target_pc;
    logic [31:0] EX_pc_plus_4;
```

```
// Inputs từ ID Stage
logic [31:0] ID_pc;
logic        ID_is_jal;
logic [31:0] ID_target_pc;

// Inputs từ IF Stage
logic [31:0] IF_pc;
logic        BTB_hit;
logic        stack_valid;

// Outputs
logic [1:0]  o_pc_mux_sel;
logic [31:0] o_pc_target_out;
logic        o_flush_IF_ID;
logic        o_flush_ID_EX;
logic        o_btb_update_en;

// Biến phụ trợ để bạn dễ nhìn trên waveform (Test Case Number)
integer case_number;

// 2. KẾT NỐI MODULE (DUT)
branch_ctr_unit dut (
    .EX_is_branch   (EX_is_branch),
    .EX_is_jal      (EX_is_jal),
    .EX_is_jalr     (EX_is_jalr),
    .EX_funct3      (EX_funct3),
    .EX_br_equal    (EX_br_equal),
    .EX_br_less     (EX_br_less),
    .EX_target_pc   (EX_target_pc),
    .EX_pc_plus_4   (EX_pc_plus_4),
    .ID_pc          (ID_pc),
    .ID_is_jal      (ID_is_jal),
    .ID_target_pc   (ID_target_pc),
    .IF_pc          (IF_pc),
    .BTB_hit        (BTB_hit),
    .stack_valid    (stack_valid),
    .o_pc_mux_sel    (o_pc_mux_sel),
    .o_pc_target_out (o_pc_target_out),
    .o_flush_IF_ID   (o_flush_IF_ID),
    .o_flush_ID_EX   (o_flush_ID_EX),
    .o_btb_update_en (o_btb_update_en)
);

// 3. KHỐI TẠO SÓNG (STIMULUS)
initial begin
    // --- CẤU HÌNH DUMP FILE VCD ---
    $dumpfile("dump.vcd");
    $dumpvars;             // Dump tất cả tín hiệu trong testbench

    // --- KHỞI TẠO ---
    initialize_inputs();
    case_number = 0;
    #10;
```

```
        // ------------------------------------------------------------
        // CASE 1: Normal Sequence (Chạy tuần tự)
        // Output mong đợi: Mux = 00 (hoặc 11 tùy default), Flush = 0
        case_number = 1;
        EX_is_branch = 0;
        BTB_hit = 0;
        #10;

        // ------------------------------------------------------------
        // CASE 2: BTB Prediction Hit (Tại IF)
        // Output mong đợi: Mux = 10 (Chọn BTB)
        case_number = 2;
        BTB_hit = 1;
        #10;
        BTB_hit = 0; // Reset

        // ------------------------------------------------------------
        // CASE 3: JAL Detected at ID (Correction sớm)
        // Output mong đợi: Mux = 01 (Chọn ID Target), Flush IF_ID = 1
        case_number = 3;
        ID_is_jal = 1;
        ID_target_pc = 32'h0000_3000;
        IF_pc        = 32'h0000_1004; // IF đang sai (PC+4)
        #10;

        // Reset cho case tiếp theo
        ID_is_jal = 0;
        #5;

        // ------------------------------------------------------------
        // CASE 4: Branch Misprediction at EX (Correction muộn - Quan trọng
nhất)
        // Output mong đợi: Mux = 00 (Force EX), Flush IF_ID = 1, Flush ID_EX
= 1
        case_number = 4;
        EX_is_branch = 1;
        EX_funct3    = 3'b000;       // BEQ
        EX_br_equal  = 1;            // TAKEN
        EX_target_pc = 32'h0000_5000;
        ID_pc        = 32'h0000_1008; // ID đang chứa lệnh sai (PC+4)
        #10;

        // ------------------------------------------------------------
        // CASE 5: Branch Correct Prediction (Dự đoán đúng)
        // Output mong đợi: Flush = 0 (Chạy mượt)
        case_number = 5;
        // Giữ nguyên các tín hiệu EX branch taken như trên
        // Nhưng giả sử ID đã load đúng target (do BTB đoán đúng từ trước)
        ID_pc        = 32'h0000_5000;
        #10;

        // ------------------------------------------------------------
```

```
        // CASE 6: JALR Misprediction
        case_number = 6;
        EX_is_branch = 0;
        EX_is_jalr   = 1;
        EX_target_pc = 32'h0000_8000;
        ID_pc        = 32'h0000_100C;   // Sai -> Flush
        #10;

        // Kết thúc mô phỏng
        #20;
        $finish;
    end

    // Task reset tín hiệu
    task initialize_inputs();
        EX_is_branch = 0; EX_is_jal = 0; EX_is_jalr = 0;
        EX_funct3 = 0; EX_br_equal = 0; EX_br_less = 0;
        EX_target_pc = 0; EX_pc_plus_4 = 0;
        ID_pc = 0;
        ID_is_jal = 0; ID_target_pc = 0;
        IF_pc = 0;
        BTB_hit = 0; stack_valid = 0;
    endtask

endmodule
```

Kết quả:



Trường hợp 1: Normal Sequence (Chạy tuần tự)

- EX_is_branch = 0, BTB_hit = 0 và ID_is_jal = 0
- Output mong đợi: o_pc_mux_sel[1:0] = 00 và không có flush

Trường hợp 2: BTB Prediction Hit (Tại IF)

- EX_is_branch = 0, BTB_hit = 1 và ID_is_jal = 0
- Output mong đợi: o_pc_mux_sel[1:0] = 10 (Chọn BTB) và không có flush

Trường hợp 3: JAL Detected at ID (Correction sớm)

- EX_is_branch = 0, BTB_hit = 0 và ID_is_jal = 1

- Output mong đợi: o_pc_mux_sel[1:0] = 01 (Chọn ID Target) và Flush IF_ID = 1

Trường hợp 4: Branch Misprediction at EX (Correction muộn)

- EX_is_branch = 1, BTB_hit = 0 và ID_is_jal = 0
- Output mong đợi: o_pc_mux_sel[1:0] = 00 và Flush IF_ID = 1, Flush ID_EX = 1

Trường hợp 5: Branch Correct Prediction (Dự đoán đúng)

- Giữ nguyên các tín hiệu EX_is_branch, BTB_hit và ID_is_jal như ở trường hợp 4, Nhưng giả sử ID đã load đúng target (do BTB đoán đúng từ trước)
- Output mong đợi: Flush = 0 (chạy mượt)

Trường hợp 6: JALR Misprediction

- EX_is_branch = 0, BTB_hit = 0 và ID_is_jalr = 1
- Output mong đợi: o_pc_mux_sel[1:0] = 01 và Flush IF_ID = 1, Flush ID_EX = 1

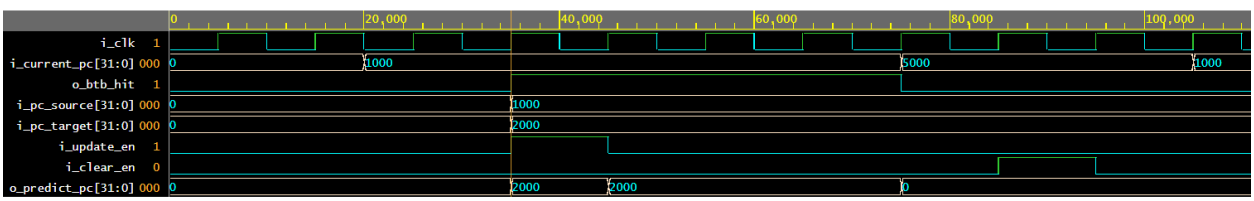
3.8. Khối BTB:

```
. `timescale 1ns / 1ps

module tb_BTB;

    // 1. Khai báo tín hiệu
    logic        i_clk;
    logic        i_rstn;

    // Phần đọc (IF Stage)
    logic [31:0] i_current_pc;
    logic [31:0] o_predict_pc;
    logic        o_btb_hit;

    // Phần cập nhật (EX Stage)
    logic        i_update_en;
    logic        i_clear_en;
    logic [31:0] i_pc_source;
    logic [31:0] i_pc_target;

    // Biến phụ trợ để debug
    integer cycle_count;

    // 2. Kết nối DUT (Device Under Test)
    BTB dut (
        .i_clk        (i_clk),
        .i_rstn       (i_rstn),
        .i_current_pc (i_current_pc),
        .o_predict_pc (o_predict_pc),
        .o_btb_hit    (o_btb_hit),
        .i_update_en  (i_update_en),
        .i_clear_en   (i_clear_en),
```

```
    .i_pc_source  (i_pc_source),
    .i_pc_target  (i_pc_target)
);

// 3. Tạo xung Clock (Chu kỳ 10ns)
initial begin
    i_clk = 0;
    forever #5 i_clk = ~i_clk;
end

// 4. Kịch bản kiểm tra (Test Scenarios)
initial begin
    // --- CẤU HÌNH DUMP WAVEFORM ---
    $dumpfile("dump_btb.vcd");
    $dumpvars;

    // --- KHỞI TẠO & RESET ---
    i_rstn = 0;
    i_current_pc = 0;
    i_update_en = 0; i_clear_en = 0;
    i_pc_source = 0; i_pc_target = 0;
    #15;
    i_rstn = 1;
    #5;

    // ------------------------------------------------------------
    // CASE 1: INITIAL MISS (Chưa học gì cả)
    // Giả sử PC = 0x1000. BTB trống trơn -> Phải MISS
    i_current_pc = 32'h0000_1000;
    #10;
    // Kỳ vọng: o_btb_hit = 0

    // ------------------------------------------------------------
    // CASE 2: UPDATE (Học lệnh nhảy)
    // Giả sử lệnh tại 0x1000 là BEQ, nhảy tới 0x2000.
    // Tại EX, ta cập nhật vào BTB.
    @(posedge i_clk);
    i_update_en = 1;
    i_pc_source = 32'h0000_1000; // Địa chỉ lệnh nhảy (Tag/Index)
    i_pc_target = 32'h0000_2000; // Đích đến
    @(posedge i_clk);
    i_update_en = 0; // Tắt ghi

    // ------------------------------------------------------------
    // CASE 3: PREDICT HIT (Đoán trúng)
    // Lần sau gặp lại 0x1000, BTB phải nhận ra ngay.
    #20;
    i_current_pc = 32'h0000_1000;
    #10;
    // Kỳ vọng: o_btb_hit = 1, o_predict_pc = 0x2000

    // ------------------------------------------------------------
    // CASE 4: CONFLICT HANDLING (Xung đột Index)
```

```
        // Giả sử có lệnh khác tại 0x5000 có cùng Index (bit [9:2]) với
0x1000
        // PC = 0x5000 -> Index giống 0x1000, nhưng Tag khác -> Phải MISS
        i_current_pc = 32'h0000_5000;
        #10;
        // Kỳ vọng: o_btb_hit = 0 (Dù valid=1 nhưng Tag mismatch)

        // ------------------------------------------------------------
        // CASE 5: CLEAR (Xóa khi dự đoán sai)
        // Giả sử lệnh tại 0x1000 lần này KHÔNG nhảy nữa (Not Taken).
        // Ta phải Invalid dòng đó đi.
        @(posedge i_clk);
        i_clear_en = 1;
        i_pc_source = 32'h0000_1000; // Xóa dòng ứng với PC này
        @(posedge i_clk);
        i_clear_en = 0;

        // Kiểm tra lại: Gặp 0x1000 lần nữa -> Phải MISS
        #10;
        i_current_pc = 32'h0000_1000;
        #10;
        // Kỳ vọng: o_btb_hit = 0

        #20;
        $finish;
    end

endmodule
```

Kết quả:



Trường hợp 1: INITIAL MISS (Chưa học gì cả)

- Giả sử PC = 0x1000. BTB trống trơn -> Phải MISS
- Kỳ vọng: o_btb_hit = 0

Trường hợp 2: UPDATE (Học lệnh nhảy)

- Giả sử lệnh tại 0x1000 là BEQ, nhảy tới 0x2000.
- Kỳ vọng: o_btb_hit = 1 và i_update_en = 1 (cho phép ghi) để cập nhật o_predict_pc[31:0] = 0x2000

Trường hợp 3: PREDICT HIT (Đoán trúng)

- Lần thứ 2 gặp lại 0x1000, BTB phải nhận ra ngay.
- Kỳ vọng: o_btb_hit = 1, i_update_en = 0 và o_predict_pc = 0x2000

Trường hợp 4: CONFLICT HANDLING (Xung đột Index)

- Giả sử có lệnh khác tại 0x5000 có cùng Index (bit [9:2]) với 0x1000. Có PC = 0x5000 -> Index giống 0x1000, nhưng Tag khác -> Phải MISS
- Kỳ vọng: o_btb_hit = 0 và o_predict_pc = 0

Trường hợp 5: CLEAR (Xóa khi dự đoán sai)

- Giả sử lệnh tại 0x1000 lần này KHÔNG nhảy nữa (Not Taken). Ta phải Invalid dòng đó đi: i_clear_en = 1
- Kiểm tra lại: Gặp 0x1000 lần nữa -> Phải MISS
- Kỳ vọng: o_btb_hit = 0 và o_predict_pc = 0

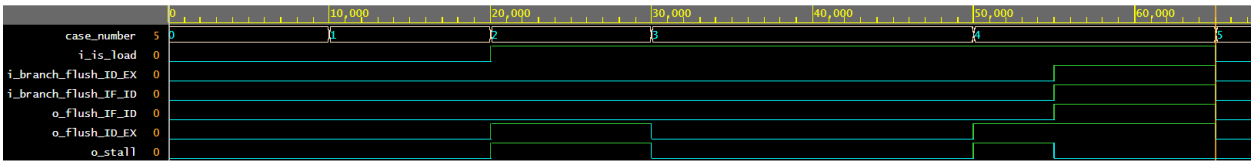
3.9. Khối Hazard Detection Unit

```
`timescale 1ns / 1ps

module tb_hazard_detection_unit;

    // 1. KHAI BÁO TÍN HIỆU
    // Inputs từ ID Stage (Lệnh hiện tại)
    logic [4:0] i_rs1_addr_ID;
    logic [4:0] i_rs2_addr_ID;
    logic       i_rs1_used_ID;
    logic       i_rs2_used_ID;

    // Inputs từ EX Stage (Lệnh đi trước 1 nhịp)
    logic [4:0] i_rd_addr_EX;
    logic       i_is_load;      // Quan trọng: Báo hiệu lệnh trước là Load

    // Inputs từ MEM Stage (Lệnh đi trước 2 nhịp - cho đủ bộ, logic chính
dùng EX)
    logic [4:0] i_rd_addr_MEM;

    // Inputs từ Branch Control (Tín hiệu ưu tiên)
    logic       i_branch_flush_ID_EX;
    logic       i_branch_flush_IF_ID;

    // Outputs
    logic       o_stall;
    logic       o_flush_IF_ID;
    logic       o_flush_ID_EX;

    // Biến phụ trợ để xem Waveform dễ hơn
    integer case_number;
```

```
// 2. KẾT NỐI MODULE (DUT)
hazard_detection_unit dut (
    .i_rs1_addr_ID        (i_rs1_addr_ID),
    .i_rs2_addr_ID        (i_rs2_addr_ID),
    .i_rs1_used_ID        (i_rs1_used_ID),
    .i_rs2_used_ID        (i_rs2_used_ID),

    .i_rd_addr_EX         (i_rd_addr_EX),
    .i_rd_addr_MEM        (i_rd_addr_MEM),
    .i_is_load            (i_is_load),

    .i_branch_flush_ID_EX (i_branch_flush_ID_EX),
    .i_branch_flush_IF_ID (i_branch_flush_IF_ID),

    .o_stall              (o_stall),
    .o_flush_IF_ID        (o_flush_IF_ID),
    .o_flush_ID_EX        (o_flush_ID_EX)
);


// 3. KỊCH BẢN KIỂM TRA (TEST CASES)
initial begin
    // --- CẤU HÌNH DUMP WAVEFORM ---
    $dumpfile("dump_hazard.vcd");
    $dumpvars;

    // --- KHỞI TẠO ---
    initialize_inputs();
    case_number = 0;
    #10;

    // ------------------------------------------------------------
    // CASE 1: NORMAL OPERATION (Không có Hazard)
    // ID dùng x1, x2. EX viết vào x3 (không trùng). Không phải Load.
    case_number = 1;
    i_rs1_addr_ID = 1; i_rs1_used_ID = 1;
    i_rs2_addr_ID = 2; i_rs2_used_ID = 1;
    i_rd_addr_EX  = 3; i_is_load = 0;
    #10;
    // Kỳ vọng: o_stall = 0, flush = 0

    // ------------------------------------------------------------
    // CASE 2: LOAD-USE HAZARD (Cần Stall)
    // Lệnh trước (EX) là LOAD ghi vào x1.
    // Lệnh hiện tại (ID) cần đọc x1.
    case_number = 2;
    i_is_load     = 1; // EX là lệnh Load
    i_rd_addr_EX  = 1; // Load ghi vào x1
    i_rs1_addr_ID = 1; // ID cần đọc x1
    #10;
    // Kỳ vọng: o_stall = 1, o_flush_ID_EX = 1
```

```
        // ------------------------------------------------------------
        // CASE 3: FALSE HAZARD (Trùng nhưng không dùng / hoặc là x0)
        // TH3a: Trùng x1 nhưng lệnh ID không dùng x1 (rs1_used = 0)
        case_number = 3;
        i_rs1_used_ID = 0;
        #10;
        // Kỳ vọng: o_stall = 0

        // TH3b: EX Load vào x0 (x0 luôn = 0, không gây hazard)
        i_rs1_used_ID = 1; // Bật lại
        i_rd_addr_EX  = 0; // Load vào x0
        i_rs1_addr_ID = 0;
        #10;
        // Kỳ vọng: o_stall = 0

        // ------------------------------------------------------------
        // CASE 4: BRANCH FLUSH PRIORITY (Quan trọng nhất)
        // Tình huống: Đang có Load-Use Hazard (như Case 2),
        // NHƯNG đồng thời có lệnh Branch bảo Flush.
        // Logic: Phải Ưu tiên Flush, hủy bỏ Stall.
        case_number = 4;
        // Setup lại Load-Use Hazard
        i_is_load     = 1;
        i_rd_addr_EX  = 5;
        i_rs1_addr_ID = 5; i_rs1_used_ID = 1;

        #5; // Lúc này Stall đang = 1

        // Bụp! Có lệnh Branch Taken yêu cầu Flush
        i_branch_flush_ID_EX = 1;
        i_branch_flush_IF_ID = 1;
        #10;
        // Kỳ vọng: o_stall về 0 (để dòng chảy tiếp tục cho việc Flush),
Output Flush = 1

        // CASE 5: CLEAN UP
        // ------------------------------------------------------------
        case_number = 5;
        initialize_inputs();
        #10;

        $finish;
    end

    // Task reset
    task initialize_inputs();
        i_rs1_addr_ID = 0; i_rs2_addr_ID = 0;
        i_rs1_used_ID = 0; i_rs2_used_ID = 0;
        i_rd_addr_EX  = 0; i_rd_addr_MEM = 0;
        i_is_load     = 0;
        i_branch_flush_ID_EX = 0;
        i_branch_flush_IF_ID = 0;
    endtask
```

```
endmodule
```



Trường hợp 1: Không có Hazard

- ID dùng x1, x2. EX viết vào x3 (không trùng). Không phải Load.
- Kỳ vọng: o_stall = 0, flush = 0

Trường hợp 2: LOAD-USE HAZARD (Cần Stall)

- Lệnh trước (EX) là LOAD ghi vào x1. Lệnh hiện tại (ID) cần đọc x1.
- Kỳ vọng: o_stall = 1, o_flush_ID_EX = 1

Trường hợp 3: FALSE HAZARD (Trùng nhưng không dùng)

- Trùng x1 nhưng lệnh ID không dùng x1 (rs1_used = 0)
- Kỳ vọng: o_stall = 0, flush = 0

Trường hợp 4: FALSE HAZARD (Load vào x0**)**

- EX Load vào x0 (x0 luôn = 0, không gây hazard)
- Kỳ vọng: o_stall = 0, flush = 0

Trường hợp 5: BRANCH FLUSH PRIORITY

- Đang có Load-Use Hazard, nhưng đồng thời có lệnh Branch bảo Flush. Theo logic: Phải Ưu tiên Flush, hủy bỏ Stall
- Kỳ vọng: o_stall từ 1 về 0 (để tiếp tục cho việc Flush), o_flush_ID_EX = 1 và o_flush_IF_ID = 1

3.10. Forwarding Control Unit:

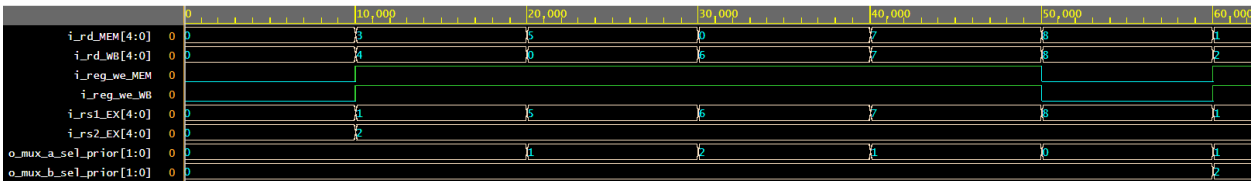
```
`timescale 1ns / 1ps

module tb_forwarding_ctr_unit;

    // 1. KHAI BÁO TÍN HIỆU

    // Input từ tầng EX (Lệnh đang cần dữ liệu)
    logic [4:0] i_rs1_EX;
    logic [4:0] i_rs2_EX;
```

```
// Input từ tầng MEM (Lệnh đi trước 1 nhịp - Nguồn Forward 1)
logic [4:0] i_rd_MEM;
logic       i_reg_we_MEM;

// Input từ tầng WB (Lệnh đi trước 2 nhịp - Nguồn Forward 2)
logic [4:0] i_rd_WB;
logic       i_reg_we_WB;

// Output điều khiển MUX
logic [1:0] o_mux_a_sel_prior; // 00: ID/EX, 01: MEM, 10: WB
logic [1:0] o_mux_b_sel_prior;

// Biến phụ trợ để xem Waveform
integer case_number;

// 2. KẾT NỐI DUT
forwarding_ctr_unit dut (
    .i_rs1_EX          (i_rs1_EX),
    .i_rs2_EX          (i_rs2_EX),

    .i_rd_MEM          (i_rd_MEM),
    .i_reg_we_MEM      (i_reg_we_MEM),

    .i_rd_WB           (i_rd_WB),
    .i_reg_we_WB       (i_reg_we_WB),

    .o_mux_a_sel_prior (o_mux_a_sel_prior),
    .o_mux_b_sel_prior (o_mux_b_sel_prior)
);

// 3. KỊCH BẢN TEST
initial begin
    // --- CẤU HÌNH DUMP FILE ---
    $dumpfile("dump_forwarding.vcd");
    $dumpvars;

    // --- KHỞI TẠO ---
    initialize_inputs();
    case_number = 0;
    #10;

    // ------------------------------------------------------------
    // CASE 1: NO FORWARDING (Chạy bình thường)
    // EX cần x1, x2. MEM ghi x3. WB ghi x4. -> Không liên quan.
    case_number = 1;
    i_rs1_EX = 1; i_rs2_EX = 2;
    i_rd_MEM = 3; i_reg_we_MEM = 1;
    i_rd_WB  = 4; i_reg_we_WB  = 1;
    #10;
    // Kỳ vọng: Mux A = 00, Mux B = 00

    // ------------------------------------------------------------
```

```
        // CASE 2: MEM HAZARD (Forwarding từ MEM - Ưu tiên 1)
        // EX cần x5. MEM đang ghi x5. WB không liên quan.
        case_number = 2;
        i_rs1_EX = 5;
        i_rd_MEM = 5; i_reg_we_MEM = 1;
        i_rd_WB  = 0;
        #10;
        // Kỳ vọng: Mux A = 01 (Chọn MEM)

        // ------------------------------------------------------------
        // CASE 3: WB HAZARD (Forwarding từ WB - Ưu tiên 2)
        // EX cần x6. MEM không có. WB đang ghi x6.
        case_number = 3;
        i_rs1_EX = 6;
        i_rd_MEM = 0;
        i_rd_WB  = 6; i_reg_we_WB = 1;
        #10;
        // Kỳ vọng: Mux A = 10 (Chọn WB)

        // ------------------------------------------------------------
        // CASE 4: DOUBLE HAZARD (Test độ ưu tiên - QUAN TRỌNG NHẤT)
        // Tình huống: Cả MEM và WB đều ghi vào cùng 1 thanh ghi (x7) mà EX
đang cần.
        // Quy tắc: Phải lấy cái mới nhất (MEM).
        case_number = 4;
        i_rs1_EX = 7;

        i_rd_MEM = 7; i_reg_we_MEM = 1; // Mới nhất (Latest)
        i_rd_WB  = 7; i_reg_we_WB  = 1; // Cũ hơn (Stale)
        #10;
        // Kỳ vọng: Mux A = 01 (Chọn MEM). Nếu ra 10 là sai logic ưu tiên!

        // ------------------------------------------------------------
        // CASE 5: WRITE ENABLE CHECK (Trùng địa chỉ nhưng không ghi)
        // Trùng x8 nhưng reg_we = 0 -> Không được forward.
        case_number = 5;
        i_rs1_EX = 8;
        i_rd_MEM = 8; i_reg_we_MEM = 0; // Tắt ghi
        i_rd_WB  = 8; i_reg_we_WB  = 0; // Tắt ghi
        #10;
        // Kỳ vọng: Mux A = 00

        // ------------------------------------------------------------
        // CASE 6: FULL COMBINATION (Forward cả A và B cùng lúc)
        // Rs1 lấy từ MEM, Rs2 lấy từ WB
        case_number = 6;
        i_rs1_EX = 1; // Cần x1
        i_rs2_EX = 2; // Cần x2

        i_rd_MEM = 1; i_reg_we_MEM = 1; // MEM cung cấp x1
        i_rd_WB  = 2; i_reg_we_WB  = 1; // WB cung cấp x2
        #10;
        // Kỳ vọng: Mux A = 01, Mux B = 10
```

```
        $finish;
    end

    // Task reset
    task initialize_inputs();
        i_rs1_EX = 0; i_rs2_EX = 0;
        i_rd_MEM = 0; i_reg_we_MEM = 0;
        i_rd_WB  = 0; i_reg_we_WB  = 0;
    endtask

endmodule
```

Kết quả:



Trường hợp 1: NO FORWARDING (Chạy bình thường)

- EX cần x1, x2. MEM ghi x3. WB ghi x4. -> Không liên quan.
- Kỳ vọng: Mux A = 00, Mux B = 00

Trường hợp 2: MEM HAZARD (Forwarding từ MEM - Ưu tiên 1)

- EX cần x5. MEM đang ghi x5. WB không liên quan.
- Kỳ vọng: Mux A = 01 (Chọn MEM)

Trường hợp 3: WB HAZARD (Forwarding từ WB - Ưu tiên 2)

- EX cần x6. MEM không có. WB đang ghi x6.
- Kỳ vọng: Mux A = 10 (Chọn WB)

Trườnghợp 4: DOUBLE HAZARD (Test độ ưu tiên)

- Cả MEM và WB đều ghi vào cùng 1 thanh ghi (x7) mà EX đang cần. Quy tắc: Phải lấy cái mới nhất (MEM).
- Kỳ vọng: Mux A = 01 (Chọn MEM). Nếu ra 10 là sai logic ưu tiên!

Trường hợp 5: WRITE ENABLE CHECK (Trùng địa chỉ nhưng không ghi)

- Trùng x8 nhưng reg_we = 0 -> Không được forward
- Kỳ vọng: Mux A = 00

Trường hợp 6: FULL COMBINATION (Forward cả A và B cùng lúc)

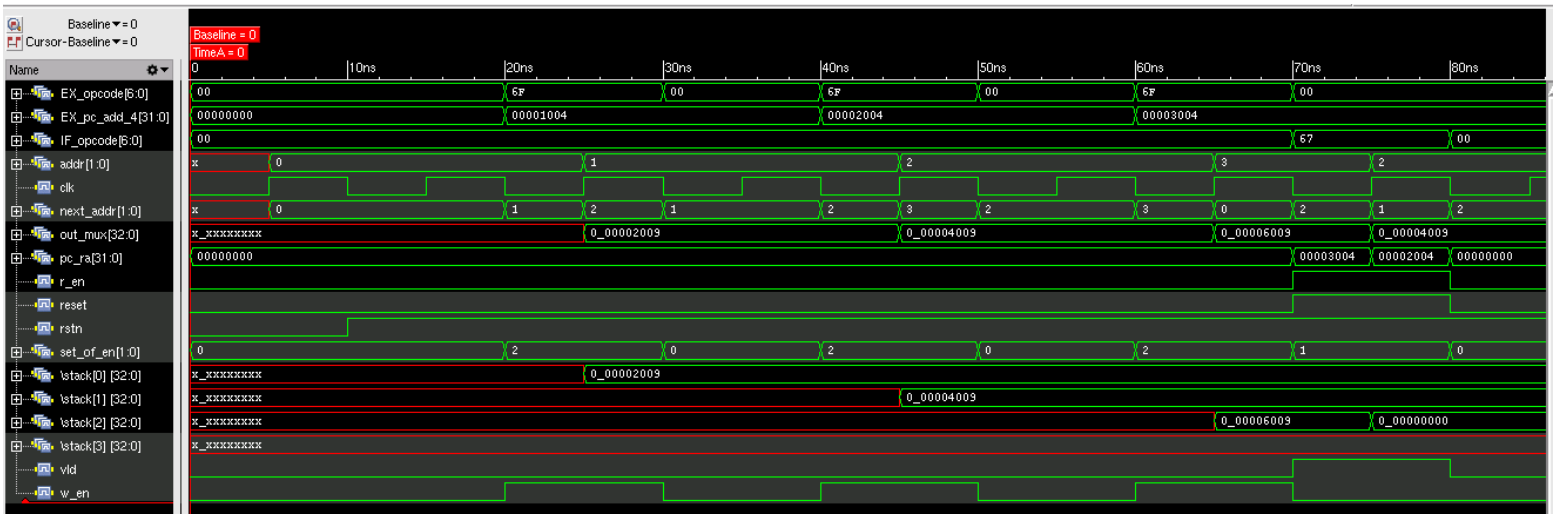
- Rs1 lấy từ MEM, Rs2 lấy từ WB
- Kỳ vọng: Mux A = 01, Mux B = 10

3.11. Khối Stack_ra_unit:

```
. module tb;

  // 1. Khai báo tín hiệu kết nối với DUT
  logic        i_clk;
  logic        i_rstn;

  // Write signals
  logic [6:0]  i_EX_opcode;
  logic [31:0] i_EX_pc_add_4;

  // Read signals
  logic [6:0]  i_IF_opcode;

  // Outputs
  logic [31:0] o_pc_ra;
  logic        o_vld;

  // 2. Khởi tạo DUT (Device Under Test)
  stack_ra_unit DUT (
    .clk         ( i_clk ),
    .rstn        ( i_rstn ),
    .EX_opcode   ( i_EX_opcode ),
    .EX_pc_add_4 ( i_EX_pc_add_4 ),
    .IF_opcode   ( i_IF_opcode ),
    .pc_ra       ( o_pc_ra ),
    .vld         ( o_vld )
  );

  // 3. Tạo xung Clock (Chu kỳ 10ns)
  initial begin
    $dumpfile("tb.vcd");
    $dumpvars(0, tb );
  end
  always #5 i_clk = ~i_clk;

  // 4. Kịch bản kiểm tra (Test Scenario)
  initial begin
    // --- Khởi tạo ban đầu ---
    i_clk         = 0;
    i_rstn        = 0; // Active low reset
    i_EX_opcode   = 0;
    i_EX_pc_add_4 = 0;
    i_IF_opcode   = 0;

    #10;
    i_rstn = 1; // Release reset
    #10;
```

```
// ============================================================
// CASE 1: Push dữ liệu vào Stack (Mô phỏng lệnh JAL)
// ============================================================
// Opcode kích hoạt Write: 7'b1100111

// Push 1: 0x1004
i_EX_opcode   = 7'b1100111;
i_EX_pc_add_4 = 32'h0000_1004;
#10;
i_EX_opcode   = 0; // Tắt ghi

// Push 2: 0x2004
#10;
i_EX_opcode   = 7'b1100111;
i_EX_pc_add_4 = 32'h0000_2004;
#10;
i_EX_opcode   = 0;

// Push 3: 0x3004
#10;
i_EX_opcode   = 7'b1100111;
i_EX_pc_add_4 = 32'h0000_3004;
#10;
i_EX_opcode   = 0;

// ============================================================
// CASE 2: Pop dữ liệu ra (Mô phỏng lệnh JALR)
// ============================================================
// Opcode kích hoạt Read: 7'b1100111

// Pop lần 1 (Kỳ vọng: 0x3004 - LIFO)
i_IF_opcode = 7'b1100111;
#10;
i_IF_opcode = 0; // Tắt đọc

// Pop lần 2 (Kỳ vọng: 0x2004)
#10;
i_IF_opcode = 7'b1100111;
#10;
i_IF_opcode = 0;

// ============================================================
// CASE 3: Tràn Stack (Wrap Around)
// ============================================================
// Nạp liên tục 5 giá trị để ghi đè vòng tròn

// Push A, B, C, D, E
i_EX_opcode = 7'b1100111;

i_EX_pc_add_4 = 32'hAAAA_AAAA; #10;
i_EX_pc_add_4 = 32'hBBBB_BBBB; #10;
i_EX_pc_add_4 = 32'hCCCC_CCCC; #10;
i_EX_pc_add_4 = 32'hDDDD_DDDD; #10;
```

```
    i_EX_pc_add_4 = 32'hEEEE_EEEE; #10; // Ghi đè lên vị trí đầu

    i_EX_opcode = 0;

    // ============================================================
    // CASE 4: Pop kiểm tra giá trị mới nhất
    // ============================================================

    #10;
    // Pop (Kỳ vọng: 0xEEEE_EEEE)
    i_IF_opcode = 7'b1100111;
    #10;
    i_IF_opcode = 0;

    $finish;
  end

endmodule
```



CASE 1: Mô phỏng gọi hàm lồng nhau

Mục đích: Kiểm tra xem mạch có ghi dữ liệu vào đúng vị trí và cập nhật con trỏ ghi hay không.

Khi gặp Jal tại EX: Tức địa chỉ cần lưu là EX_pc_add_4

- Gặp lệnh JAL (rãi bằng opcode), địa chỉ trả về là 0x1004.
- Tiếp tục gặp JAL lồng bên trong, địa chỉ 0x2004.
- Tiếp tục JAL, địa chỉ 0x3004.

Dạng sóng:

- **Lần 1:** Ghi 0x1004 vào stack[0]. Con trỏ addr tăng từ 00 lên 01.
- **Lần 2:** Ghi 0x2004 vào stack[1]. Con trỏ addr tăng từ 01 lên 10.
- **Lần 3:** Ghi 0x3004 vào stack[2]. Con trỏ addr tăng từ 10 lên 11.

CASE 2: Mô phỏng trả về từ hàm (Pop Operation)

Mục đích: Kiểm tra tính chất quan trọng nhất của Stack là **LIFO (Last In, First Out)** - Vào sau thì phải ra trước.

Trường hợp: Gặp lệnh JALR tại IF 2 lần liên tiếp.

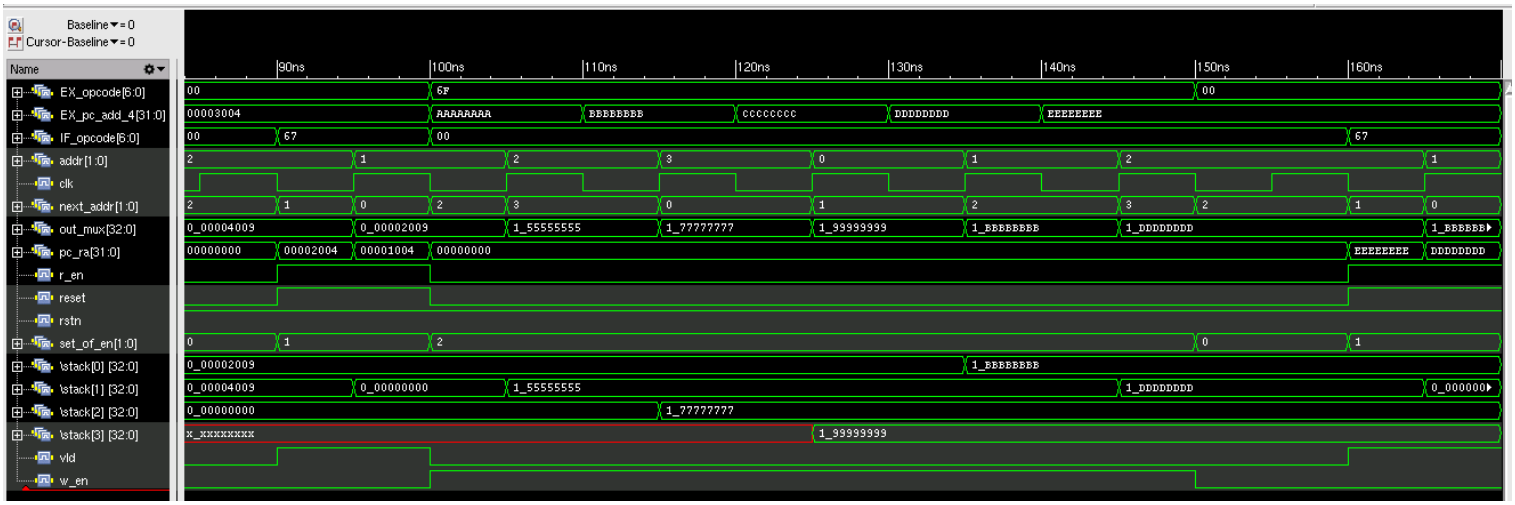
Dạng sóng:

Pop lần 1: FSM thấy addr đang là 11. Nó sẽ điều khiển MUX đọc dữ liệu tại vị trí trước đó (stack[2]).

Kết quả: Đầu ra pc_ra phải là **0x3004** (Giá trị mới nhất). Con trỏ addr lùi về 10.

Pop lần 2: FSM thấy addr là 10. MUX đọc dữ liệu tại stack[1].

Kết quả: Đầu ra pc_ra phải là **0x2004**. Con trỏ addr lùi về 01.



CASE 3: Kiểm tra tràn bộ nhớ (Stack Overflow / Wrap Around)

Mục đích: Kiểm tra cơ chế "vòng tròn". Stack phần cứng chỉ có 4 chỗ, nếu gọi hàm đệ quy quá sâu (ví dụ 5 lần), nó phải ghi đè lên giá trị cũ nhất thay vì bị treo.

Hành động: Đẩy liên tiếp 5 giá trị: A, B, C, D, E.

Diễn biến:

Push A (AAAA) vào stack[0]. addr=01.

Push B (BBBB) vào stack[1]. addr=10.

Push C (CCCC) vào stack[2]. addr=11.

Push D (DDDD) vào stack[3]. addr=00 (Quay vòng về đầu). Tức bị tràn. Do stack của khối này hoạt động theo cơ chế xoay vòng khi bị tràn.

Push E (EEEE) $\rightarrow$ Ghi vào stack[0] (Ghi đè lên A). addr=01.

CASE 4: Lấy dữ liệu sau khi tràn

Mục đích: Xác nhận rằng dù bị ghi đè vòng tròn, mạch vẫn biết đâu là dữ liệu mới nhất.

Hành động: Thực hiện Pop.
Diễn biến:
Hiện tại addr đang là 01.
Theo logic FSM: khi Pop ở trạng thái 01, nó sẽ trỏ về 00 để đọc stack[0].
Kết quả: Đầu ra o_pc_ra là **0xEEEE_EEEE**.

5. Advanced Design

6. Result

6.1. Quartus:

Thực hiện gán chân trực tiếp các pin của pipelined với các pin của DE2 thông qua file danh sánh chân được nạp trên quartus.

Model 2: Forwarding always not taken		
Maximum Frequency	39.12Mhz	39.12MHz
Logic Utilization/Total logic elements	4458	33216
Total registers	1875	33216
Model 3: Forwarding always taken		
Maximum Frequency	35.42MHz	35.42MHz
Logic Utilization/Total logic elements	5315	33216
Total registers	2388	33216

Dựa vào flow file và sta file từ Quatus:

Model 2:

```
+-----------------------------------------------------------------------------------+
; Flow Summary                                                                      ;
+-----------------------------------+-----------------------------------------------+
; Flow Status                       ; Successful - Mon Jan 05 03:41:57 2026          ;
; Quartus II 64-Bit Version         ; 13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition ;
; Revision Name                     ; pipelined                                       ;
; Top-level Entity Name             ; pipelined                                       ;
; Family                            ; Cyclone II                                      ;
; Device                            ; EP2C35F672C6                                    ;
; Timing Models                     ; Final                                           ;
; Total logic elements              ; 4,458 / 33,216 ( 13 % )                         ;
;     Total combinational functions ; 4,142 / 33,216 ( 12 % )                         ;
;     Dedicated logic registers     ; 1,875 / 33,216 ( 6 % )                          ;
; Total registers                   ; 1875                                            ;
; Total pins                        ; 220 / 475 ( 46 % )                              ;
; Total virtual pins                ; 0                                               ;
; Total memory bits                 ; 75,168 / 483,840 ( 16 % )                       ;
; Embedded Multiplier 9-bit elements ; 0 / 70 ( 0 % )                                 ;
; Total PLLs                        ; 0 / 4 ( 0 % )                                   ;
+-----------------------------------+-----------------------------------------------+

+-----------------------------------------------------+
; Slow Model Fmax Summary                             ;
+-----------+-----------------+------------+------+
; Fmax      ; Restricted Fmax ; Clock Name ; Note ;
+-----------+-----------------+------------+------+
; 39.12 MHz ; 39.12 MHz       ; CLOCK_50   ;      ;
+-----------+-----------------+------------+------+
```

Model 3:

```
+-----------------------------------------------------------------------------------+
; Fitter Summary                                                                    ;
+-----------------------------------+-----------------------------------------------+
; Fitter Status                     ; Successful - Sat Dec 13 12:51:27 2025           ;
; Quartus II 64-Bit Version         ; 13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition ;
; Revision Name                     ; pipelined                                       ;
; Top-level Entity Name             ; pipelined                                       ;
; Family                            ; Cyclone II                                      ;
; Device                            ; EP2C35F672C6                                    ;
; Timing Models                     ; Final                                           ;
; Total logic elements              ; 5,315 / 33,216 ( 16 % )                         ;
;     Total combinational functions ; 4,942 / 33,216 ( 15 % )                         ;
;     Dedicated logic registers     ; 2,388 / 33,216 ( 7 % )                          ;
; Total registers                   ; 2388                                            ;
; Total pins                        ; 220 / 475 ( 46 % )                              ;
; Total virtual pins                ; 0                                               ;
; Total memory bits                 ; 88,992 / 483,840 ( 18 % )                       ;
; Embedded Multiplier 9-bit elements ; 0 / 70 ( 0 % )                                 ;
; Total PLLs                        ; 0 / 4 ( 0 % )                                   ;
+-----------------------------------+-----------------------------------------------+
```

Nhận xét: ánh xạ trên BRAM M4K được phản ánh qua thông số “Total memory bits” là 88992/483840 (18%).

Slow Model Fmax Summary			
Fmax	Restricted Fmax	Clock Name	Note
35.42 MHz	35.42 MHz	CLOCK_50	

Code dmem giúp ánh xạ lên BRAM:

```
module dmem (
   input logic [31:0] data_a, data_b,
   input logic [13:0] addr_a, addr_b,
        input logic [3:0] be_a, be_b,
   input logic  we, clk,
   output logic [31:0] q_a, q_b
);
   byte_mem byte_mem0 (
         .clk    ( clk ),
         .data_a ( data_a [7:0] ),
         .data_b ( data_b [7:0] ),
         .we_a   ( be_a [0] & we ),
         .we_b   ( be_b [0] & we ),
         .addr_a ( addr_a ),
         .addr_b ( addr_b ),
         .q_a    ( q_a [7:0] ),
         .q_b    ( q_b [7:0] )
        );
        byte_mem byte_mem1 (
         .clk    ( clk ),
         .data_a ( data_a [15:8] ),
         .data_b ( data_b [15:8] ),
         .we_a   ( be_a [1] & we ),
         .we_b   ( be_b [1] & we),
         .addr_a ( addr_a ),
         .addr_b ( addr_b ),
         .q_a    ( q_a [15:8] ),
         .q_b    ( q_b [15:8] )
        );
   byte_mem byte_mem2 (
         .clk    ( clk ),
         .data_a ( data_a [23:16] ),
         .data_b ( data_b [23:16] ),
         .we_a   ( be_a [2] & we),
         .we_b   ( be_b [2] & we),
         .addr_a ( addr_a ),
         .addr_b ( addr_b ),
         .q_a    ( q_a [23:16] ),
```

```
        .q_b     ( q_b [23:16] )
      );
   byte_mem byte_mem3 (
        .clk     ( clk ),
        .data_a  ( data_a [31:24] ),
        .data_b  ( data_b [31:24] ),
        .we_a    ( be_a [3] & we),
        .we_b    ( be_b [3] & we),
        .addr_a  ( addr_a ),
        .addr_b  ( addr_b ),
        .q_a     ( q_a [31:24] ),
        .q_b     ( q_b [31:24] )
      );
endmodule

module byte_mem (
   input logic [7:0] data_a, data_b,
   input logic [13:0] addr_a, addr_b,
   input logic  we_a, we_b, clk,
   output logic [7:0] q_a, q_b
);
// Declare the RAM variable
   logic [7:0] ram[2047:0];
   always @ (posedge clk) begin // Port A
     if (we_a) begin
       ram[addr_a] <= data_a;
       q_a <= data_a;
     end else q_a <= ram[addr_a];
   end

   always @ (posedge clk) begin // Port b
     if (we_b) begin
       ram[addr_b] <= data_b;
       q_b <= data_b;
     end else q_b <= ram[addr_b];
   end
endmodule
```

```
+-------------------------------------------+-------------------------+
; Global signals                            ; 2                       ;
; M4Ks                                      ; 20 / 105 ( 19 % )        ;
; Total block memory bits                   ; 75,168 / 483,840 ( 16 % ) ;
; Total block memory implementation bits    ; 92,160 / 483,840 ( 19 % ) ;
; Embedded Multiplier 9-bit elements        ; 0 / 70 ( 0 % )           ;
; PLLs                                      ; 0 / 4 ( 0 % )            ;
; Global clocks                             ; 2 / 16 ( 13 % )          ;
; JTAGs                                     ; 0 / 1 ( 0 % )            ;
; ASMI blocks                               ; 0 / 1 ( 0 % )            ;
; CRC blocks                                ; 0 / 1 ( 0 % )            ;
; Average interconnect usage (total/H/V)    ; 8% / 8% / 9%             ;
; Peak interconnect usage (total/H/V)       ; 57% / 55% / 59%          ;
; Maximum fan-out                           ; 1684                      ;
; Highest non-global fan-out                ; 1684                      ;
; Total fan-out                             ; 21769                     ;
; Average fan-out                           ; 3.38                      ;
+-------------------------------------------+-------------------------+
```

Ảnh được chụp từ fit file xuất từ Quartus, ánh xạ trên 20 M4Ks

6.2. Performance

6.2.1. IPC

Benchmark	Model 1	Model 2	Model 3
Instruction	4825	4825	4825
Cycle	11116	7090	5510
IPC	0.43	0.68	0.88
Normalization	1	1	1

6.2.2. Branch Prediction

Benchmark	Model 1	Model 2	Model 3
Instruction	4825	4825	4825
Cycle	11116	7090	5510
Misprediction Rate	90.45%	90.45%	30.77%
Normalization	0%	0%	0%

Các ảnh:

Model 1: Non-forwarding

Model 2: Forwarding

```
slli.....PASS
srl......PASS
srli.....PASS
sra......PASS
srai.....PASS
lw.......PASS
lh.......PASS
lhu......PASS
lb.......PASS
lbu......PASS
sw.......PASS
sh.......PASS
sb.......PASS
auipc....PASS
lui......PASS
beq......PASS
bne......PASS
blt......PASS
bltu.....PASS
bge......PASS
bgeu.....PASS
jal......PASS
jalr.....PASS
malgn....PASS
iosw.....PASS

=================== Result ===================
Total Clock Cycles Executed = 7090
Total Instructions Executed = 4825
Total Branch Instructions   = 1602
Total Branch Mispredictions = 1449

---------------------------------------------
Instruction Per Cycle (IPC) = 0.68
Branch Misprediction Rate   = 90.45 %

END of ISA tests
```

Model 3: Forwarding Always Taken

```
lui......PASS
beq......PASS
bne......PASS
blt......PASS
bltu.....PASS
bge......PASS
bgeu.....PASS
jal......PASS
jalr.....PASS
malgn....PASS
iosw.....PASS

=================== Result ===================
Total Clock Cycles Executed = 5510
Total Instructions Executed = 4825
Total Branch Instructions   = 1602
Total Branch Mispredictions = 493

---------------------------------------------
Instruction Per Cycle (IPC) = 0.88
Branch Misprediction Rate   = 30.77 %

END of ISA tests
```